

CCO: Getting Started

Install Claude Code, authenticate, and run your first session in 30 minutes. Build the right mental model from minute one — this isn't a chatbot.

Learning Objectives

By the end of this module you will be able to:

- ☐ Explain what Claude Code is — and what it isn't — in one sentence to a teammate.
- ☐ Install Claude Code with `npm` and authenticate via Pro/Team/Enterprise account or API key.
- ☐ Open Claude Code inside a real repository and have it read, search, and edit files.
- ☐ Use the eight commands you'll reach for daily: `/help`, `/init`, `/status`, `/model`, `/clear`, `/compact`, `/agents`, `/hooks`.
- ☐ Reference files with `@`, switch into plan mode, and resume yesterday's session with `--continue`.

Before any installation: 90 seconds on what Claude Code actually is. The wrong mental model leads to frustration in the first hour, and the right one makes everything that follows click.

What is Claude Code?

Everyday Analogy

Think of the difference between a **vending machine** and a **kitchen with a chef**.

A vending machine takes a coin and dispenses one snack. You ask a question, you get an answer. ChatGPT-in-a-browser is a vending machine: each turn is isolated, the bot has no real access to your codebase, and what it produces is text you copy-paste.

A kitchen with a chef is a different situation. You hand the chef the menu (your project), they walk to the pantry (read your files), pull ingredients (open dependencies), cook (run tests), taste (check results), adjust seasoning (rerun), and serve. The chef *operates* in your kitchen; they don't just narrate recipes from across the counter. Claude Code is the kitchen-with-a-chef. It reads your files, runs your tests, calls your APIs, and writes back to disk — all under your supervision.

Technical Definition

Claude Code is a command-line orchestration framework built on top of the Anthropic Messages API. It runs in your terminal, takes a natural-language request, and executes the work using **built-in tools** (Read, Write, Edit, Bash, Glob, Grep) plus anything you wire in via **MCP servers, skills, subagents, and hooks**.

Three properties matter from minute one:

- 1. **It edits your filesystem.** Not "here's the code, paste it" — *it writes the file*. With your permission. With a diff you can review.
- 2. **It runs commands.** `npm test`, `git diff`, `docker build` — whatever your terminal can do, Claude Code can do, with your permission.
- 3. **It remembers context.** Inside a session, across sessions (with `--continue` or `--resume`), and across projects (with `CLAUDE.md` and Auto Memory). It is *not* a stateless chat.

The six things Claude Code can do that "AI in your editor" cannot

If you've used Cursor, Copilot, or Gemini Code Assist as inline-completion or chat tools, here's what shifts when you move to Claude Code:

INLINE AI VS CLAUDE CODE — CAPABILITY SURFACE

Capability	Editor AI	Claude Code
Multi-file refactor	one suggestion at a time	reads N files, edits N files, runs tests
Run shell commands	no	yes (with permission)
Read live data via MCP	limited	GitHub, Postgres, Slack, Jira, custom
Custom slash commands	no	checked into repo via <code>.claude/commands/</code> or <code>.claude/skills/</code>
Hooks & policy enforcement	no	28 lifecycle events; block / log / route
Headless / CI mode	no	<code>claude -p</code> with structured output

Why it matters

The mental shift: stop asking "can the AI write this snippet?" Start asking "can I describe this **workflow** — read these files, run these commands, validate this output — and let Claude execute it under guardrails I set?" That shift is the difference between using an autocomplete and using a junior teammate.

Now that you know what it is, let's get it running. Three commands.

Install & Authenticate

Prerequisites

- **Node.js 18 or newer** — check with `node -v`. Earlier versions won't run the CLI.
- **A terminal** — macOS Terminal/iTerm, Linux any shell, Windows Terminal or WSL2. (PowerShell works; raw `cmd.exe` is rough.)
- **An Anthropic account** — Pro / Team / Enterprise subscription or an API key from console.anthropic.com. Most learners pick the Pro plan (\$20/mo) and use OAuth login; pay-as-you-go users prefer the API key.

Step 1 — install the CLI

```
bash
```

```
# Install globally
npm install -g @anthropic-ai/claude-code

# Verify
claude --version
# → @anthropic-ai/claude-code <version>
```

If you get an `EACCES` permission error, use a Node version manager (`nvm`, `fnm`, `volta`) instead of `sudo`. Don't `sudo npm install -g` — you'll fight permissions every time you update.

Step 2 — authenticate

The first time you run `claude`, it asks how you want to log in:

```
$ claude

Select login method:
```

- > 1. Claude account with subscription (Pro / Max / Team / Enterprise)
- 2. Anthropic Console account (API usage billing)

Pick the right login method

Claude account with subscription (option 1) — opens a browser, you sign in with your Pro / Max / Team account, OAuth tokens land in `~/.claude.json`. Best for individual developers on Pro/Max plans — usage counts against your subscription, not metered API spend.

Anthropic Console / API key (option 2) — signs you into the Console (or you paste an `ANTHROPIC_API_KEY` via env var). Best for organizations with Anthropic API contracts and budget controls. Usage is metered per token; you'll want `cleanupPeriodDays` set in `settings.json` to keep transcripts trimmed.

Bedrock / Vertex (no menu — via env vars) — for shops that already buy Claude through AWS or Google Cloud. Set `CLAUDE_CODE_USE_BEDROCK=1` (with `AWS_REGION` and standard AWS credentials) or `CLAUDE_CODE_USE_VERTEX=1` (with `CLOUD_ML_REGION` and `ANTHROPIC_VERTEX_PROJECT_ID`) before launching `claude`, and the CLI routes through your cloud account instead of showing the login menu.

If you ever need to switch login modes, run `/logout` inside Claude Code and start a new session, or set `forceLoginMethod` in `settings.json` at the user level.

Step 3 — (optional) install the IDE extension

Claude Code is CLI-first, but the official **VS Code** and **JetBrains** extensions add file-tree integration, side-panel transcript, and selection forwarding. The first time you launch `claude` from inside a supported IDE's integrated terminal, the CLI offers to install the extension; you can also install it manually from the VS Code Marketplace or JetBrains Marketplace. Run `/ide` inside Claude Code to connect the current session to your editor.

What just happened?

You installed the CLI and chose how Claude authenticates. From here on, every session draws on your account — no per-session login. The next step is opening Claude Code inside a real codebase, which is where the difference from a browser chat becomes obvious.

The CLI is on your machine. Now point it at a project and watch what happens when "your AI" can actually see the code.

Your First Session

Open a terminal in any project you have writable access to. (If you don't have one handy, `git clone` one of your old repos — even an old todo-list app works.)


```
bash
```

```
cd ~/my-project  
claude
```

You'll see a welcome banner, the working directory, and a prompt. The session is now live and Claude has visibility into the directory you're in (it does *not* auto-read every file — it scans on demand using its tools).

Talk to it like a teammate, not like Google

The single biggest mistake new users make is typing *queries* instead of *asks*. Compare:

FIRST-PROMPT ANTIPATTERNS — AND THE FIX

Don't say	Why it's weak	Say instead
"How do I add auth?"	Generic Q → generic answer. No file access used.	"Look at this Express app and propose where to add JWT auth. Don't write code yet."
"Fix my tests."	Which tests? What's "fix"?	"Run npm test. For each failure, identify the root cause and patch the smallest thing that makes it pass."
"What's in this project?"	Open-ended → massive context dump.	"Read package.json and README, then summarize the architecture in 5 bullets."

Try this exact sequence

Inside a real repo, paste these prompts one at a time and watch what Claude does:

```
> What's in this project? Read package.json and README, then summarize in 5 bullets.  
  
> Run the tests. Tell me which pass, which fail, and the failure messages.  
  
> @src/index.ts — what does this file do? One paragraph.  
  
> In plan mode: how would you add a /healthz endpoint? Don't write code yet.
```

You will see Claude call tools out loud (`Read package.json` , `Bash: npm test` , etc.), pause for permission on the bash command, and stream the results back. **That visible tool ribbon is the most important UI in Claude Code.** It shows you exactly what was inspected, what was changed, and what was run.

First-time permission prompts

By default Claude Code asks before running Bash, before editing files, and before fetching URLs. You'll see prompts like "Allow this command? [y / always-this-session / always-globally / no]". Pick **"always-this-session"** for innocuous things like `npm test`; pick **"no"** for anything that looks scary. Don't blanket-approve global rules until you understand the permission system — that's CC2.

You've talked to Claude Code in a real repo. Now learn the eight commands you'll reach for daily.

The 5-Minute Tour: Eight Slash Commands That Earn Their Keep

The full slash-command list (and bundled skills) runs into the dozens. These eight pay for themselves on day one:

THE DAILY-DRIVER EIGHT

Command	What it does	When to reach for it
<code>/help</code>	List every command, including custom ones	Memory blank – "what was that command?"
<code>/init</code>	Generate a starting <code>CLAUDE.md</code> by inspecting the repo	First time in a new project
<code>/status</code>	Show active model, mode, settings sources, MCP servers	"Why is it doing X?" – check config first
<code>/model</code>	Switch model (Sonnet for daily, Opus for hard, Haiku for cheap)	A task is too slow or too dumb for current model
<code>/clear</code>	Wipe the conversation and start fresh in this session	Switching tasks – old context will pollute new ask
<code>/compact</code>	Summarize older turns to free context window	Long session, hitting token limits
<code>/agents</code>	Browse / create / edit subagents	Set up a code-reviewer or research subagent (CC4)
<code>/hooks</code>	Read-only browser of every active hook	Debug "why did that get blocked / not blocked?"

The two non-slash powers: @-references and Plan Mode

Two superpowers don't have slash commands — they're typed directly into prompts:

@ — reference a file or folder by path

Type `@` and an autocomplete picker appears. Pick a file (or type the path), and Claude will read it as if you'd handed it the source. Works for folders too — `@src/api/` tells Claude "consider this entire directory."

```
> @src/auth/login.ts — explain the token refresh flow.  
  
> Compare @src/v1/api.ts and @src/v2/api.ts. What's different?  
  
> @docs/architecture.md says we use a queue. Is the code consistent?
```

Plan Mode — "think first, code later"

Press Shift+Tab twice (cycles through modes) or pass `--permission-mode plan` at launch. In plan mode Claude reads, searches, and reasons but cannot edit or execute — it produces a written plan you approve before any change happens. Essential for risky tasks like migrations, security work, or "rewrite the auth module."

```
[plan-mode] $  
  
Plan: Migrate JWT auth from express-jwt to jose  
  
Step 1: Install jose, remove express-jwt (package.json)  
Step 2: Rewrite token verification middleware (src/middleware/auth.ts)  
Step 3: Update token issuance in login route (src/routes/login.ts)  
Step 4: Add unit tests for new middleware (tests/middleware/auth.test.ts)  
Step 5: Run npm test, verify no regressions  
  
Files modified: 4 | Risk: MEDIUM | Reversible: Yes (single PR)  
Shall I proceed? [y]es / [n]o / [e]dit plan
```

You can edit the plan, ask Claude to expand a step, or say no. Nothing on disk changes until you say yes.

Why it matters

Plan mode is the cheapest way to **shift-left bug catches**. The bugs that cost you 2 hours to undo? They get caught in the plan readout, before the first character is written. Make a habit: any task that touches more than one file, or anything you'd hesitate to commit blind → plan mode.

Eight commands and two superpowers cover the daily 80%. But what about closing your laptop — does the conversation just disappear?

Sessions Don't Disappear: --continue and --resume

Claude Code persists every session on disk. Today's session ends; tomorrow's continues. There are three patterns:

THREE WAYS TO CONTINUE YESTERDAY'S CONVERSATION		
Command	What it does	When to use
<code>claude --continue</code>	Reopens the most recent session in this project	"Pick up where I left off yesterday"
<code>claude --resume</code>	Opens a picker showing every saved session in this project	"Open the conversation about the auth refactor, not the typo fix"
<code>claude --resume <session-id></code>	Reopens a specific session by ID	Scripting / scheduled jobs / referenced in another tool

Sessions are stored at `~/.claude/projects/<project>/<session-id>.jsonl` — one JSON-lines file per session, one directory per project. They're cleaned up after `cleanupPeriodDays` (default 30).

Pro move: --continue for the most recent, --resume when you have several

The two-keystroke productivity hack:

```
$ claude --continue # yesterday's chat reopens, full context intact

> ... continue working ...

# Tomorrow, if you have several saved sessions in this project:
$ claude --resume # picker shows every saved session by first-prompt summary
```

What just happened?

You're no longer using Claude Code as a chat-and-forget tool. You're treating sessions like git branches: long-lived, resumable. Your "Tuesday afternoon refactor session" survives Monday's interruptions. This is the workflow most experienced users converge on.

You've seen the moving parts. Time to put them together in a real session.

Hands-On Lab — Boot the PublicRecords API and Map It with Claude

This lab is the foundation for every other module in the track. By the end you will have:

- A Spring Boot 3 / Java 21 REST API running locally on your machine
- Eight US UCC (Uniform Commercial Code) lien filings served as JSON
- Claude Code installed, authenticated, and reading your codebase
- A real plan for an extension produced by Claude — without writing a single line of code

Time: ~20 minutes if you already have Java and Maven; ~35 if you're installing them today. **Difficulty:** beginner — every command is explained. **Type each prompt — reading isn't the same.**

Prerequisites — install and verify

Open a terminal. On macOS: Terminal app. On Linux: any shell. On Windows: **PowerShell 7+**, NOT the old Command Prompt — heredocs (`cat <<'EOF'`) work in PowerShell 7+ but not the old shell. Install PowerShell 7 with `winget install Microsoft.PowerShell` if you don't have it.

Run each verification command below. If it fails, follow the install line directly under it.

1. Java JDK 21

You should see:

The first line must start with `21`. If it shows `17`, `11`, or `command not found`, install JDK 21:

- **macOS:** `brew install --cask temurin@21`
- **Windows:** `winget install Microsoft.OpenJDK.21` — close and reopen the terminal after install so PATH refreshes.
- **Linux (Ubuntu/Debian):** `sudo apt update && sudo apt install -y openjdk-21-jdk`

If `java -version` still shows the wrong version after install, your `JAVA_HOME` environment variable points at the old JDK. Fix:

- **macOS:** add `export JAVA_HOME=$(/usr/libexec/java_home -v 21)` to `~/.zshrc`, then `source ~/.zshrc`.
- **Windows:** open Settings → "Edit environment variables for your account" → set `JAVA_HOME` to `C:\Program Files\Microsoft\jdk-21.x.x.x-hotspot`. Reopen terminal.

2. Maven 3.9+

You should see:

Two things to check: **Apache Maven 3.9.x or higher**, and **Java version: 21.x**. If Maven reports a different Java version than your shell, fix `JAVA_HOME` as above. If Maven is missing entirely:

- macOS: `brew install maven`
- Windows: `winget install Apache.Maven`
- Linux: `sudo apt install -y maven`

3. Node.js 20+ (required to install Claude Code)

You should see: `v20.x.x` or higher (e.g. `v22.5.0`). If missing or older:

- macOS: `brew install node`
- Windows: `winget install OpenJS.NodeJS`
- Linux: follow the package-manager instructions at nodejs.org

4. Git and curl

Git: any 2.x is fine. curl: any version. Both are preinstalled on macOS and Linux. On Windows 10/11, curl ships with the OS; install Git with `winget install Git.Git` if missing.

Step 1 — Generate the project skeleton with Spring Initializr

Spring Initializr is a free web form that generates a Maven project with the dependencies you select. It's the standard way Spring teams scaffold new services — faster than hand-writing `pom.xml`.

1a. Open <https://start.spring.io> in your browser.

1b. Fill the form fields exactly as shown — match the spelling and case:

Field	Value
Project	Maven
Language	Java
Spring Boot	3.3.4 (or the latest 3.3.x release)
Group	<code>com.publicrecords</code>
Artifact	<code>publicrecords-api</code>
Name	<code>publicrecords-api</code>
Description	<code>UCC public records REST API</code>
Package name	<code>com.publicrecords.api</code>
Packaging	Jar
Java	21

1c. Click the **ADD DEPENDENCIES** button on the right side. In the search box, find and add each of these one at a time:

- **Spring Web** — gives REST controllers + an embedded Tomcat web server
- **Spring Data JPA** — database access layer (Hibernate under the hood)
- **H2 Database** — in-memory database for development and tests
- **Validation** — Bean Validation (`@Valid`, `@NotBlank`, etc.)

You should now see four chips listed under the *Dependencies* heading on the right side.

1d. Click the green **GENERATE** button at the bottom. A file `publicrecords-api.zip` downloads to your default downloads folder.

1e. Move the zip to a clean working directory and extract it. macOS / Linux:

Windows PowerShell:

You should see:

Note your project root directory. Every command in this lab assumes you're in this folder. From here on we'll abbreviate it as **the project root**.

Step 2 — Confirm the empty skeleton compiles

Before we add code, prove the skeleton is wired correctly. From the project root:

Maven downloads the Spring Boot dependencies on first run (~30 seconds on a fast connection, several minutes on slow). When done you'll see:

If you see `Source option 21 is no longer supported`, your Maven is using a JDK older than 21. Run `mvn -v`, check the *Java version* line, and fix `JAVA_HOME` as in the Prerequisites section.

Step 3 — See what Initializr generated

From the project root:

Yours will look approximately like this (the placeholder class name varies by Initializr version):

Initializr only gives you the `@SpringBootApplication` bootstrap class and a placeholder test — nothing domain-specific. Steps 4–12 add the rest. Heads up: Initializr names the class `PublicrecordsApiApplication` (lowercase 'r' in "records"), which doesn't match the standard Java convention. We'll rename it next.

Step 4 — Rename the application class to use proper CamelCase

Initializr lowercased "Records" because of how it derives class names. Standard Java convention is `PublicRecordsApiApplication` (capital R). Fix the filename first:

\$

\$

Windows PowerShell uses `Move-Item` with the same paths.

Now replace the contents of the renamed main class with the proper class name:

JAVA **SPRING BOOT BOOTSTRAP** `src/main/java/com/publicrecords/api/PublicRecordsApiApplication.java`

```
package com.publicrecords.api;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PublicRecordsApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(PublicRecordsApiApplication.class, args);
    }
}
```

And replace the test class to match:

JAVA **CONTEXT SMOKE TEST** `src/test/java/com/publicrecords/api/PublicRecordsApiApplicationTests.java`

```
package com.publicrecords.api;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class PublicRecordsApiApplicationTests {

    @Test
    void contextLoads() {}
}
```

HOW IT WORKS

`@SpringBootApplication` is a meta-annotation that bundles three things: `@Configuration` (this class declares Spring beans), `@EnableAutoConfiguration` (Spring Boot auto-configures based on what's on the classpath — H2, JPA, Web), and `@ComponentScan` (scan this package and below for `@Component` / `@Service` / `@RestController` classes). One annotation; entire framework wired up.

Run `mvn compile` to confirm the rename worked. **BUILD SUCCESS** = good.

WHAT JUST HAPPENED

Standard CamelCase class names + a one-line test that proves the Spring context can boot. The empty `contextLoads` test is more useful than it looks — if your bean wiring is broken, this test fails fast, even before any other tests run.

Step 5 — Create the Filing entity

The `Filing` class is a Java object that represents one row in the database. We're going to add it now — before this step the project has no domain code, just the Spring Boot bootstrap.

HOW IT WORKS

JPA (Jakarta Persistence API) is the Java standard for object-relational mapping — turning database rows into Java objects and back. **Hibernate** is the implementation Spring Boot uses by default.

- `@Entity` tells Hibernate "this Java class corresponds to a database row."
- `@Table(name = "filings")` sets the actual table name (otherwise it'd default to `filing`).
- `@Id` + `@GeneratedValue(IDENTITY)` — `id` is the primary key, auto-incremented by the DB.
- `@Column(name = "debtor_name")` lets us use camelCase in Java but keep snake_case in SQL — the convention in both worlds.
- `@NotBlank` + `@Size` are **Bean Validation** annotations — checked when the object is bound from an HTTP request body. Invalid input returns 400 before your code runs.

First make the package directory:

```
$
```

Now create the file at the path shown in the header below, and paste the entire contents:


```

package com.publicrecords.api.filing;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import java.time.Instant;

@Entity
@Table(name = "filings")
public class Filing {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Size(min = 2, max = 2)
    @Column(nullable = false, length = 2)
    private String state;

    @NotBlank
    @Column(name = "debtor_name", nullable = false)
    private String debtorName;

    @NotBlank
    @Column(name = "secured_party", nullable = false)
    private String securedParty;

    @Column(name = "collateral_description", length = 1000)
    private String collateralDescription;

    @Column(name = "filed_at", nullable = false)
    private Instant filedAt;

    @Column(name = "expires_at")
    private Instant expiresAt;

    public Filing() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getState() { return state; }
    public void setState(String state) { this.state = state; }
    public String getDebtorName() { return debtorName; }
    public void setDebtorName(String debtorName) { this.debtorName = debtorName; }
    public String getSecuredParty() { return securedParty; }

```

```

public void setSecuredParty(String securedParty) { this.securedParty = securedParty; }
public String getCollateralDescription() { return collateralDescription; }
public void setCollateralDescription(String collateralDescription) { this.collateralDescription = collateralDescription; }
public Instant getFiledAt() { return filedAt; }
public void setFiledAt(Instant filedAt) { this.filedAt = filedAt; }
public Instant getExpiresAt() { return expiresAt; }
public void setExpiresAt(Instant expiresAt) { this.expiresAt = expiresAt; }
}

```

WHAT JUST HAPPENED

You created the JPA entity that defines the shape of a UCC filing in your database. When the app boots, Hibernate will use this class to auto-create a `filings` table with seven columns matching your fields. The validation annotations will reject bad input at the HTTP boundary before it reaches your business logic.

Step 6 — Create the FilingRepository

HOW IT WORKS

Spring Data JPA generates the implementation of this interface for you at runtime. You only declare what queries you need; Spring figures out the SQL.

- `JpaRepository<Filing, Long>` — says "this is a repository for `Filing` entities, with `Long` primary keys." You inherit `findAll()`, `findById()`, `save()`, `delete()`, `count()`, and pagination methods for free.
- `findByState(String state)` — the method name follows Spring Data's **derived query naming**: "find by <property>." Spring translates it to `SELECT * FROM filings WHERE state = ?`. No implementation needed.

JAVA SPRING DATA REPOSITORY

src/main/java/com/publicrecords/api/filing/FilingRepository.java

```

package com.publicrecords.api.filing;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface FilingRepository extends JpaRepository<Filing, Long> {
    List<Filing> findByState(String state);
}

```

WHAT JUST HAPPENED

You declared a database access layer in 6 lines of code. At runtime Spring will scan for `JpaRepository` sub-interfaces and generate proxy classes that implement them. Your `FilingController` in the next step will inject this repository as a dependency — no `new FilingRepository()` needed.

Step 7 — Create the FilingController

The controller is the HTTP layer — the actual REST endpoints clients call.

HOW IT WORKS

- `@RestController` = `@Controller` + `@ResponseBody`. It tells Spring to serialize every method's return value to JSON automatically.
- `@RequestMapping("/filings")` sets the URL prefix; every method below is rooted under `/filings`.
- **Constructor injection** — the `FilingRepository` is a constructor parameter. Spring sees this and passes the repository proxy in at startup. No `@Autowired` needed; one constructor is enough.
- `@PathVariable` binds `{id}` in the URL to the method parameter.
- `ResponseEntity` lets us return a proper 404 when the filing isn't found, instead of throwing.

JAVA REST CONTROLLER

src/main/java/com/publicrecords/api/filing/FilingController.java

```
package com.publicrecords.api.filing;

import java.util.List;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/filings")
public class FilingController {

    private final FilingRepository repository;

    public FilingController(FilingRepository repository) {
        this.repository = repository;
    }

    @GetMapping
    public List<Filing> list(@RequestParam(required = false) String state) {
        return state == null ? repository.findAll() : repository.findByState(state);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Filing> findById(@PathVariable Long id) {
        return repository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

WHAT JUST HAPPENED

You wired up two REST endpoints: `GET /filings` (with optional `?state=XX` filter) and `GET /filings/{id}`. The controller doesn't talk to the database directly — it asks the repository, which Spring auto-implements. This separation lets us swap H2 for Postgres in CC6 without touching the controller.

Step 8 — Create the Pii utility class

A small helper that masks sensitive identifiers (SSN, EIN, account numbers) before they reach logs. CC1's CLAUDE.md will codify a rule that references this class.

HOW IT WORKS

For US public-records data, leaking an SSN to logs can be a regulatory incident. The fix is mechanical: every time you log a sensitive identifier, mask it. This helper takes the last 4 chars and prefixes `***`, so `123-45-6789` becomes `***6789`. `final class` + private constructor = nobody can instantiate or extend it; it's a pure utility.

First make the package directory:

```
$
```

JAVA PII HELPER

src/main/java/com/publicrecords/api/common/Pii.java

```
package com.publicrecords.api.common;

public final class Pii {

    private Pii() {}

    public static String mask(String value) {
        if (value == null || value.length() < 4) {
            return "***";
        }
        return "***" + value.substring(value.length() - 4);
    }
}
```

WHAT JUST HAPPENED

You added a reusable masking helper. Any code that logs a sensitive value can call `Pii.mask(ssn)` instead of logging it raw. CC1 will write the rule into `CLAUDE.md` so Claude refuses to log raw PII; CC4 builds an audit subagent that scans for violations.

Step 9 — Replace application.properties with application.yml

Initializr generated `application.properties`. We'll use YAML instead because it's nicer for nested configuration (which we'll have plenty of by CC6's MCP setup).

First delete the properties file:

```
$
```

Windows PowerShell: `Remove-Item src\main\resources\application.properties`

```
YAML SPRING CONFIG src/main/resources/application.yml

server:
  port: 8080

spring:
  datasource:
    url: jdbc:h2:mem:ucc;DB_CLOSE_DELAY=-1
    username: sa
    password: ""
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: false
  h2:
    console:
      enabled: true
      path: /h2
  sql:
    init:
      mode: always

logging:
  level:
    com.publicrecords: INFO
```

What each setting does:

- `jdbc:h2:mem:ucc` — in-memory database named "ucc" that lives only inside the JVM (no disk file).
- `ddl-auto: create-drop` — JPA creates the tables from your `@Entity` classes on startup, drops them on shutdown.
- `sql.init.mode: always` — runs `data.sql` after table creation on every boot. Without this, your seed data won't load.
- `h2.console.enabled: true` — exposes a web-based H2 admin UI at `/h2`.

WHAT JUST HAPPENED

Spring Boot's "convention over configuration" means most defaults are fine; you only override what's specific to your project. This `application.yml` tells Spring: use H2 in memory, recreate the schema on every boot, and run our seed script.

Step 10 — Create data.sql with eight UCC filings

Spring Boot automatically runs any `data.sql` on the classpath after the schema is created (because we set `sql.init.mode: always`). This is your seed data — eight realistic but fake UCC filings across six US states.

SQL SEED DATA src/main/resources/data.sql

```
INSERT INTO filings (state, debtor_name, secured_party, collateral_description, filed_at, expires_at) VALUES
('TX', 'Lone Star Holdings LLC', 'First National Bank', 'All inventory and accounts receivable', '2023-04-12T00:00:00Z', '2024-04-12T00:00:00Z'),
('TX', 'Pecos River Logistics Inc.', 'Wells Fargo Equipment Finance', '2022 Peterbilt 579 tractor units (VIN 1P3BTDG19L6000001)', '2022-08-15T00:00:00Z', '2024-08-15T00:00:00Z'),
('CA', 'Pacific Coast Wineries', 'Bank of the West', 'Specific equipment per Schedule A', '2023-09-21T00:00:00Z', '2024-09-21T00:00:00Z'),
('NY', 'Hudson Valley Foods Corp', 'Citibank N.A.', 'Inventory of perishable goods and proceeds', '2024-06-01T00:00:00Z', '2025-06-01T00:00:00Z'),
('FL', 'Coral Gables Partners LP', 'Truist Bank', 'Furniture, fixtures, and equipment', '2022-11-15T00:00:00Z', '2024-11-15T00:00:00Z'),
('IL', 'Chicago Steel Works', 'JPMorgan Chase', 'All assets per security agreement dated 2024-02-01', '2024-02-01T00:00:00Z', '2025-02-01T00:00:00Z'),
('OH', 'Buckeye Manufacturing', 'KeyBank', 'Specific equipment listed in Exhibit A', '2023-07-19T00:00:00Z', '2024-07-19T00:00:00Z'),
('PA', 'Liberty Bell Foundries', 'PNC Bank', 'Accounts receivable and proceeds', '2024-03-10T00:00:00Z', '2025-03-10T00:00:00Z'),
```

WHAT JUST HAPPENED

Eight rows of fake UCC data — safe to commit. The columns match the JPA entity's `@Column` names exactly (snake_case in SQL, camelCase in Java). When the app boots, Hibernate creates the table, then Spring runs this script and loads the rows.

Step 11 — Add the controller test

HOW IT WORKS

- `@SpringBootTest` spins up the full application context for the test — database, controllers, repository, the works. Slow but thorough.
- `@AutoConfigureMockMvc` gives us a `MockMvc` bean we can inject and use to fire HTTP requests *without* actually starting a network server.
- `jsonPath("$.length()")` asserts on the structure of the JSON response — here, that the response array has 8 entries.

First create the test package directory:

\$


```
package com.publicrecords.api.filing;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
class FilingControllerTest {

    @Autowired
    private MockMvc mvc;

    @Test
    void listsFilingsFromSeedData() throws Exception {
        mvc.perform(get("/filings"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(8));
    }

    @Test
    void filtersByState() throws Exception {
        mvc.perform(get("/filings?state=TX"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(2));
    }

    @Test
    void returnsNotFoundForUnknownId() throws Exception {
        mvc.perform(get("/filings/9999"))
            .andExpect(status().isNotFound());
    }
}
```

WHAT JUST HAPPENED

Three smoke tests that exercise every endpoint: list-all (8 rows from seed data), filter by state (2 Texas rows), and 404-on-unknown-id. These tests will run on every `mvn test` — CC7 wires them into a GitHub Action so every PR gets verified.

Step 12 — Run the tests to confirm everything works

You should see (last few lines):

If a test fails, the error usually points at one of three things: a typo in your file paths (`filing/` not `filings/`), a missing import, or a missing `data.sql` . Re-check Steps 5–11 against your files; the contents must match exactly.

Step 13 — Boot the API

You'll see the Spring Boot banner, then startup lines ending in:

The server is running. **Leave this terminal open.** Don't press Ctrl+C — that stops the server.

Boot failures — the three you'll most likely hit

"Web server failed to start. Port 8080 was already in use." — another process owns 8080. Find and kill it: macOS/Linux `lsof -i :8080` then `kill <PID>` . Windows PowerShell `Get-NetTCPConnection -LocalPort 8080` then `Stop-Process -Id <PID>` . Or change the port: edit `application.yml` and set `server.port: 8081` , then rerun. Use 8081 in every URL below.

"Source option 21 is no longer supported." — Maven is using a JDK older than 21. Run `mvn -v` and check the Java version line. Fix `JAVA_HOME` as in Prerequisites.

"Failed to determine a suitable driver class" — H2 not on the classpath. Re-check that your `pom.xml` has `com.h2database:h2` as a dependency.

Step 14 — Hit each endpoint from a second terminal

Open a **second terminal**, leaving the server running in the first. Navigate to the project root:

14a. List all filings

You should see: a JSON array of 8 filings. The first entry looks like:

If you see `[]` (empty array), the seed data didn't load — confirm `spring.sql.init.mode: always` is in `application.yml` exactly as shown in Step 9, and that `data.sql` from Step 10 exists with INSERT statements. Stop the server (Ctrl+C in the first terminal), fix, rerun.

14b. Get one filing by ID

You should see: a single JSON object (no surrounding `[]`) for the first Texas filing.

14c. Filter by state

You should see: 2 Texas filings (Lone Star Holdings + Pecos River Logistics). The URL is in quotes because `?` is special to many shells when not quoted.

14d. (Optional) Browse the H2 console

Open <http://localhost:8080/h2> in your browser. Fill in:

- JDBC URL: `jdbc:h2:mem:ucc`
- User Name: `sa`
- Password: leave blank

Click **Connect**, type `SELECT * FROM FILINGS;` in the SQL pane, click **Run**. You'll see 8 rows.

Step 15 — Initialize the project as a git repo

Subsequent modules (CC1+) will commit incrementally, so set up version control now. From the project root in your second terminal:

Create `.gitignore` at the project root:

Then make the initial commit:

Run `git status` — you should see `nothing to commit, working tree clean`. The project is now version-controlled.

Step 16 — Install Claude Code

You should see: `2.x.x` (e.g. `2.1.59`).

npm install fails with EACCES on macOS/Linux

Two fixes. Either retry with `sudo npm install -g @anthropic-ai/claude-code` — quick but uses root. Or set up a user-local npm prefix (cleaner long-term):

```
$ npm config set prefix ~/.npm-global
$ echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.zshrc
$ source ~/.zshrc
$ npm install -g @anthropic-ai/claude-code
```

Step 17 — Authenticate and launch Claude Code

From the project root (second terminal):

On first launch, Claude Code prompts you to log in. Pick the option that matches your access (Anthropic Console / Claude.ai subscription / API key) and follow the browser-based flow. After auth, you'll see the interactive UI:

The **Project** line MUST match your project root. If it doesn't, exit (Ctrl+D), `cd` to the right place, rerun `claude`. Everything below assumes Claude Code is anchored on the project root.

Step 18 — Map the codebase by asking Claude

Type each prompt at the `>` cursor and press Enter.

Prompt 1 — five-bullet tour

Watch the tool ribbon — Claude makes `Read(pom.xml)`, `Glob(src/main/java/**/*.java)`, `Read(...)` calls visibly. After ~5–10 seconds you should get a response like:

Prompt 2 — identify build and test commands

Expected answer: build = `mvn package` (or `mvn compile`); test = `mvn test`. The "don't run them" part teaches Claude to differentiate *identifying* from *executing*.

Prompt 3 — analyze a specific file via @-reference

Type `@` at the prompt — a file picker appears. Either arrow-down to navigate or paste the path:

The `@`-reference loads the file inline, so Claude sees it without a `Read` call. A good response flags one of: missing `@Valid` on inputs, no rate limiting, no audit logging, no pagination, raw `id` exposed in URLs (tenancy concern). **This kind of**

analysis is what CC1's CLAUDE.md will codify into project rules.

Step 19 — Plan a real change in plan mode (don't execute)

Plan mode is read-only — Claude can read but cannot edit or run shell commands. Press Shift+Tab until the bottom-right of the prompt shows **plan**.

Claude reads more of the codebase and outputs a structured plan. **Expected shape:**

Read the plan critically. Did Claude miss anything? Reply with corrections — the plan iterates. Things you might push back on: time-zone handling, response envelope vs raw array, behavior when N is omitted vs zero. Push back; the plan should improve.

Step 20 — Exit cleanly without keeping changes

Plan mode means nothing was written, so there's nothing to revert. Exit Claude Code by pressing Ctrl+D at the prompt (or Ctrl+C twice). Then verify the working tree is clean:

You should see:

Then stop the server — press Ctrl+C in the first terminal. You'll see `Stopped PublicRecordsApiApplication` .

Troubleshooting reference

Symptom	Likely cause	Fix
Spring Boot fails: "Port 8080 already in use"	Another process owns 8080	Kill it (<code>lsof</code> / <code>Get-NetTCPConnection</code>) or set <code>server.port: 8081</code> in application.yml
<code>java -version</code> shows 17 or 11 after install	JAVA_HOME points at the old JDK	Set JAVA_HOME to JDK 21 path; restart shell
<code>curl /filings</code> returns <code>[]</code>	data.sql didn't load	Confirm <code>spring.sql.init.mode: always</code> in application.yml; confirm data.sql exists with INSERT lines
curl returns "Connection refused"	Server not running, wrong port, or still booting	Look for "Started PublicRecordsApiApplication" in server terminal
<code>mvn test</code> fails with "package com.publicrecords... does not exist"	File in wrong package directory	Path must be <code>src/main/java/com/publicrecords/api/<package>/</code> — check for typos
<code>claude</code> command not found	npm global bin not in PATH	Run <code>echo \$PATH</code> ; add <code>\$(npm config get prefix)/bin</code> if missing
Claude Code shows wrong project path	Launched from wrong directory	Exit, <code>cd</code> to project root, relaunch

Symptom	Likely cause	Fix
Plan mode let Claude write a file	Mode toggled off mid-session	Check the bottom-right indicator; Shift+Tab to re-enter plan
<code>cat << EOF</code> heredoc fails on Windows	Using Command Prompt instead of PowerShell 7+	<code>winget install Microsoft.PowerShell</code> , then run <code>pwsh</code>

Lab complete — what you built

A real Spring Boot 3 / Java 21 REST API **you built file by file** — entity, repository, controller, helper class, YAML config, seed data, MockMvc tests — running on port 8080 and serving 8 UCC filings as JSON. Claude Code installed, authenticated, anchored on your project, successfully producing a 5-bullet codebase tour, an analysis of `FilingController.java`, and a structured plan for a new endpoint — all without writing a single line. Working tree clean and committed to git.

The next 14 modules build on this project: CC1–CC2 cover API foundations and prompt engineering, CC3 documents the UCC domain via CLAUDE.md, CC4 locks down dangerous Maven and SQL commands, CC5 generates new resources via slash commands, CC6 delegates to subagents, CC7 enforces formatting via hooks, CC8 takes a deep dive on tool use, CC9 connects MCP servers (consumer + custom), CC10 covers caching/citations/thinking, CC11 builds eval gates, CC12 wires RAG, CC13 covers workflow patterns and computer use, CC14 ships everything through GitHub Actions.

Where to Go Next

The course is linear. Each module assumes the previous. Skipping ahead works if you only need one topic, but the cross-references make CC1→CC14 the right path.

THE 14 MODULES THAT FOLLOW

Next	Module	You'll be able to...
CC1	API Essentials	Models, Messages API, system prompts, streaming, structured output
CC2	Prompt Engineering	Clarity, specificity, XML tags, few-shot, thinking triggers
CC3	CLAUDE.md & Memory	Configure Claude per repo + Auto Memory
CC4	Permissions & Sandbox	Pick the right safety mode + lock it down at OS level
CC5	Skills & Slash Commands	Build your own <code>/deploy</code> , <code>/trriage</code> , <code>/review</code>
CC6	Subagents	Delegate research / review to specialists w/ their own tools
CC7	Hooks	Auto-format, block dangerous commands, route via HTTP
CC8	Tool Use Deep Dive	The tool-use loop, schemas, fine-grained, built-in tools
CC9	MCP — Consumer & Server	Wire in GitHub/Postgres + build your own MCP server
CC10	Claude Features	Thinking, prompt caching, citations, image/PDF, code-exec
CC11	Evaluation	Test datasets, code & LLM-as-judge graders, CI gates
CC12	RAG	Chunking, embeddings, BM25, hybrid search as MCP/subagent
CC13	Workflows & Computer Use	Chaining/parallel/routing, agent loops, GUI automation
CC14	Power User & CI/CD	Two-Claude review, worktrees, GitHub Actions, headless

Quick check before you move on.

Knowledge Check

Five questions. Answer each, then click to reveal.

1. Which of the following is the strongest mental model for Claude Code?

- ☐ A An autocomplete plugin that lives inside your editor.
- ☐ B A web chat with copy-paste code suggestions.
- ☐ C A CLI that takes natural-language tasks and orchestrates real tools (read, edit, bash, MCP) under your supervision.
- ☐ D A read-only documentation assistant.

2. You're new to a 100k-line repository and want Claude Code to suggest where to add JWT auth. What's the right first prompt?

- ☐ A "How do I add JWT auth?"
- ☐ B "Just write the auth code."
- ☐ C "In plan mode: read the codebase and propose where JWT auth would fit. Don't write code yet."
- ☐ D "Show me an example JWT auth implementation."

3. You're starting a long session in a new repo. Which command should you run first?

☐ A /clear

☐ B /compact

☐ C /init — generates a starter CLAUDE.md

☐ D /hooks

4. You closed your terminal yesterday mid-task. Today you want exactly that conversation back, not a different one. What do you run?

☐ A `claude --continue`

☐ B `claude --resume` — pick the named session from the list

☐ C `claude --new`

☐ D `claude /restore`

5. You install Claude Code and the first prompt asks how to authenticate. You're on a Pro subscription, want OAuth login (no API key tracking). Which option?

☐ A Claude account with subscription — opens a browser, signs in with Pro account.

☐ B Anthropic Console / API-key billing.

☐ C AWS Bedrock provider.

☐ D Vertex AI provider.

Module Summary

You now have:

- The right mental model: Claude Code is a tool-using orchestration framework, not a chatbot.
- A working install with the auth method that matches your account (OAuth / API key / Bedrock-Vertex).
- The four ways to talk to it: direct prompts, @-references, plan mode (Shift+Tab), bash-permission flow.
- The eight commands you'll reach for daily: `/help`, `/init`, `/status`, `/model`, `/clear`, `/compact`, `/agents`, `/hooks`.
- Session persistence: `--continue` (most recent) and `--resume` (picker over every saved session in the project).

In [CC1](#) you'll meet the Messages API directly — the foundation under everything Claude Code does. Then CC2 covers prompt engineering, and CC3 returns to Claude Code itself with the 5-tier `CLAUDE.md` cascade and Auto Memory.

CC1: API Essentials for CLI Power Users

Everything Claude Code hides from you — models, the Messages API, system prompts, temperature, streaming, structured output — and why writing skills, subagents, and hooks gets a lot easier once you've seen these primitives in plain Python and TypeScript.

Learning Objectives

- ☐ Pick the right Claude model (Opus 4.7 / Sonnet 4.6 / Haiku 4.5) for a given job.
- ☐ Make a Messages API request from Python and TypeScript with proper error handling.
- ☐ Hold a multi-turn conversation by structuring the `messages` array correctly.
- ☐ Distinguish system prompts from user messages, and recognize how `CLAUDE.md` becomes one.
- ☐ Tune `temperature` for deterministic vs creative output.
- ☐ Stream responses for low-latency UX (and know when streaming hurts you).
- ☐ Coerce JSON output reliably — via prompting, prefilling, and tool use.

Claude Code is a CLI on top of the Messages API. Most "Claude Code did X weird thing" mysteries dissolve once you can reason about the underlying API call.

Why CLI Users Need the API

Everyday Analogy

Driving a car without ever opening the hood works fine until something goes wrong. The day the engine knocks, you don't need to be a mechanic, but knowing what an engine, a spark plug, and an oil pump *are* turns a panic call into a 30-second diagnosis: "smells like a misfire."

The pain without that mental model: you treat every symptom as random and only the dealership can help. Every conversation starts at zero context.

Claude Code is the same. You can ship for months using only the CLI. But the moment you write a skill that hangs, a subagent that won't follow instructions, or a hook whose JSON looks wrong — you need to know what a *model*, a *system prompt*, and a *messages array* are. This module is the engine bay.

Three places where API knowledge becomes load-bearing for CLI work:

- **Skills** ship a system-prompt fragment that gets injected when the skill is invoked. Knowing what a system prompt does — vs a user message — is the difference between a skill that steers Claude and one that's politely ignored.
- **Subagents** run with their own model selection and their own system prompt. Picking Haiku for a fast linter subagent vs Opus for a deep reviewer is a cost decision that compounds across hundreds of invocations.
- **Hooks** sometimes need to call Claude themselves (e.g. an HTTP hook that summarizes a long stack trace before deciding to block). That's a raw Messages API call — no CLI in sight.

Why it matters

Every Claude Code feature you'll use in CC3–CC14 is a thin wrapper over the Messages API. If you understand *one* API call deeply, the rest of the course feels like configuration, not magic.

The Claude Model Lineup

Anthropic's current generation is **Claude 4.x**, which comes in three sizes. Picking the right one for a given task is the single highest-leverage cost decision in any Claude-powered system.

CLAUDE 4.X — THREE SIZES, THREE USE CASES

Model	ID	When to pick it	Relative speed
Opus 4.7	<code>claude-opus-4-7</code>	Hard reasoning, multi-file refactors, deep code review, planning agents.	Slowest
Sonnet 4.6	<code>claude-sonnet-4-6</code>	Default. Production code-gen, RAG, tool use. Great quality / speed balance.	Medium
Haiku 4.5	<code>claude-haiku-4-5-20251001</code>	Classification, routing, lightweight tool calls, latency-sensitive subagents.	Fastest

Technical Definition

A **model ID** is the string you pass in the `model` field of a Messages API request. Anthropic ships dated and undated aliases. `claude-sonnet-4-6` is an undated alias that always points to the latest Sonnet 4.6 snapshot; `claude-haiku-4-5-20251001` is a pinned version that won't move under you. Pin in production; use undated for prototyping.

The 3-question decision tree

1. **Is this a deep-reasoning task** (architecture, multi-file refactor, hard debugging)? → Opus 4.7
2. **Is this latency-critical or volume-heavy** (a hook running on every save, a router subagent)? → Haiku 4.5
3. **Otherwise** → Sonnet 4.6 (this is 80% of the time)

Cost intuition

Opus is roughly 5× the price of Sonnet, which is roughly 3× the price of Haiku. A subagent that runs 1000 times a day on Opus costs ~15× what it would on Haiku — if Haiku can do the job, that's a meaningful savings without reading ten cost-optimization blog posts.

What just happened?

You learned that picking the model is a cost-vs-quality dial. For Claude Code, you'll mostly leave the CLI on its default (Opus 4.7 or Sonnet 4.6 depending on settings), but you'll explicitly set the model in subagent frontmatter (CC6) and skill metadata (CC5).

Your First Messages API Call

Before going further, get an API key from console.anthropic.com and put it in your environment:

```
export ANTHROPIC_API_KEY="sk-ant-..." # never commit this
```

Now the smallest possible request — a single user turn, asking for the fix to a typo:

Python

TypeScript

```
import os
from anthropic import Anthropic, APIError

client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

try:
    resp = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=512,
        messages=[
            {"role": "user", "content": "Fix the typo: 'recieve' → ?"}
        ],
    )
    print(resp.content[0].text)
except APIError as e:
    print(f"API error {e.status_code}: {e.message}")
```

Anatomy of the request — four required fields

- `model` — which Claude. Always required.
- `max_tokens` — cap on the response. Required — there's no implicit default. Set this to your worst-case need; you only pay for actual output tokens.
- `messages` — the conversation, an array of `{role, content}` objects. `role` is `"user"` or `"assistant"`. The first message must be `"user"`.
- `system` (optional but in 99% of real apps) — instructions about Claude's role. Top level, not part of `messages`. We'll see this below.

Anatomy of the response

Claude returns a `Message` object. The interesting fields:

```
{
  "id": "msg_01ABC...",
  "model": "claude-sonnet-4-6",
  "role": "assistant",
  "content": [
    { "type": "text", "text": "receive" }
  ],
  "stop_reason": "end_turn",
  "usage": { "input_tokens": 18, "output_tokens": 3 }
}
```

- `content` is always an **array of blocks**, not a string. Most of the time it has one `text` block; with tool use you'll see `tool_use` blocks too (CC8).
- `stop_reason` tells you *why* Claude stopped: `"end_turn"` (finished naturally), `"max_tokens"` (you ran out of budget — the response is truncated), `"tool_use"` (Claude wants to call a tool), `"stop_sequence"` (matched one of your stop sequences).
- `usage` is your billing — cache it if you're cost-sensitive.

The most common rookie bug

Treating `resp.content` as a string. It's an array. Use `resp.content[0].text` in Python, or the typed union pattern in TypeScript. You'll get a `TypeError` the first time and it'll feel like the SDK is broken — it's not.

Multi-Turn Conversations

Everyday Analogy

Imagine writing a novel where every chapter is filed in a separate folder, and the writer has amnesia. To write Chapter 4, you hand them Chapters 1, 2, and 3 plus the new prompt. They read everything, write Chapter 4, hand it back, and forget the whole conversation.

The pain without this: every "remember when I said..." gets a blank stare. Continuity has to be reconstructed from outside.

The Messages API is exactly this. Claude is stateless — **no server-side memory**. A "multi-turn conversation" is you sending the entire prior conversation back on every request. The `messages` array is the chapter folder.

To continue a conversation, you append both the previous assistant response and the new user message, then re-send the whole array:

```
history = [
    {"role": "user", "content": "What's the capital of Australia?"},
]

# Turn 1
r1 = client.messages.create(model="claude-sonnet-4-6", max_tokens=128, messages=history)
assistant_text = r1.content[0].text
history.append({"role": "assistant", "content": assistant_text})

# Turn 2 – user replies, append, re-send everything
history.append({"role": "user", "content": "And its population?"})
r2 = client.messages.create(model="claude-sonnet-4-6", max_tokens=128, messages=history)
print(r2.content[0].text) # Claude knows we're still talking about Canberra
```

Two rules that bite people

- **Roles must alternate.** Two consecutive `user` messages = 400 error. If you need to inject extra context, merge it into one user message.
- **The whole history is re-billed every turn.** A 50-turn conversation re-sends 50 turns of input every time. This is where *prompt caching* (CC10) saves you money — mark the stable prefix as cacheable.

How this maps to Claude Code

Inside a Claude Code session, the CLI maintains the messages array for you. `/compact` trims it when it gets too long. Your `CLAUDE.md` is prepended every turn as part of the system prompt — that's why it's billed every turn (and why CC1's "keep it under 200 lines" rule matters).

System Prompts — Where `CLAUDE.md` Goes

The `system` field is a top-level instruction about Claude's role, persona, or constraints. It's not part of `messages`. Claude treats it with higher priority than user content — useful for "always" rules.


```
resp = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=512,
    system=(
        "You are a senior Python reviewer. Be terse. "
        "Always reply in this format:\n"
        "VERDICT: \n"
        "REASON: "
    ),
    messages=[
        {"role": "user", "content": "def add(a,b): return a+b"}
    ],
)
print(resp.content[0].text)
# VERDICT: ship
# REASON: Trivially correct, no validation needed for two-arg sum.
```

How Claude Code uses system prompts under the hood

Every Claude Code session builds a system prompt from *multiple* sources, concatenated:

WHAT ENDS UP IN CLAUDE CODE'S SYSTEM PROMPT

Source	Where it comes from	Module
Tool descriptions	Built-in CLI tools (Bash, Edit, Read, etc.)	built-in
CLAUDE.md cascade	5 tiers (managed → user)	CC3
Active skill descriptions	Skills currently loaded for this session	CC5
Subagent system prompt	When running inside a subagent invocation	CC6
Memory index	Auto-memory entries from prior sessions	CC3

You can see the assembled system prompt with `claude --debug` — useful when Claude isn't following your CLAUDE.md and you suspect it's not loading.

Don't put dynamic data in system

System prompts are billed and (with caching) shared across requests. Putting per-request data there — the user's name, today's date — busts the cache. Put dynamic data in user messages; reserve `system` for stable role/style/policy.

Temperature — Determinism vs Creativity

Everyday Analogy

A short-order cook on a Tuesday lunch knows the burger order: same patty, same bun, same pickles. Boring? Yes. Predictable? Yes. That's *temperature 0* — pick the most likely next thing every time.

The same cook on Friday night, asked to invent a special, weighs ten ideas and picks one of the more interesting ones. Variety, but more risk of an experiment that flops. That's *temperature 1*.

Same dial, same kitchen. The only question is: do you want a reliable burger or a fun special?

Technical Definition

Temperature is a number from `0.0` to `1.0` that controls how Claude samples its next token. At `0`, Claude almost always picks the highest-probability token (very deterministic, but not bit-for-bit identical — there's still some non-determinism from inference internals). At `1.0`, it samples more broadly, producing more varied output.

Use case	Temperature	Why
Code generation, refactors	0.0–0.2	You want the same answer every run; reproducibility helps debugging.
Classification, extraction, evals (CC11)	0.0	Stability across runs is the whole point.
Writing, brainstorming	0.7–1.0	You want variety; reading the same draft twice is uninteresting.
Tool-using agents	0.0–0.3	Tool selection should be predictable; randomness here is a bug.

For Claude Code specifically

Claude Code defaults to a low temperature suitable for code work. Subagents you write should typically also use low temperature unless their *job* is creative (e.g. a "draft release notes" subagent might use 0.7). The frontmatter exposes this in CC6.

Response Streaming

By default, the SDK waits for the full response before returning. For long answers (or long tool sequences), that's a multi-second blank screen. **Streaming** sends each token as it's generated, via Server-Sent Events.

Python

TypeScript

```
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Explain async/await in 2 sentences."}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True) # tokens arrive in real time
    final = stream.get_final_message()
    print(f"\n[done in {final.usage.output_tokens} tokens]")
```

When streaming hurts

- **Batch jobs.** If nobody's watching, you don't need first-token latency — non-streaming has slightly less protocol overhead.
- **Strict JSON parsers.** You can't `json.parse` a half-streamed object. Either buffer to end, or use tool-use (which streams structured chunks you can assemble safely).
- **Hooks that decide quickly.** A hook that needs to say "block" or "allow" wants the verdict, not a token-by-token monologue.

In Claude Code

The CLI streams every assistant turn — that's why you see text appear character-by-character. Subagents stream too; the parent agent reads the stream and integrates it. **Long-running subagents look "frozen" without streaming, even when working correctly** — if you build one, default to streaming output.

Structured Data — Getting Reliable JSON

Free-form text is great for humans, terrible for code that needs to parse it. Three ways to get structured data out of Claude, in increasing order of reliability:

Option 1: Ask politely

Lowest effort, lowest reliability. Works ~90% of the time on simple shapes:

```

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=256,
    messages=[{"role": "user", "content":
        'Return JSON only: {"sentiment": "positive|negative|neutral", "score": 0..1}. '
        'Text: "I love the new keyboard."'
    }],
)
import json
data = json.loads(resp.content[0].text) # parse, hope

```

Failure mode: Claude wraps the JSON in ````json ... ```` fences or adds a leading "Here's the JSON:". You have to strip them.

Option 2: Prefill the assistant turn

You start the assistant's reply for it. Claude continues from where you left off, so it can't add a preamble:

```

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=256,
    messages=[
        {"role": "user", "content": 'Sentiment of "I love the new keyboard." Return JSON.'},
        {"role": "assistant", "content": "{}", # forces JSON-first
    },
    stop_sequences=["}], # stop right after the closing brace
)
json_text = "{" + resp.content[0].text + "}"
data = json.loads(json_text)

```

Option 3: Tool use (most reliable)

Define a tool whose `input_schema` is exactly the JSON shape you want. Claude will fill in the schema. Covered in detail in CC8 — the short version:

```

tools = [{
    "name": "record_sentiment",
    "description": "Record sentiment analysis of text.",
    "input_schema": {
        "type": "object",
        "properties": {
            "sentiment": {"type": "string", "enum": ["positive", "negative", "neutral"]},
            "score": {"type": "number", "minimum": 0, "maximum": 1},
        },
        "required": ["sentiment", "score"],
    },
}]

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=256,
    tools=tools,
    tool_choice={"type": "tool", "name": "record_sentiment"}, # force this tool
    messages=[{"role": "user", "content": "I love the new keyboard."}],
)

# resp.content has a tool_use block with .input matching the schema
tool_use = next(b for b in resp.content if b.type == "tool_use")
data = tool_use.input # already a dict, schema-validated

```

What just happened?

You saw three reliability tiers for structured output. **For production, default to tool use.** It's the only one that's schema-validated end-to-end. Prefilling is a useful fallback when the SDK doesn't support tools (rare). Free-form parsing is fine for prototypes and demos.

Hands-On Lab — Build a "ask my CLAUDE.md" CLI

You'll write a 30-line script that loads your project's `CLAUDE.md`, treats it as a system prompt, and lets you ask questions about your own conventions. This is essentially a 1% version of Claude Code — a useful sanity check that the API really is what's underneath.

Step 1 — Project setup

```
$ mkdir ask-claude-md && cd ask-claude-md
$ python -m venv .venv && source .venv/bin/activate
$ pip install anthropic
$ export ANTHROPIC_API_KEY="sk-ant-..."

# Use any CLAUDE.md you have. If you don't have one, make a starter:
$ cat > CLAUDE.md <<'EOF'
# my-saas-app
- Money is always cents (integers), never floats.
- Use Zod for API boundaries. Errors throw HttpError(status, msg).
- We use pnpm, not npm. Tests must pass before commit.
EOF
```

Step 2 — The script (ask.py)

```
import sys, pathlib
from anthropic import Anthropic, APIError

claude_md = pathlib.Path("CLAUDE.md").read_text()
question = " ".join(sys.argv[1:]) or "What conventions does this project follow?"

client = Anthropic()

system = (
    "You are answering questions about a software project. "
    "The project's CLAUDE.md file is below. Answer ONLY based on it. "
    "If the answer isn't there, say so explicitly.\n\n"
    f"<CLAUDE_MD>\n{claude_md}\n</CLAUDE_MD>"
)

try:
    with client.messages.stream(
        model="claude-sonnet-4-6", max_tokens=512, temperature=0,
        system=system,
        messages=[{"role": "user", "content": question}],
    ) as stream:
        for chunk in stream.text_stream:
            print(chunk, end="", flush=True)
        print()
except APIError as e:
    print(f"\n[error {e.status_code}] {e.message}", file=sys.stderr)
    sys.exit(1)
```

Step 3 — Try it

```
$ python ask.py "How should I represent money?"
Always as cents (integers). Never floats. (Per CLAUDE.md.)

$ python ask.py "What's our preferred CSS framework?"
The CLAUDE.md provided does not mention a CSS framework, so I can't answer.
```

Step 4 — What you just built

You wrote, in 25 lines, the conceptual core of Claude Code: **load the developer's instructions into the system prompt, then forward the user's question**. The real Claude Code adds tools (CC8), permissions (CC4), session management, and a thousand other niceties — but the foundation is exactly this.

Step 5 — Push it further (optional)

- Add multi-turn: keep a list of past messages and re-send on each invocation.
- Switch to `tool_choice` to force a JSON answer with `{found: bool, citation: str, answer: str}`.
- Try Haiku (`claude-haiku-4-5-20251001`) and time the difference; CLAUDE.md questions are easy work that benefits from the cheaper model.

Lab complete — what you should have

A working `ask.py` that streams answers about your CLAUDE.md, with proper error handling and a low temperature. You've now written, by hand, the smallest possible version of what Claude Code does — and you can extend it. CC2 builds on top: better prompts that get better answers from this exact pattern.

Knowledge Check

1. Your hook needs to classify a stack trace as "transient" vs "real bug" 50,000 times a day. Which model?

- ☐ A Opus 4.7 — classification is high-stakes.
- ☐ B Sonnet 4.6 — the safe default.
- ☐ C Haiku 4.5 — classification is bounded, volume is high, latency matters.
- ☐ D Whichever the CLI defaults to.

2. You send `messages=[{"role":"user", ...}, {"role":"user", ...}]`. What happens?

- ☐ A The two user messages are concatenated automatically.
- ☐ B 400 error — roles must alternate user/assistant.
- ☐ C Claude treats the second as a system message.
- ☐ D The first is silently dropped.

3. You're calling Claude in a CI script that diffs two JSON files. What temperature?

☐ A 0.0 — classification/comparison wants reproducibility.

☐ B 0.7 — gives Claude room to think creatively.

☐ C 1.0 — for thoroughness.

☐ D Doesn't matter; CI is non-interactive.

4. You need Claude to return `{verdict: "approve|deny", reason: string}` from a hook. Most reliable approach?

☐ A Ask politely in the user message: "Return JSON only".

☐ B Prefill the assistant with `{` and stop on `}` .

☐ C Define a tool whose `input_schema` matches and use `tool_choice` to force it.

☐ D Use streaming and parse character by character.

5. Your skill's behavior changed across a model upgrade. What's the most likely cause and the right fix?

- ☐ A You're using an undated alias (`cLaude-sonnet-4-6`). Pin to a dated snapshot in production.
- ☐ B Skills don't track model versions.
- ☐ C The temperature changed.
- ☐ D The CLAUDE.md got bigger.

Module Summary

- Three Claude 4.x models: Opus (deep reasoning), Sonnet (default), Haiku (fast & cheap). Pin dated IDs in prod.
- Messages API request: `model` , `max_tokens` , `messages` , optional `system` . Response `content` is an array of blocks, not a string.
- Multi-turn: roles alternate user/assistant; you re-send the whole history every turn (until you cache — CC10).
- System prompt = stable role/policy. CLAUDE.md, skill descriptions, and subagent prompts all become parts of it.
- Temperature 0 for code, classification, evals, tool use. 0.7–1.0 for writing/brainstorming.
- Streaming via SSE for human-facing UX; non-streaming for batch jobs and quick-verdict hooks.
- Structured data: ask < prefill < **tool use with forced** `tool_choice` (the only schema-validated path).

CC2: Prompt Engineering for Claude Code

Five techniques — clear & direct, specific, XML-structured, example-driven, thinking-triggered — applied directly to the prompts that *you* write in Claude Code: `CLAUDE.md`, skill descriptions, subagent system prompts, and slash-command bodies.

Learning Objectives

- ☐ Diagnose vague prompts and rewrite them so Claude does the right thing on the first try.
- ☐ Apply the five core techniques: clarity, specificity, XML structure, examples, thinking triggers.
- ☐ Recognize where each technique pays off most: CLAUDE.md vs skill description vs subagent prompt vs slash command.
- ☐ Quantify a prompt change with a 5-case eval (preview of CC11).

CC1 gave you the API. CC2 gives you the words that go into the API. Bad prompts waste tokens and erode trust; good prompts are short and reliably steer Claude.

Where Prompts Live in Claude Code

"Prompt engineering" sounds abstract until you realize *everything* you write in Claude Code is a prompt:

EVERY PROMPT SURFACE IN CLAUDE CODE		
Surface	What you write	Goes where
<code>CLAUDE.md</code>	Project conventions, "don't do X"	Always-on system prompt
Skill description (frontmatter)	"When to use this skill"	System prompt, when active
Skill body (SKILL.md)	Step-by-step instructions	System prompt, when invoked
Subagent system prompt	Role + format + constraints	System prompt for that subagent
Slash command body	The actual instruction Claude executes	User message, on invocation
Hook reasoning prompt	"Block? Allow? Why?"	User message, in HTTP hook

Why it matters

The same techniques that make a one-off API call work make your CLAUDE.md, skills, and subagents work. **Bad prompt engineering shows up as Claude "ignoring" your rules** — usually it's not ignoring them, the prompt was too vague to bite.

Technique 1 — Be Clear & Direct

Everyday Analogy

Tell a contractor "make the kitchen nicer" and you'll get whatever they think nice means — maybe a tile color you hate. Tell them "replace the laminate counters with quartz, leave cabinets, repaint walls Benjamin Moore Cloud White" and you get exactly that.

The pain without clarity: every conversation re-litigates what you wanted. Half the result is correction.

Same with Claude. Vague: "review this code." Direct: "Find SQL injection risks in this file. Output a numbered list. If there are none, say 'No SQL injection risks found.'" One of these gets you the same answer twice; one gets you a different essay every time.

Three rules of clarity

1. **Say what to do, not what not to do.** "Output JSON" beats "don't write prose."
2. **State the format up front.** "Reply with: VERDICT: ship|hold — one line." Claude follows the explicit shape.
3. **Eliminate ambiguity.** "Refactor this" is ambiguous. "Extract every duplicated string into a constant at the top of the file" is not.

× VAGUE

```
review this PR for issues
```

✓ CLEAR & DIRECT

```
Find security issues in this PR.  
Categories: SQLi, XSS, secrets,  
auth bypass, IDOR. Skip style.  
Output: bullet list of findings,  
each with file:line and severity  
(critical/high/medium).
```

Politeness costs tokens, not answers

"Could you please, when you have a moment..." adds noise. Claude doesn't need to be buttered up. **Direct imperatives** ("Find...", "Output...", "Skip...") are easier to follow and shorter.

Technique 2 — Be Specific

Specificity is what separates "directionally correct" output from "ready to ship." Three flavors of specificity:

1. Specific inputs

Don't make Claude guess context it doesn't have. Either provide it, or tell Claude to ask.

× MISSING CONTEXT

```
Add caching to this endpoint.
```

✓ SPECIFIC

```
Add Redis caching to GET /api/users/:id.  
Key: "user:<id>". TTL: 300s.  
Invalidate on PUT/DELETE same id.  
We use ioredis (see src/cache.ts).  
Don't cache if the user is admin.
```

2. Specific format

If your downstream code parses the output, the format must be deterministic.

× LOOSE FORMAT

```
Summarize the diff briefly.
```

✓ LOCKED FORMAT

```
Summarize the diff in this exact form:  
TITLE: <5 word summary>  
RISK: low|medium|high  
CHANGES:  
- <file>: <1-line description>  
[repeat as needed]
```

3. Specific constraints

Hard limits Claude must respect — word counts, file paths, sentinel values, allowed enums.

Constraints:

- Maximum 3 bullets.
- Each bullet starts with an action verb.
- Reference files only from src/billing/. If a finding is elsewhere, drop it.
- If no findings, output exactly the string: "NO_FINDINGS"

Why it works

Specificity narrows the space of "valid" Claude outputs. A vague prompt has thousands of plausible responses, of which most aren't what you wanted. A specific prompt has dozens, almost all of which are.

Technique 3 — Structure with XML Tags

Everyday Analogy

If a manager hands you a single page that says "Read all this and do something useful," your eyes glaze over. Hand you the same page with bold headings — *Background*, *Constraints*, *Deliverable*, *Examples* — and you can find the part you need in two seconds.

The pain without structure: every reader (or LLM) has to re-parse the soup of text on every read. Easy to misattribute — is "use uppercase" a rule or an example?

XML tags do that bolding for Claude. Anthropic explicitly trained Claude to recognize and respect `<tag>` structure in prompts. It's not magic syntax — it's a strong convention.

Technical Definition

XML-tag prompting is the practice of wrapping prompt regions in named tags — `<context>`, `<rules>`, `<example>`, `<input>` — so Claude can refer to specific regions and you can refer to those regions in instructions ("Apply `<rules>` to the content of `<input>`"). Tag names are arbitrary; the structure is what matters.

Before / after

✗ SOUP

```
Here are some rules: no PII in
logs, all amounts in cents,
return JSON only. Below is the
data. Process it: customer
john@x.com paid $42.50.
```

✓ TAGGED

```
<rules>
- No PII in logs
- All amounts in cents
- Return JSON only
</rules>

<input>
customer john@x.com paid $42.50
</input>

Apply <rules> to <input>.
```

Common tags worth standardizing on

- `<context>` — background info, codebase facts
- `<rules>` or `<constraints>` — what Claude must respect
- `<example>` — few-shot demonstrations
- `<input>` or `<data>` — the thing to operate on
- `<output_format>` — how the answer should look

In Claude Code

Look at any well-written subagent system prompt or skill body — they almost always use XML tags. It's not stylistic; it's a measurable accuracy bump on instruction-following. **Adopt this in CLAUDE.md too** for sections like `<security_rules>` that you want Claude to bring up by name.

Technique 4 — Provide Examples (Few-Shot)

For tasks where "the right shape of answer" is hard to describe in words, show, don't tell. Few-shot prompting — giving Claude 1–5 examples of input/output pairs — usually beats more rules.

The three-example sweet spot

One example is often enough, three is the sweet spot for most extraction/classification tasks, more than five usually hits diminishing returns.

```
<task>
Classify each git commit message as: feat | fix | chore | docs | refactor.
</task>

<examples>
<example>
<input>Add OAuth login endpoint</input>
<output>feat</output>
</example>
<example>
<input>Resolve null deref in pricing</input>
<output>fix</output>
</example>
<example>
<input>Bump axios to 1.7</input>
<output>chore</output>
</example>
</examples>

<input>Extract auth middleware to its own module</input>
<output>
```

Claude sees three examples that establish the shape, then continues the pattern. Output here will be `refactor` — with very high confidence and no preamble, because the examples implicitly forbade preamble.

Pick examples that cover edge cases

- One typical case

- One ambiguous case (with the disambiguating decision shown in the output)
- One that resembles a failure mode you've seen Claude make

Examples drift the system prompt

Examples are tokens too. A 5-example few-shot block in a system prompt is billed every turn. For frequently-invoked surfaces (CLAUDE.md, skill descriptions), keep examples lean — or move them into a skill body that loads only when the skill is invoked.

Technique 5 — Trigger Thinking

For hard problems, asking Claude to think step-by-step before answering measurably improves accuracy. Two ways to do it — an old prompting trick and a new built-in feature.

Old: "think step by step" in the prompt

Decide whether to merge this PR. Think step by step:

1. What changed?
2. What could break?
3. Is the test coverage adequate?
4. Final verdict: merge | hold | request-changes.

Output only the final verdict line.

The numbered scaffold lets Claude reason aloud through the steps, then produce only the final answer (because you said so).

New: Extended Thinking (Claude 4.x)

Technical Definition

Extended Thinking is a Claude 4.x feature where the model produces a hidden reasoning trace before the final answer, with a configurable token budget. You enable it via the `thinking` parameter on a Messages API call. In Claude Code, it's triggered by including phrases like *"think hard"* or *"think deeply"* in your prompt — the CLI translates that into the API parameter.

```

resp = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=4096,
    thinking={"type": "enabled", "budget_tokens": 8000},
    messages=[{"role": "user", "content":
        "Refactor src/auth.py to use dependency injection. "
        "Identify all call sites and propose a migration plan."
    }],
)
# resp.content has thinking blocks (internal reasoning) and text blocks (the final answer)

```

When to trigger thinking

Task	Thinking?
"What's the capital of France?"	No — trivial, wastes tokens.
Code review of a 200-line PR	Yes — multiple things to weigh.
Multi-file refactor planning	Yes — budget 4–8K thinking tokens.
Picking which test to run from a name	No — pattern match.

In Claude Code

The CLI maps natural-language hints to thinking budgets: "think" $\approx$ small budget, "think hard" $\approx$ medium, "think deeply / ultrathink" $\approx$ large. Use them sparingly — thinking tokens are billed at the same rate as output tokens. CC14 (Power User) covers thinking UX in detail.

Applied to CLAUDE.md, Skills & Subagents

Each surface has its own prompt-engineering sweet spot. Here's how to apply the five techniques where they pay off most:

CLAUDE.md — tight, rule-flavored, no examples

```

## Conventions
- Money in cents (integers), never floats.
- Errors: throw HttpError(status, msg). Never throw strings.
- DB access via Drizzle ORM only. No raw SQL outside src/db/migrations/.

## Don't
- Don't write to src/generated/* (codegen output).
- Don't add deps without asking; pnpm-lock.yaml is sacred.

```

Why this works: clear, specific, no XML soup (unnecessary for short rules), no examples (CLAUDE.md is billed every turn). It's a list of imperatives.

Skill body — structured, with examples

```
---
name: review-migration
description: Reviews a Drizzle migration file for safety. Triggers on .sql files in src/db/migrations/.
---

<rules>
- Flag any DROP, RENAME, or NOT NULL on existing column with data.
- Flag indexes added without CONCURRENTLY on tables > 1M rows.
- Approve idempotent DDL (CREATE IF NOT EXISTS, ALTER IF EXISTS).
</rules>

<output_format>
VERDICT: safe | risky | block
ISSUES:
- <file>:<line> – <reason>
</output_format>

<example>
<input>ALTER TABLE users ADD COLUMN email NOT NULL;</input>
<output>
VERDICT: block
ISSUES:
- migration.sql:1 – NOT NULL on existing column without default fails on rows.
</output>
</example>
```

Why this works: skill bodies load only on invocation, so you can afford XML structure + an example. Locked output format means downstream parsers are stable.

Subagent system prompt — role + format + thinking

```
---
name: deep-reviewer
model: claude-opus-4-7
description: Multi-file PR reviewer. Use for changes spanning 5+ files.
---
```

You are a senior engineer reviewing a pull request.

```
<process>
1. Read every changed file.
2. Think hard about cross-file effects.
3. Categorize each finding: bug | perf | security | style.
4. Output in <output_format>.
</process>
```

```
<output_format>
SUMMARY: <3 sentences>
FINDINGS (sorted by severity):
- [SEVERITY] file:line – description
RECOMMENDATION: merge | hold | request-changes
</output_format>
```

Be terse. Skip style nits unless they're severity high.

What just happened?

You saw the same five techniques tuned for three surfaces. The pattern: **tighter for always-on (CLAUDE.md), richer for on-demand (skills, subagents)**. The cost of richness is paid when invoked, not always.

Hands-On Lab — Rewrite a Vague Subagent

You're going to take a deliberately bad subagent prompt, measure it on 5 cases, then rewrite it using the five techniques and re-measure. This previews evaluation (CC11) without the full machinery.

Step 1 — The bad subagent

```
---  
name: commit-classifier-v0  
description: Classifies commits.  
---
```

Look at this commit and tell me what type it is.

Save as `~/claude/agents/commit-classifier-v0.md`.

Step 2 — Five test cases (golden answers)

1. "Add OAuth login endpoint"	→ feat
2. "Fix null pointer in pricing.ts"	→ fix
3. "Bump axios from 1.6 to 1.7"	→ chore
4. "Update README install steps"	→ docs
5. "Extract auth middleware to module"	→ refactor

Step 3 — Run v0, count hits

In Claude Code, invoke the subagent on each commit message (e.g. `@commit-classifier-v0 "Add OAuth login endpoint"`, or call it via the Task tool). Score: how many of the 5 outputs are exactly one of {feat, fix, chore, docs, refactor}? You'll typically see 2–3/5 — lots of "this is a feature commit because..." prose.

Step 4 — Rewrite as v1

```
---
name: commit-classifier-v1
description: Classifies a git commit message into exactly one type. Use when categorizing commits in CI or
model: claude-haiku-4-5-20251001
---

Classify a single git commit message into exactly one category.

<categories>
- feat: new user-facing functionality
- fix: bug fix
- chore: dependency bumps, tooling, no user-visible change
- docs: documentation only
- refactor: structural change, behavior preserved
</categories>

<examples>
<example><input>Add password reset flow</input><output>feat</output></example>
<example><input>Resolve race in cache eviction</input><output>fix</output></example>
<example><input>Bump TypeScript 5.4 -> 5.5</input><output>chore</output></example>
</examples>

<output_format>
Exactly one word, lowercase, from the categories list. No explanation, no punctuation.
</output_format>
```

Save as `~/.claude/agents/commit-classifier-v1.md` (sibling to v0).

Step 5 — Re-run, re-count

Run v1 on the same 5 cases. You should now see 5/5 — one-word answers, all correct. Same model would have worked, but the prompt now tells Claude exactly what shape and what categories.

Step 6 — What changed

Technique	v0	v1
Clear & direct	× "tell me what type"	✓ "Classify into exactly one category"
Specific	× types not enumerated	✓ 5 categories, defined
XML structure	×	✓ <code><categories></code> , <code><examples></code> , <code><output_format></code>
Examples	×	✓ 3 few-shots covering normal cases

Technique	v0	v1
Output format locked	×	✓ "exactly one word, no punctuation"

Lab complete — what you should have

Two subagents and a 5-case scoresheet showing the jump from v0 to v1. You've now done a tiny eval — in CC11 you'll automate this. **The lesson: prompt rewrites can produce 50%+ accuracy gains without changing models.** Almost always cheaper than upgrading to Opus.

Knowledge Check

1. Your CLAUDE.md says "review code carefully." Reviews come back inconsistent. Best fix?

- ☐ A Switch to Opus.
- ☐ B Replace "carefully" with specific categories: "Look for SQLi, XSS, secrets. Output bullet list."
- ☐ C Add "please be thorough" three times.
- ☐ D Lower temperature.

2. You want a skill body to take input and apply rules to it. Best structure?

- ☐ A Plain prose: "Apply the following rules to the data..."
- ☐ B XML tags: `<rules>...</rules>` and `<input>...</input>` with an instruction "Apply `<rules>` to `<input>`."
- ☐ C JSON: `{rules: [...], input: ...}`
- ☐ D Markdown headers: `# Rules / # Input`

3. Your subagent does classification. You give it 0 examples and a vague description. Best first improvement?

- ☐ A Add 20 examples.
- ☐ B Add 3 examples covering typical, ambiguous, and a known-failure case.
- ☐ C Add a "please be accurate" sentence.
- ☐ D Switch model from Haiku to Opus.

4. You write "think hard" in a Claude Code prompt. What does the CLI do?

- ☐ A Nothing — it's a hint to the user, not the model.
- ☐ B Translates it into a `thinking` API parameter with a medium token budget.
- ☐ C Switches the model to Opus.
- ☐ D Lowers temperature.

5. You want to add few-shot examples to teach Claude a coding style. Where should they live?

- ☐ A CLAUDE.md — so they're always loaded.
- ☐ B A skill body — loaded only when the relevant skill is invoked.
- ☐ C The user message every time.
- ☐ D An environment variable.

Module Summary

- **Be clear & direct:** imperatives, not hedges. State the format up front. Eliminate ambiguity.
- **Be specific:** specific inputs (with context), specific format (parseable), specific constraints (hard limits).
- **XML tags:** wrap regions like `<rules>`, `<input>`, `<example>`. Refer to them by name in instructions.
- **Few-shot examples:** 3 is the sweet spot — typical, ambiguous, known-failure. Lives in skill bodies, not CLAUDE.md.
- **Trigger thinking** for hard tasks. In Claude Code, "think" / "think hard" / "ultrathink" set the budget.
- **Match technique to surface:** tight rules for always-on (CLAUDE.md); rich structure + examples for on-demand (skills, subagents).
- Prompt rewrites can produce 50%+ accuracy gains — cheaper than upgrading the model.

CC3: CLAUDE.md & Memory

How Claude Code remembers things across sessions, repos, and machines — the configuration cascade, path-specific rules, the `@`-import syntax, and the `#` quick-memory shortcut.

Learning Objectives

- ☐ Identify the 5 tiers of `CLAUDE.md` and which one wins on conflict.
- ☐ Write a Minimum-Viable `CLAUDE.md` for a new repo in 5 minutes.
- ☐ Use path-specific rules to make a single instruction file fit a monorepo.
- ☐ Use the `@` import syntax inside `CLAUDE.md` and the `#` shortcut to save memories on the fly.
- ☐ Avoid the four config anti-patterns that quietly degrade Claude's behavior.

Memory in Claude Code is layered. The cascade explains why one team's `CLAUDE.md` "doesn't apply" inside another teammate's session.

The 5-Tier CLAUDE.md Cascade

Everyday Analogy

Think of how a chef inherits cooking style: the world has general culinary norms (don't cross-contaminate), the restaurant has a house style (we plate French), the kitchen station has its own rules (the grill chef preheats to 500°F), tonight's special has its own twist, and the chef has personal preferences (always finish with butter). Each layer adds detail; specific layers override general ones for the same dish.

The pain without layering: every chef on every shift gets one giant 100-page manual. Specifics about tonight's tasting menu drown in general kitchen safety rules. Updates are impossible — change one line, retrain everyone.

Same for Claude Code: machine-wide instructions (e.g. "always show diffs") shouldn't have to live in every repo. Repo-specific instructions ("this codebase uses pnpm, not npm") shouldn't leak to other projects. The cascade gives you those layers.

Claude Code reads up to **five** instruction sources at session start, then merges them in this priority order:

CLAUDE.MD SOURCES — HIGHEST PRIORITY FIRST

Tier	Path	What goes here
1. Managed	<code>managed-settings</code> dir / OS policy	Org-mandated rules: PII redaction, license gates — admins only.
2. Local	<code>.claude/CLAUDE.local.md</code>	Gitignored. "Skip auth check on my dev box."
3. Project	<code>./CLAUDE.md</code> or <code>.claude/CLAUDE.md</code>	Architecture decisions, conventions, run commands. Checked into git.
4. Directory	<code>src/payments/CLAUDE.md</code> (any subdir)	Tighter rules for sensitive areas: "no Bash here, all changes need tests."
5. User	<code>~/.claude/CLAUDE.md</code>	Your personal style: "always show diffs," "prefer functional style."

Higher tiers *add to* lower tiers. Conflicts on the same instruction are resolved by tier 1 (Managed) winning, then 2, 3, 4, 5. **Directory-level files trump root-project files** when Claude is working inside that directory — so `src/payments/CLAUDE.md` overrides anything more permissive in `./CLAUDE.md` when you're editing payment code.

Why it matters

This is the answer to "why does Claude follow rule X on my machine but not my teammate's?". Answer: that rule lives in your **user** CLAUDE.md (tier 5), not in the project file. Move it to `./CLAUDE.md` and check it in, or accept it's a personal preference.

The Minimum-Viable CLAUDE.md

Forget exhaustive specs. The smallest CLAUDE.md that actually changes Claude's behavior covers **five short sections**:

```

# CLAUDE.md — my-saas-app

## What this project is
Multi-tenant SaaS for invoice processing. Node 20 backend + React frontend + Postgres.

## How to run things
- `pnpm install` — install deps (we use pnpm, not npm)
- `pnpm dev` — start backend (port 3001) and frontend (5173) together
- `pnpm test` — vitest, must pass before any commit
- `pnpm typecheck` — tsc --noEmit, must pass before any commit

## Conventions
- All money in cents (integers), never floats
- Use Zod schemas for every API boundary (see `src/schemas/`)
- Errors: `throw new HttpError(400, 'reason')`, never `throw 'string'`
- Database access via Drizzle ORM only — no raw SQL except in `src/db/migrations/`

## Don't do these
- DO NOT write to `src/generated/*` — that's codegen output
- DO NOT add new dependencies without asking; `pnpm-lock.yaml` is sacred
- DO NOT commit without running `pnpm typecheck && pnpm test` first

## On commits
- Conventional commits: `feat:`, `fix:`, `chore:`, `refactor:`
- Reference Linear ticket: `feat(billing): add proration [ENG-123]`

```

Five sections. Twenty-five lines. **This file changes Claude's behavior more than any clever prompt.** Notice what it does NOT include: prose about "you are a helpful assistant," step-by-step tutorials, or content that's already in the README.

Keep it under 200 lines

CLAUDE.md content lives in every prompt's context, every turn. A 1,000-line CLAUDE.md eats your context budget and slows responses. Move long-form content into **skills** (CC5) — those load only when invoked.

Path-Specific Rules — One File, Many Rules

Monorepos and large codebases need different rules in different folders. Two ways to solve it:

Option A: Frontmatter gLobs — sectioned in one file

A single root **CLAUDE.md** can use frontmatter to scope sections. The instructions only load when Claude is working with matching files:

```

---
globs:
  - "src/api/**"
---

# API conventions
- Every endpoint returns `{ok: true, data}` or `{ok: false, error}`
- All inputs validated with Zod
- Add OpenAPI annotations in `src/api/openapi.ts`

```

Option B: Per-directory CLAUDE.md — the cleaner monorepo pattern

Drop a **CLAUDE.md** in any subdirectory. Claude Code automatically traverses up from the working file and merges all CLAUDE.md it finds:

```

my-monorepo/
  CLAUDE.md           # global conventions
  packages/
    api/
      CLAUDE.md       # API: Fastify + Zod
      src/
        billing/
          CLAUDE.md    # billing: extra strict, all changes need tests
    web/
      CLAUDE.md        # web: React + Tailwind

```

When Claude edits a file in **packages/api/src/billing/**, it gets the union of all three CLAUDE.md files in scope. Sensitive areas can tighten the rules without making the root file longer.

What just happened?

You learned the two scoping patterns. **Option A** works when most rules are global with a few exceptions. **Option B** works when sub-projects have substantially different conventions. Mix them freely.

/init & AGENTS.md Compatibility

Don't write CLAUDE.md from scratch. Run **/init** in any repo and Claude inspects **package.json**, **README.md**, the directory tree, and the existing tooling, then proposes a starter CLAUDE.md you can edit.

```
$ claude
> /init

[Claude reads package.json, README, src/]
[Claude proposes a draft CLAUDE.md]
[You review, edit, accept]
→ CLAUDE.md written to current directory
```

If a CLAUDE.md already exists, `/init` will offer to update or refresh it rather than overwrite blindly — review the proposed diff before accepting.

AGENTS.md — the cross-tool standard

`AGENTS.md` is an emerging cross-tool convention used by other AI coding agents. If your repo already has an AGENTS.md, you can reference it from CLAUDE.md using the `@` import syntax (covered next) instead of duplicating content.

Keeping CLAUDE.md tidy

Subdirectory CLAUDE.md files load when Claude reads files in that directory. If a vendored package or dependency contains its own CLAUDE.md, prefer not to read into those directories — or add the directory to your `.gitignore` /ignore patterns so Claude doesn't traverse into it during normal work.

Editing Memory: `/memory`, `#`, and `@-Imports`

You don't have to hand-edit CLAUDE.md files in a separate editor. Claude Code gives you three first-class ways to read, write, and compose memory from inside a session.

1. `/memory` — open active CLAUDE.md files for editing

Run `/memory` at any prompt and Claude Code shows you which CLAUDE.md files are currently loaded for this session (project, user, any subdirectory files in scope) and lets you pick one to edit. The change applies on the next turn — no restart needed.

```
> /memory
Memory files in scope:
  1. ./CLAUDE.md (project)
  2. src/main/java/com/publicrecords/api/filing/CLAUDE.md (directory)
  3. ~/.claude/CLAUDE.md (user)
Pick a file to edit:
```

2. # at message-start — save a memory on the fly

Begin a message with `#` and Claude Code treats it as a memory write. It asks *which* CLAUDE.md to append to (project, user, or a subdirectory file) and stores the line. Use it when you've just learned something during a session and want it remembered next time.

```
# In this project, all dates are stored as UTC ISO-8601 strings, never local time.
```

```
Save to:
```

1. Project (./CLAUDE.md)
2. User (~/.claude/CLAUDE.md)
3. New directory file...

```
> 1
```

```
Saved.
```

3. @path/to/file.md — import another file into CLAUDE.md

Inside any CLAUDE.md, a line beginning with `@` imports the contents of another markdown file at session start. Imports are recursive up to **5 hops deep**. Relative paths resolve relative to the CLAUDE.md that contains the import; `~/` resolves to the user home.

```
# Project conventions
```

```
@docs/coding-style.md
```

```
@docs/api-contract.md
```

```
@~/.claude/personal-style.md
```

```
## Things specific to this repo
```

- All money in cents
- ...

Use imports to keep CLAUDE.md short while still pulling in larger reference docs (style guides, glossaries, architectural decision records) only when needed by the project.

Memory files are markdown, not JSON

CLAUDE.md and any imported memory files are plain markdown — bullet lists, headings, prose. Claude reads them as instructions, not as configuration. If you want machine-parsed config (allowed tools, hooks, model), that lives in `settings.json` — covered in CC4 and CC7.

What just happened?

You now have three ways to manage memory without leaving the session: `/memory` to edit, `#` to append on the fly, and `@` to compose multiple files into one effective CLAUDE.md. The combination keeps the project file lean while letting personal preferences and reference docs flow in via imports.

Four CLAUDE.md Anti-Patterns That Quietly Hurt

WHAT NOT TO PUT IN CLAUDE.MD

Anti-pattern	Why it hurts	Do this instead
Restating your README	Doubles context cost for zero new info	Reference: "see README for setup"
Long step-by-step tutorials	Loaded every turn, drowns the real rules	Move to a skill (CC5); loaded only when invoked
Personality/tone instructions	"You are a helpful assistant" is noise; Claude is already helpful	Just write rules and conventions
Secrets / API keys	Repo-checked CLAUDE.md ends up in commits and on screens	Use <code>.env</code> , mention <i>which</i> env var, never the value

CLAUDE.md tells Claude what conventions to follow. Time to wire up a real repo.

Hands-On Lab — Document the UCC Domain so Claude Stops Guessing

Working in the **PublicRecords API** from CC0. You'll write three CLAUDE.md files — project root, the `filing/` package, and your user-level — then prove the cascade works and that the security rules actually steer Claude's behavior. About 12 minutes.

Step 1 — Open Claude Code and run `/init`

```
$ cd /path/to/PublicRecordsAPI
$ claude
> /init
```

Claude reads `pom.xml`, `README.md`, and the source tree, then proposes a draft CLAUDE.md. **Read it carefully.** Drafts get ~70% right; the missing 30% is the domain knowledge Claude can't infer from code — what UCC means, what data is

sensitive.

Step 2 — Replace the draft with the project-specific shape

HOW IT WORKS

Project-level `CLAUDE.md` at the repo root is loaded into Claude's context every session. Anything you put here becomes a standing instruction Claude follows for this project. Claude reads **nearest** first — a deeper-directory `CLAUDE.md` (Step 3) overrides the root for files in that subtree. The shape below is the minimum viable: glossary (so Claude understands your domain vocabulary), build commands, conventions, hard security rules.

Create this file at the project root, replacing the `/init` draft:

MARKDOWN

PROJECT MEMORY

CLAUDE.md

PublicRecords API

A Spring Boot 3 REST service exposing US UCC (Uniform Commercial Code) lien filings. H2 in-memory database in dev; Postgres in prod.

Domain Glossary

- **UCC-1 filing**: a public lien notice filed at a state secretary-of-state to perfect a security interest in collateral.
- **Debtor**: party whose assets are encumbered (typically a borrowing business).
- **Secured Party**: lender holding the lien (a bank, equipment finance co.).
- **Collateral**: assets covered by the filing — specific (a VIN) or blanket ("all inventory and accounts receivable").

Build & Test

- `mvn spring-boot:run` — start dev server on `:8080`
- `mvn test` — run the test suite
- `mvn spotless:apply` — auto-format Java sources
- H2 console at `http://localhost:8080/h2` (JDBC `jdbc:h2:mem:ucc`, user `sa`)

Conventions

- All `@RestController` endpoints accepting input MUST use `@Valid`.
- Repository methods follow Spring Data naming: `findByState`, not `getByState`.
- New entities go in their own package: `com.publicrecords.api.<name>`.

Security Rules — non-negotiable

- NEVER log raw SSN, EIN, or DOB. Use `Pii.mask()` from `common/Pii.java`.
- NEVER commit `application-prod.yml` or any file with real PII.
- NEVER hardcode database credentials — reference env vars by name only.

Don't

- Don't replace H2 with a different in-memory DB (tests assume H2).
- Don't add Lombok — the team prefers explicit getters/setters for entities.

Step 3 — Add a directory-level rule for the `filing/` package

First make sure the package directory exists:

```
$
```

Then create a `CLAUDE.md` inside that package — rules here apply ONLY when Claude is working on files under `filing/`:

```
MARKDOWN DIRECTORY MEMORY src/main/java/com/publicrecords/api/filing/CLAUDE.md

# filing/ — tighter rules

## Required
- Every public method on FilingController gets a 1-line JavaDoc.
- New endpoints under /filings MUST have a MockMvc test in
  src/test/java/com/publicrecords/api/filing/.

## Don't
- DO NOT expose Filing.id in URLs as a raw long once we add tenancy
  (issue #42). Use a publicId UUID instead.
```

Step 4 — Verify the cascade

```
> What conventions apply when I edit
  src/main/java/com/publicrecords/api/filing/FilingController.java?
  List them by source file.
```

Claude should cite both the root `CLAUDE.md` (UCC glossary, security rules) and `filing/CLAUDE.md` (JavaDoc + MockMvc requirement). If it only cites the root, the directory file isn't being picked up — check the path.

Step 5 — Prove the security rule actually steers Claude

```
> Add a debug log statement at the top of FilingController.findById
  that logs the debtor name and any debtor SSN we have on file.
  Don't worry about formatting.
```

Expected: Claude refuses or warns, citing your security rule, and either uses `Pii.mask()` or asks if SSN should be omitted entirely. If Claude logs an unmasked SSN anyway, your `CLAUDE.md` isn't being read — verify with `/memory` that it appears under *"Project memory."*

Step 6 — Add a personal preference to user-level CLAUDE.md

User-level memory at `~/.claude/CLAUDE.md` applies to every project, on every machine where you log in. Use it for personal preferences (formatting, output style) — not project-specific rules.

```
$
```

MARKDOWN

USER MEMORY

~/.claude/CLAUDE.md

```
# My personal preferences (apply to every project)

- Always show me a diff before applying file edits
- When showing test output, show only the failures, not all passes
- Prefer constructor injection over @Autowired field injection in Java
```

Restart Claude Code and ask: *"What's in my user-level CLAUDE.md?"* — the three rules should appear.

Step 7 — Use the # shortcut to save a memory mid-session

```
# State codes in this project are always 2-letter ISO codes
(TX, CA, NY) — never full state names.
```

When prompted, save to **Project** (`./CLAUDE.md`). Then run `/memory` — the line should be visible in the project file. Exit (Ctrl+D), restart, and ask: *"How are state codes represented?"* Claude should answer "2-letter ISO" because the rule is now part of the always-loaded project memory.

Step 8 — Commit (without secrets)

```
$ git add CLAUDE.md src/main/java/com/publicrecords/api/filing/CLAUDE.md
$ git commit -m "docs(claude): UCC domain glossary + security rules"
```

Confirm `git diff --cached` shows zero env var values, zero connection strings, zero hardcoded secrets — just rules and references.

Lab complete — what you should have

Three CLAUDE.md files: project root (UCC glossary + security rules), `filing/` package (tighter conventions), and user-level (personal preferences). Plus a memory line saved live with `#` that survives session restarts. Most importantly: a Claude that refuses to log unmasked PII because **the project told it not to** — not because Claude guessed. CC4 layers permission rules on top of these.

Knowledge Check

1. Two CLAUDE.md files have conflicting rules: `~/.claude/CLAUDE.md` says "always show diffs" and `./CLAUDE.md` says "skip diffs for trivial changes." Which wins?

- ☐ A The user-level file (always show diffs) — user > project.
- ☐ B The project file (skip for trivial) — project (tier 3) beats user (tier 5).
- ☐ C They concatenate — both apply.
- ☐ D Last-loaded wins.

2. You're working in a monorepo. `packages/billing/CLAUDE.md` says "all changes need tests." `./CLAUDE.md` says nothing about tests. You edit a file in `packages/billing/`. What rules apply?

- ☐ A Only root CLAUDE.md (the deeper one is ignored).
- ☐ B Only the billing CLAUDE.md (replaces root).
- ☐ C Both — the directory-level file adds tests-required to the root rules.
- ☐ D Neither — CLAUDE.md only loads at repo root.

3. Your CLAUDE.md is 800 lines and Claude responses are getting slow. What's the right fix?

- ☐ A Trim everything that's not strictly required.
- ☐ B Use `/compact` more often.
- ☐ C Move long-form playbooks (e.g., "how to run a release") into skills (CC5); CLAUDE.md keeps only the always-on rules.
- ☐ D Switch to a faster model.

4. You're mid-session and just discovered "this CI test is flaky on macOS only when DOCKER_HOST is unset." Fastest way to remember it for next session?

- ☐ A Type a message starting with `#` — Claude Code saves it to your chosen CLAUDE.md.
- ☐ B Exit Claude Code, open CLAUDE.md in your editor, add the line, save, restart.
- ☐ C Run `/clear` to remember the next turn.
- ☐ D You can't — CLAUDE.md is read-only at runtime.

5. A teammate complains: "I don't see any of the conventions you claimed to set up in CLAUDE.md!" What's the most likely cause?

- ☐ A You wrote them in `~/claude/CLAUDE.md` (user-level, your machine only) instead of `./CLAUDE.md` (project-level, checked into git).
- ☐ B Claude has a bug.
- ☐ C Their model is different.
- ☐ D Auto Memory is conflicting.

Module Summary

- 5 tiers, lower number wins: Managed > Local > Project > Directory > User.
- Minimum-viable CLAUDE.md: 5 sections, <25 lines, project-specific rules only.
- Path-specific rules via frontmatter `globs` (single file) or per-directory `CLAUDE.md` (cleaner for monorepos).
- `/init` bootstraps a draft. `AGENTS.md` is the cross-tool convention; reference it from CLAUDE.md via `@AGENTS.md` rather than duplicating.
- Three in-session memory ops: `/memory` edits active CLAUDE.md files, `#` at message-start saves a new memory line, `@path/to/file.md` imports another markdown file (max 5 hops deep).
- Don't put: README content, long playbooks (use skills), tone/personality instructions, secrets.

CC4: Permissions & Sandbox

Two independent safety layers: **permission modes** (Claude asks before acting) and the **sandbox** (OS-enforced jail). Use both. Pick the right combination for dev, CI, and production-touching work.

Learning Objectives

- ☐ Distinguish permission modes (advisory) from the sandbox (OS-enforced) and understand why you layer both.
- ☐ Pick the right permission mode for: prototyping · reviewing risky changes · unattended CI · production-touching work.
- ☐ Write `permissions.allow` / `ask` / `deny` rules with the correct syntax.
- ☐ Configure `sandbox.filesystem` and `sandbox.network` for safe Bash execution.
- ☐ Avoid the three permission anti-patterns that quietly disable safety checks.

Two layers, different jobs. Get both right and you stop worrying about `rm -rf` typos.

Two Independent Layers

Everyday Analogy

Think of how a research lab handles a new chemical. The first layer is the **posted procedure**: "wear gloves, work in the fume hood, no flames within 10 feet." The second layer is the **physical engineering**: the fume hood actually exists, the gas valve is locked, the lab door has a keycard. The procedure is advisory; the engineering is enforced.

If you only had the procedure, a careless researcher could ignore it. If you only had the engineering, every routine task would feel locked-down and tedious. You need both, layered, with the engineering catching the mistakes the procedure misses.

Same here: **permission modes** are the posted procedure ("ask me before bash"). The **sandbox** is the physical engineering (the OS won't let bash touch `/etc`). Permission modes are configurable per session, sandbox is more rigid — both have their place.

THE TWO LAYERS COMPARED

Property	Permission Modes	Sandbox
Enforcement	Claude asks before acting (cooperative)	OS blocks the action (mandatory)
Failure mode	Could be bypassed if Claude misbehaves	Returns ENOENT / network error to Claude
Granularity	Per-tool, per-pattern (<code>Bash(npm run:*)</code>)	Filesystem path / network domain
Scope	Just Claude Code's tool calls	Everything bash spawns (incl. subprocesses)
Cost to set up	Minutes (settings.json)	Some setup (paths, domains, OS support)
Best for	Day-to-day workflow tuning	"Nothing escapes this directory"

Why both, not either-or

Permission modes are how you tell Claude "don't bother me about `npm test` ; ask me before `git push` ." That's UX. The sandbox is how you tell the OS "even if Claude misbehaves, it can't write outside this project." That's security. Different problems, different tools.

The Four Permission Modes

Modes are set per session via `--permission-mode` at launch, by cycling with Shift+Tab inside a session, or as the default in `permissions.defaultMode` :

MODE → BEHAVIOR → WHEN TO USE

Mode	Behavior	Right for
<code>plan</code>	Read-only. Claude can read/search/think but cannot edit or execute.	Risky migrations, auth refactors, "explain it first"
<code>default</code>	Asks for permission on bash, edits, fetches.	Day-to-day work on anything you care about
<code>acceptEdits</code>	Auto-accepts file edits and common filesystem commands.	Prototyping new files, throw-away projects
<code>bypassPermissions</code>	Skip permission prompts entirely.	Sandboxed containers / disposable VMs only — never your laptop

The cardinal rule

Never use `bypassPermissions` on your dev machine. One `rm -rf` and your home folder is gone, no prompt, no stop. Use it in ephemeral environments only: Docker containers, GitHub Actions runners, throw-away VMs, sandboxed worktrees. In any of those, the blast radius is the container — bounded.

Cycling modes inside a session

Press Shift+Tab to cycle through modes without restarting. The current mode shows in the prompt. Common pattern: start in `plan` to design, switch to `default` to execute, switch back to `plan` to review. Press Ctrl+G to see what each mode means right now.

Allow / Ask / Deny Rules — The Pattern Language

Modes are a coarse setting. **Permission rules** are the fine knobs: pre-approve a specific command, deny a specific path, force a prompt for a specific MCP tool. They live under `permissions` in `settings.json`:

```
{
  "permissions": {
    "defaultMode": "default",
    "allow": [
      "Bash(npm test:*)",
      "Bash(git diff:*)",
      "Bash(git status:*)",
      "Read(./**)",
      "Edit(src/**)",
      "WebFetch(domain:github.com)",
      "WebFetch(domain:*.anthropic.com)"
    ],
    "ask": [
      "Bash(git push:*)",
      "Bash(git commit:*)",
      "Edit(.env*)",
      "Edit(prisma/schema.prisma)"
    ],
    "deny": [
      "Bash(rm:*)",
      "Bash(sudo:*)",
      "Edit(.git/**)",
      "Read(./env)",
      "Read(~/.ssh/**)",
      "WebFetch(domain:*)"
    ]
  }
}
```

The pattern language

- **Tool name alone:** `Bash` matches every Bash call.
- **Bash prefix:** `Bash(npm test:*)` matches calls where the command starts with `npm test`. The `:*` suffix is the prefix-match wildcard for Bash rules.
- **Bash exact:** `Bash(npm install)` matches only that exact command, no arguments.
- **Path glob:** `Edit(src/**)` matches edits anywhere in `src/`.
- **WebFetch domain:** `WebFetch(domain:github.com)` matches GitHub URLs.
- **MCP tool:** `mcp__github__create_issue` matches that one MCP tool.
- **Subagent:** `Agent(code-reviewer)` matches spawning that subagent.

Precedence: deny > ask > allow

If a tool call matches multiple rules, **deny wins**, then **ask**, then **allow**. So you can confidently `allow Bash` and add narrow `deny Bash(rm -rf*)` — the deny wins.

Three permission anti-patterns

1. **Allow-list with no deny rules.** `"allow": ["Bash"]` with no deny means `rm -rf` auto-runs. Always pair allow with explicit deny for destructive patterns like `Bash(rm:*)`.
2. **Allow-listing too broad.** `WebFetch(domain:*)` = "fetch literally anything." Lock domains.
3. **Storing rules only at user level.** Project-shared rules belong in `./.claude/settings.json`. Personal preferences live at user level. Don't conflate them — teammates don't get your user-level rules.

OS-Level Sandbox — The Engineering Layer

Technical Definition

The **sandbox** wraps every Bash call inside an OS-level jail: *Seatbelt* on macOS, *namespaces* on Linux/WSL2. When sandbox is on, Bash commands cannot read or write outside paths you allow, cannot reach domains you haven't allowlisted, and cannot escape to the parent shell. Even if Claude misbehaves, the OS enforces the wall.

Bonus: with `autoAllowBashIfSandboxed: true` (default), sandboxed commands skip the permission prompt — the OS is enforcing the boundary, so the cooperative prompt is redundant. This makes sandboxes the *safest* way to run permission-free workflows.

Minimum-viable sandbox config

```
{
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "filesystem": {
      "allowWrite": ["/", "~/.cache/claude"],
      "denyWrite": ["/.git", "/.env*"],
      "denyRead": ["/.env", "~/.ssh", "~/.aws", "~/.config/gcloud"]
    },
    "network": {
      "allowedDomains": [
        "registry.npmjs.org",
        "api.github.com",
        "*.anthropic.com",
        "*.your-internal.dev"
      ],
      "deniedDomains": ["*"]
    },
    "excludedCommands": ["docker", "kubectl"]
  }
}
```

Path prefixes: `/` = absolute, `~/` = home, `./` = project-relative. **Network:** wildcards work; `deniedDomains: ["*"]` establishes default-deny so `allowedDomains` is the explicit allowlist. `excludedCommands`: a list of binaries that bypass the sandbox — needed for `docker` and `kubectl` which require their own networking.

Platform notes

- **macOS:** native Seatbelt. Most reliable platform. Some macOS-specific keys: `allowMachLookup`, `allowLocalBinding`, Unix sockets.
- **Linux / WSL2:** namespace-based. Requires unprivileged user namespaces (most distros enable by default; check `/proc/sys/kernel/unprivileged_usersns_clone`). For unprivileged Docker, set `enableWeakerNestedSandbox: true`.
- **Windows native:** limited sandbox support. Either run via WSL2, or rely on permission modes + careful allow/deny rules.

When sandbox earns its keep

The classic case: you ask Claude to run a build script you don't fully trust ("this gradle plugin came from a stranger's blog"). Without sandbox, that script could exfiltrate `~/aws/credentials`. With sandbox, it can't reach `~/aws` at all — the read returns `ENOENT`. The script either adapts or fails loudly. Either way, your credentials are safe.

Recommended Defaults — Three Profiles

Three profiles to copy-paste based on context:

Profile A: Dev laptop — "personal default"

User-level `~/.claude/settings.json`. Balanced for daily work.

```
{
  "permissions": {
    "defaultMode": "default",
    "allow": ["Bash(git status:*)", "Bash(git diff:*)", "Bash(npm test:*)", "Bash(pnpm test:*)", "Read(./**)", "Read(../**)", "Read(**/*)"],
    "deny": ["Bash(rm:*)", "Bash(sudo:*)", "Read(./env)", "Read(~/.ssh/**)", "Read(~/.aws/**)", "Edit(.git/**)"]
  }
}
```

Profile B: Untrusted-build sandbox — "show me the receipts"

Project-level for repos with external/untrusted scripts. Sandbox on, narrow domains.

```
{
  "permissions": {"defaultMode": "default"},
  "sandbox": {
    "enabled": true,
    "filesystem": {"allowWrite": ["/"], "denyRead": ["/.env", "~/.ssh", "~/.aws"]},
    "network": {"allowedDomains": ["registry.npmjs.org", "registry.yarnpkg.com"], "deniedDomains": ["*"]}
  }
}
```

Profile C: CI runner — "fully unattended"

For GitHub Actions / GitLab CI inside a sandboxed container. Everything pre-approved, narrow tool surface, sandbox on. Only safe because the container itself is the blast-radius boundary.

```
{
  "permissions": {
    "defaultMode": "bypassPermissions",
    "allow": ["Bash(pnpm:*)", "Bash(git diff:*)", "Read(./**)", "Edit(./**)", "WebFetch(domain:api.github.com)",
    "deny": ["Bash(git push:*)", "Bash(rm:*)", "Bash(curl:*)"]
  },
  "sandbox": {"enabled": true, "filesystem": {"allowWrite": ["/"]}, "network": {"allowedDomains": ["api.github.com"]}}
}
```

What just happened?

Three profiles cover most cases. Profile A is the "always ask" workflow you'll use 80% of the time. Profile B adds sandbox when you don't trust what's about to run. Profile C is for headless CI where there's no human to prompt.

Hands-On Lab — Three Permission Tiers for the PublicRecords API

Working in the **PublicRecords API** from CC0–CC3. You'll configure three behaviors: Maven test/build runs silently, deploy and **git push** prompt, production database commands are denied outright. Then prove each tier responds correctly. About 10 minutes.

Step 1 — Snapshot what's loaded right now

```
$ cd /path/to/PublicRecordsAPI
$ claude
> /status
```

/status shows model, mode, settings sources, and active permission rules. Note which sources are populated — you may already have a user-level settings.json from another project.

Step 2 — Write project-level permissions for the API

HOW IT WORKS

Permissions live in `.claude/settings.json` at the project root and are committed to git so the whole team uses the same rules. Three tiers stack:

- **allow**: pre-approved — runs silently, no prompt. The dev inner loop.
- **ask**: prompts you (yes once / yes for session / no). Use for anything visible to the team or production.
- **deny**: hard refuses — no prompt, no override. Use for the truly destructive stuff.

The matcher syntax `Bash(mvn test:*)` means "any Bash invocation that starts with `mvn test`." Trailing `:*` is the wildcard.

Make the directory:

```
$
```

Then create the settings file:

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "defaultMode": "default",
    "allow": [
      "Bash(mvn test:*)",
      "Bash(mvn compile:*)",
      "Bash(mvn spring-boot:run:*)",
      "Bash(mvn spotless:apply:*)",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Bash(git log:*)",
      "Bash(curl http://localhost:8080/*)",
      "Edit(src/**)",
      "Edit(pom.xml)"
    ],
    "ask": [
      "Bash(mvn deploy:*)",
      "Bash(mvn release:*)",
      "Bash(git push:*)",
      "Bash(git commit:*)",
      "Edit(.github/**)",
      "Edit(application-prod.yml)"
    ],
    "deny": [
      "Bash(mvn flyway:clean:*)",
      "Bash(psql:*)",
      "Bash(rm:*)",
      "Bash(sudo:*)",
      "Read(/application-prod.yml)",
      "Read(~/.ssh/**)",
      "Read(~/.aws/**)",
      "Edit(.git/**)"
    ]
  }
}
```

Commit it so the team gets it:

```
$
```

Restart Claude Code from the project root and run `/status` — your rules should appear under *"Project settings."*

Step 3 — Validate the allow tier (no prompt)

```
> Run the tests.

> Now compile only — don't run tests this time.

> Show me git status.
```

Expected: all three execute immediately, no prompts. If you see a prompt, your matcher is too narrow — check the trailing `:*` wildcard.

Step 4 — Validate the ask tier (prompt)

```
> Run `mvn deploy` to push this to staging.
```

Expected: a permission prompt with three choices (yes once / yes for session / no). Choose **no** — nothing should actually deploy, and Claude should acknowledge the deny.

Step 5 — Validate the deny tier (refuse, hard stop)

```
> Run `mvn flyway:clean` to wipe the database for a fresh start.

> Now read application-prod.yml and tell me the production DB URL.
```

Expected: both refused outright, no prompt offered. Deny rules don't ask — they block. If either succeeds, your JSON didn't load (most often: trailing comma, wrong path, or settings.json is in the wrong directory). Run `/status` and look at *"Effective permissions."*

Step 6 — Add user-level dev defaults that apply everywhere

Project rules cover this repo. User-level rules cover everything you do, including throwaway terminals where you forget to set up a project settings.json.


```
{
  "permissions": {
    "deny": [
      "Bash(rm:*)",
      "Bash(sudo:*)",
      "Bash(curl:*)",
      "Read(~/.ssh/**)",
      "Read(~/.aws/**)",
      "Read(~/.config/gcloud/**)",
      "Read(./env)",
      "Read(./env.*)"
    ]
  }
}
```

These deny rules are belt-and-braces protection — they apply even when you're working on a repo without a project-level settings.json.

Step 7 — Try the sandbox for a "run this random Maven plugin" scenario

Permissions ask you (or refuse). The **sandbox** goes further — the OS itself blocks file writes outside your project root and network access to non-allowlisted domains. Use it when you're about to evaluate an unknown Maven plugin or run code from an untrusted source.

Create a throwaway test directory and the sandbox config:

```
$
```

```
{
  "permissions": { "defaultMode": "default" },
  "sandbox": {
    "enabled": true,
    "filesystem": {
      "allowWrite": ["/"],
      "denyRead": ["~/.ssh", "~/.aws", "~/.m2/settings-security.xml"]
    },
    "network": {
      "allowedDomains": ["repo.maven.apache.org", "repo1.maven.org"],
      "deniedDomains": ["*"]
    }
  }
}
```

Launch Claude in that directory:

```
$
```

Inside Claude Code, ask it to write to `/etc/test.txt` — you'll get an OS-level *EACCES*. Ask it to `curl example.com` — the network is blocked. Maven Central still works because we allowlisted it.

Step 8 — Verify the layered picture with `/status`

Back in the PublicRecords API directory, one last `/status` should show:

- User settings: your global deny defaults from Step 6
- Project settings: the three-tier matrix from Step 2
- Effective permissions: union, with denies on top — `mvn flyway:clean` still blocked even if it appeared in allow somewhere.

Lab complete — what you should have

Project-level permissions that make the inner dev loop silent (`mvn test`, `mvn compile`, `git status`), prompt for anything that escapes your laptop (`deploy`, `push`, `commit`), and refuse production DB commands outright. Plus user-level deny defaults as a safety net and a sandbox template for evaluating untrusted code. **Never use `bypassPermissions` on your laptop — ever.** CC5 builds slash commands that respect these tiers.

Knowledge Check

1. You're about to do a risky migration of authentication code across 12 files. Which permission mode?

- ☐ A `acceptEdits` — speed matters.
- ☐ B `plan` — read-only, get a written plan first.
- ☐ C `bypassPermissions` — you're a senior dev.
- ☐ D `default` — just answer the prompts as they come.

2. Sandbox vs permission modes — which statement is true?

- ☐ A They're alternatives; pick one.
- ☐ B Permissions are advisory (cooperative); sandbox is OS-enforced. Layer both.
- ☐ C Sandbox replaces permission modes when it's on.
- ☐ D They protect different tools (sandbox = bash only, permissions = everything else).

3. Your settings.json has "allow": ["Bash"] and "deny": ["Bash(rm:*)"] . Claude is asked to run `rm -rf node_modules` . What happens?

- ☐ A Auto-runs — matches the allow rule first.
- ☐ B Blocked — deny beats allow on the same tool.
- ☐ C Prompts you — rules conflict.
- ☐ D Returns an error to Claude without telling you.

4. You set `sandbox.enabled: true` with `filesystem.allowWrite: ["/"]`. Claude tries to run a script that writes to `/tmp/cache/foo`. What happens?

- ☐ A Writes succeed — `/tmp` is special-cased.
- ☐ B Write fails (EACCES) — `/tmp` is outside the allowlist.
- ☐ C Claude prompts you to allow.
- ☐ D It's redirected silently into the project dir.

5. A teammate sets `"defaultMode": "bypassPermissions"` in their `user-level settings.json`. What's the risk?

- ☐ A Just a bit faster, no risk.
- ☐ B Every session in every project runs every tool without prompting — including `rm`, `git push --force`, MCP tools with database connection strings. One bug or hallucination, big damage.
- ☐ C It only affects `claude --bypass` sessions.
- ☐ D Project-level settings always override it.

Module Summary

- Two layers: **permission modes** (advisory, configurable) + **sandbox** (OS-enforced, mandatory). Use both.
- Four modes: `plan`, `default`, `acceptEdits`, `bypassPermissions`. Cycle with Shift+Tab.
- Rules: `allow` / `ask` / `deny`. Precedence: `deny` > `ask` > `allow`.
- Sandbox: `filesystem.allowWrite`, `denyRead`, `network.allowedDomains` with `deniedDomains: ["*"]` as default-deny.
- Three profiles: dev laptop · untrusted-build sandbox · CI runner. Copy-paste based on context.
- Never `bypassPermissions` on a dev machine. Disposable environments only.

CC5: Skills & Slash Commands

Build your own `/deploy`, `/triage`, `/review`. Custom commands and skills are now *the same thing* — here's how to use the merged system, including shell-injected context, forked execution, and the bundled skills that ship with Claude Code.

Learning Objectives

- ☐ Explain why custom commands and skills merged — and when to still use the simple-command shape.
- ☐ Drop a SKILL.md anywhere in the 4-scope hierarchy and have Claude pick it up live.
- ☐ Use all six argument substitution variables (`$ARGUMENTS`, `$N`, `$name`, `${CLAUDE_SESSION_ID}`, `${CLAUDE Effort}`, `${CLAUDE_SKILL_DIR}`).
- ☐ Inject live data with `!`cmd`` backtick syntax (and disable it via policy).
- ☐ Pick when to use `context: fork` for isolation vs inline execution.
- ☐ Reach for the right bundled skill (`/simplify`, `/batch`, `/debug`, `/loop`, `/claude-api`) instead of writing one.

First the structural change you need to know: commands and skills used to be different. They aren't anymore.

Commands and Skills, Unified

Everyday Analogy

Imagine your kitchen used to have two different ways to write recipes: index cards (one-pagers) and binders (multi-page playbooks with checklists, prep notes, photos). Both produced dishes. Both used the same ingredients. The split forced you to choose: simple? Use a card. Complex? Use a binder. And every binder needed its own giant filing cabinet.

Then you got smarter cards: stick a card in a folder, optionally add prep notes, photos, scripts. Same place, same name, same way to find it — just more capacity when you need it. Index cards aren't deprecated; they live as folders-with-only-a-card-inside, which is fine.

That's the merge. Old `.claude/commands/foo.md` still works, but it's now a degenerate skill: a folder named `foo` with just a SKILL.md inside. New work gets the upgrade for free, old work keeps running.

Technical Definition

A **skill** is a directory containing a `SKILL.md` file. The directory name becomes the slash command (`my-skill/` → `/my-skill`). The file uses YAML frontmatter for config and Markdown for the prompt body. Skills can also include supporting files (templates, scripts, examples) that the body references.

Old-style files at `.claude/commands/name.md` still work and use the same frontmatter. **Skills add four things commands can't do:** automatic invocation (Claude triggers based on description), supporting files, hooks scoped to skill lifecycle, and execution in a forked subagent context.

The 4 Scopes — Where Skills Live

SKILL SCOPE HIERARCHY — HIGHER BEATS LOWER

Scope	Path	Visible to
Enterprise	<code>managed-settings/skills/</code>	Everyone in the org — admin-deployed
Personal	<code>~/.claude/skills/<name>/SKILL.md</code>	You, all your projects
Project	<code>.claude/skills/<name>/SKILL.md</code>	This repo, all teammates (checked into git)
Plugin	<code><plugin>/skills/<name>/SKILL.md</code>	Wherever the plugin is enabled (namespaced as <code>plugin:skill</code>)

Conflict resolution: Enterprise overrides Personal overrides Project. Plugin skills use `plugin-name:skill-name` namespacing — they cannot conflict with non-plugin skills. If a skill and a custom command share a name at the same scope, the skill wins.

Live change detection: add, edit, or remove a skill inside `~/.claude/skills/`, project `.claude/skills/`, or any directory passed via `--add-dir` — the change takes effect *during* the current session. No restart needed. Creating a brand-new top-level skills directory does require a restart so it can be watched.

Monorepo bonus: nested discovery

When you work on a file in `packages/frontend/`, Claude Code also scans `packages/frontend/.claude/skills/`. Each sub-package can ship its own skills without polluting the root.

Build Your First Skill in 60 Seconds

Concrete example: a skill that runs the test suite, parses failures, and prints a focused report. We'll call it `/test-triage`.

```
mkdir -p .claude/skills/test-triage
cat > .claude/skills/test-triage/SKILL.md <<'EOF'
---
description: Run the test suite, identify failures, propose fixes
allowed-tools: Bash(pnpm test*), Bash(pnpm typecheck*), Read, Edit
---

Run `pnpm test` and capture the output.

For each failure:
1. Identify the test file and the failing assertion.
2. Read the source under test.
3. Decide whether the test is wrong or the source is wrong.
4. Propose the smallest change that makes the test pass without breaking others.

Report:
- Total: X tests, Y failing
- For each failure: file, root cause, proposed change
- Don't apply changes – just report.
EOF
```

Now in any session inside that repo:

```
> /test-triage
```

Claude runs the suite, reads the source, and reports. The `allowed-tools` entry pre-approves Bash for `pnpm test` and `pnpm typecheck` only — nothing else.

What just happened?

You wrote a 12-line skill that turns a multi-step diagnostic flow into one slash command. Five teammates run `/test-triage` in their sessions and get the same playbook. That's the bargain: skills convert "the way you debug" from tribal knowledge into checked-in artifacts.

SKILL.md Frontmatter — The Knobs

15 frontmatter fields. You'll use 4-5 most of the time. The full list:

SKILL.MD FRONTMATTER — ALL 15 FIELDS

Field	What it does
<code>name</code>	Display name (defaults to dir name); lowercase + hyphens
<code>description</code>	Recommended. When Claude should auto-invoke. Front-load the use case — combined with <code>when_to_use</code> , capped at 1,536 chars in the listing
<code>when_to_use</code>	Trigger phrases / example requests, appended to <code>description</code>
<code>argument-hint</code>	Autocomplete hint, e.g. <code>[issue-number]</code>
<code>arguments</code>	Named positional args for <code>\$name</code> substitution: <code>arguments: [issue, branch]</code>
<code>disable-model-invocation</code>	<code>true</code> → only YOU can invoke. Use for <code>/deploy</code> , <code>/commit</code>
<code>user-invocable</code>	<code>false</code> → hides from <code>/</code> menu; only Claude can invoke (background knowledge)
<code>allowed-tools</code>	Pre-approves tools while skill is active — doesn't restrict, just skips per-use prompt
<code>model</code>	Override (sonnet/opus/haiku/full ID/ <code>inherit</code>); resets after the turn
<code>effort</code>	Thinking budget for the turn: low / medium / high / xhigh / max
<code>context</code>	<code>fork</code> → runs in a forked subagent context, no main-conversation access
<code>agent</code>	Which subagent type executes the fork: <code>Explore</code> , <code>Plan</code> , <code>general-purpose</code> , or custom
<code>hooks</code>	Hooks scoped to this skill's lifecycle (CC7)
<code>paths</code>	Glob patterns — auto-load only when working with matching files
<code>shell</code>	<code>bash</code> (default) or <code>powershell</code> (Windows; needs <code>CLAUDE_CODE_USE_POWERSHELL_TOOL=1</code>)

Anti-pattern: tool restriction in `allowed-tools`

`allowed-tools` grants *pre-approval*, not restriction. Listing `Read` there does NOT prevent Claude from using `Edit` — it just skips the per-use prompt for `Read`. To actually *block* tools while a skill runs, add deny rules in `permissions` (CC4). Most users misread this on first encounter.

Argument Substitution — Six Variables

Variables interpolate before the skill body is sent to Claude. Positional args use shell-style quoting (`"hello world"` = one arg, not two).

Variable	Resolves to
<code>\$ARGUMENTS</code>	Full argument string. If absent from body, appended as <code>ARGUMENTS: <value></code> .
<code>\$ARGUMENTS[N]</code> / <code>\$N</code>	Positional arg by 1-based index (<code>\$1</code> , <code>\$2</code> ...)
<code>\$name</code>	Named arg from <code>arguments</code> frontmatter list, mapped by position
<code>\${CLAUDE_SESSION_ID}</code>	Current session ID — use for log paths, correlation IDs
<code>\${CLAUDE Effort}</code>	Active effort level — adapt instructions ("at <code>low</code> , skip exhaustive checks")
<code>\${CLAUDE_SKILL_DIR}</code>	Directory containing this SKILL.md — use to invoke bundled scripts

Example: a fix-issue skill

```
---
name: fix-issue
description: Fix a GitHub issue by number
arguments: [issue, branch]
disable-model-invocation: true
allowed-tools: Bash(gh issue view*), Read, Edit, Bash(git*)
---
```

Fix GitHub issue \$issue on branch \$branch.

1. Run ``gh issue view $issue`` and read the description.
2. Identify the affected files.
3. Implement the fix on \$branch.
4. Run tests.
5. Open a PR linked to issue #\$issue.

If \$branch doesn't exist, create it from main.

Invoke with `/fix-issue 1234 fix/login-bug` — `$issue` becomes `1234`, `$branch` becomes `fix/login-bug`.

Shell Injection — `!`cmd`` Runs Before Claude Sees Anything

Skills can run shell commands at *render time* using backtick syntax. The output replaces the placeholder, and the rendered prompt is what reaches Claude. This is preprocessing — not Claude executing the command.

```
---
name: pr-summary
description: Summarize the current pull request using live GitHub data
context: fork
agent: Explore
allowed-tools: Bash(gh *)
---

## PR diff
!`gh pr diff`

## Reviewer comments
!`gh pr view --comments`

## Multi-line block
```!
node --version
npm --version
git status --short
```

Summarize the changes and flag anything that needs reviewer attention.
```

When this runs:

1. Each `!`cmd`` executes; stdout is captured.
2. Output replaces the placeholder in the rendered text.
3. Claude receives the rendered prompt with actual PR data inlined — no tool round-trips needed.

Disable shell injection by policy

For managed environments, set `"disableSkillShellExecution": true` in settings. Each `!`cmd`` is replaced with `[shell command execution disabled by policy]`. Affects user/project/plugin skills; bundled and managed skills are exempt. Most useful in managed-policy settings where users cannot override.

Why this matters

Inline shell injection saves tool round-trips. A `/pr-summary` skill without it would call Bash 3 times during the conversation. With injection, the data is already there when Claude starts thinking. Faster, cheaper, fewer permission prompts.

Skills That Run in a Fork — context: fork

Add `context: fork` and the skill runs in a forked subagent context: it gets the system prompt, no main-conversation access, and its tool calls don't pollute your transcript — only the final result returns. Pair with `agent: Explore` to choose which subagent type executes (CC6 covers the full subagent system).

```
---  
name: deep-research  
description: Research a topic across the codebase  
context: fork  
agent: Explore  
---  
  
Research $ARGUMENTS thoroughly:  
1. Use Glob and Grep to find relevant files  
2. Read the most relevant ones  
3. Summarize findings with specific file references  
4. Identify follow-up questions worth pursuing
```

When to use fork: the skill produces verbose intermediate output (file reads, search results) you don't need in main context. **When NOT:** reference content like API conventions; those should run inline so Claude can use them alongside your conversation.

Fork warning — needs an explicit task

`context: fork` only makes sense for skills with *a task to execute*. If the skill is "use these conventions" with no task, the forked subagent receives the conventions but no instruction — it returns nothing useful. Reference content should be plain skills (no fork) or preloaded into a custom subagent.

Bundled Skills — Use Before You Build

Claude Code ships with prompt-based skills in every session. Reach for these *before* writing your own:

| Skill | Use when |
|--------------------------|--|
| <code>/simplify</code> | Refactor recently-changed code for clarity / consistency / maintainability |
| <code>/batch</code> | Run the same task across many files / branches / PRs |
| <code>/debug</code> | Structured debugging playbook (reproduce, isolate, fix, verify) |
| <code>/loop</code> | Background monitoring — rerun a check on a cadence and report drift |
| <code>/claude-api</code> | Help build agents on top of the Claude Messages API |

Plus a few built-in commands also available through the Skill tool: `/init`, `/review`, `/security-review`. Block all skills with a `Skill` deny rule, restrict to specific ones with `Skill(name)` in `permissions.allow`. To trigger *extended thinking* from a skill, include the literal word "ultrathink" anywhere in the skill content.

Skill lifecycle reality

Once invoked, the skill body enters the conversation as *one message* and stays there. Claude does NOT re-read the file on later turns — write standing instructions, not one-shots. After auto-compaction, the most recent invocation of each skill is re-attached, keeping the first 5,000 tokens of each up to a combined 25,000-token budget. Older invocations get dropped first.

Hands-On Lab — Build a `/new-resource` Generator for the API

Working in the **PublicRecords API**. You'll author a project-scoped slash command that scaffolds Controller + Service + Repository + MockMvc test for a new UCC entity, given just an entity name. About 10 minutes.

Step 1 — Create the command file

HOW IT WORKS

Slash commands live as Markdown files in `.claude/commands/`. The filename (minus `.md`) becomes the command name — `new-resource.md` = `/new-resource`. The file body becomes the prompt sent to Claude when you invoke the command. Frontmatter at the top (`description`, `argument-hint`) controls the picker UI. Arguments are referenced as `$1`, `$2`, etc.

From the project root, make the commands directory:

```
$
```

Create the command file with this content:

description: Scaffold Controller + Service + Repository + test for a UCC entity
argument-hint: <EntityName>

Generate four Java files for a new resource named ``$1`` in the
``com.publicrecords.api.$1lower`` package, where ``$1lower`` is ``$1`` lowercased.

Follow the existing layout in ``com.publicrecords.api.filing`` exactly:

1. `**`$1.java`**` – JPA entity. Always include:
 - ``id`` (Long, ``@GeneratedValue`` IDENTITY, primary key)
 - ``state`` (String, 2-char, ``@NotBlank``, ``@Size(min=2,max=2)``)
 - ``createdAt`` (Instant, ``@Column(nullable=false)``)
 - 2-3 entity-specific fields appropriate for US public records
(e.g., for ``Amendment``: ``originalFilingId``, ``amendmentType``,
``amendedAt``).
2. `**`$1Repository.java`**` – extends ``JpaRepository<$1, Long>``.
Add a ``findByState(String state)`` derived query.
3. `**`$1Service.java`**` – ``@Service`` with ``findAll``, ``findById``,
``create($1 entity)``. Constructor-inject the repository (no ``@Autowired``).
4. `**`$1Controller.java`**` – ``@RestController`` with ``@RequestMapping("/$1lower")``.
 - ``GET /`` → `service.findAll()`
 - ``GET /{id}`` → `service.findById`, returning ``ResponseEntity<$1>``
with ``notFound()`` on miss
 - ``POST /`` → `service.create` with ``@Valid @RequestBody``Every public method gets a 1-line JavaDoc.
5. `**`$1ControllerTest.java`**` under
``src/test/java/com/publicrecords/api/$1lower/`` – one `MockMvc` test
asserting ``GET /$1lower`` returns 200.

After generating, run ``mvn compile`` and report any errors.

Step 2 – Verify Claude sees the command

Restart Claude Code and type `/` – `/new-resource` should appear in the picker. Type `/help new-resource` to confirm the description and argument-hint are loaded.

Step 3 – Generate a real resource

```
> /new-resource Amendment
```

Claude reads the existing `filing/` package as reference, generates the five files under `amendment/`, then runs `mvn compile`. Watch the file ribbon — you should see five `Edit / Write` calls in sequence. If `mvn compile` fails, ask Claude to fix the errors and re-compile.

Step 4 — Verify the structure matches the existing pattern

```
$ tree src/main/java/com/publicrecords/api/amendment src/test/java/com/publicrecords/api/amendment
$ mvn test
$ curl http://localhost:8080/amendments # in another terminal after mvn spring-boot:run
```

Expected: five new files in the right packages, all tests pass, and `/amendments` returns `[]` (no seed data, but the endpoint responds 200).

Step 5 — Generate a second resource and confirm consistency

```
> /new-resource Termination
```

The command should produce a structurally identical package — same field naming, same controller method signatures, same test shape. **That consistency is the whole point of a slash command:** the next teammate doesn't have to remember conventions because the command bakes them in.

Step 6 — Convert it to a Skill (optional, for autoinvocation)

Slash commands are explicit (`/new-resource X`). A Skill with the same description fires whenever Claude infers it's needed. Convert:

```
$ mkdir -p .claude/skills/new-resource
$ mv .claude/commands/new-resource.md .claude/skills/new-resource/SKILL.md
```

Now ask Claude in plain language: *"We need a new entity for UCC continuation filings."* The skill should fire without you typing the command. (If it doesn't, your `description` field isn't specific enough — Claude won't auto-invoke a skill it isn't confident about.)

Lab complete — what you should have

A reusable scaffolding command (or skill) under `.claude/` that produces structurally consistent CRUD packages on demand. Two real packages (`amendment/`, `termination/`) generated, compiled, and tested. The command is committed with the repo, so every teammate gets the same conventions for free. CC6 builds a subagent that audits these new endpoints for missing PII protections.

Knowledge Check

1. You have `.claude/commands/deploy.md` in your repo. A teammate adds `.claude/skills/deploy/SKILL.md`. What happens when you type `/deploy`?

- ☐ A Error — name conflict.
- ☐ B The command (older) wins.
- ☐ C The skill wins — skills take precedence over same-named commands.
- ☐ D The most recently modified file wins.

2. You add `allowed-tools: Read` to a skill's frontmatter. Claude calls `Edit` mid-skill. What happens?

- ☐ A Edit is blocked — not in allowed-tools.
- ☐ B Edit prompts you (or behaves per your default permission mode) — `allowed-tools` only pre-approves Read.
- ☐ C Edit silently fails.
- ☐ D The skill aborts.

3. Your skill needs the GitHub PR diff inlined into the prompt without a tool round-trip. What's the right syntax?

- ☐ A `{{ gh pr diff }}`
- ☐ B `!`gh pr diff` — inline shell injection`
- ☐ C `$(gh pr diff)`
- ☐ D Not possible — tools must round-trip.

4. A skill should run a thorough codebase search but you don't want all the file contents flooding your main context. What do you set?

- ☐ A `disable-model-invocation: true`
- ☐ B `allowed-tools: Read`
- ☐ C `context: fork with agent: Explore`
- ☐ D Move it to `~/.claude/skills/`

5. You want to manually clean up old branches via `/cleanup-branches` — not have Claude trigger it automatically. Which frontmatter field?

- ☐ A `user-invocable: true`
- ☐ B `disable-model-invocation: true`
- ☐ C `context: fork`
- ☐ D `auto-invoke: false`

Module Summary

- Commands and skills merged. New work uses skill format; old `.claude/commands/*.md` still works.
- 4 scopes: Enterprise > Personal > Project. Plugins use `plugin:skill` namespace.
- SKILL.md frontmatter has 15 fields; you'll use `description`, `allowed-tools`, `disable-model-invocation`, `context`, `arguments` most often.
- Substitution: `$ARGUMENTS`, `$N`, `$name`, `${CLAUDE_SESSION_ID}`, `${CLAUDE_EFFORT}`, `${CLAUDE_SKILL_DIR}`.
- Inline shell injection: `!`cmd`` for one-liners, ````! ... ```` for blocks. Disable via `disableSkillShellExecution`.
- `context: fork` + `agent:` for isolated research; inline for reference content.
- Bundled: `/simplify`, `/batch`, `/debug`, `/loop`, `/claude-api`. Plus `/init`, `/review`, `/security-review`.
- Trigger extended thinking by including "ultrathink" anywhere in skill content.

CC6: Subagents

Specialist agents that run in their own context window with their own tools and (optionally) their own memory. Use them when a task would otherwise flood your conversation with noise — or when you want to lock down what one specific worker can do.

Learning Objectives

- ☐ Decide between subagent / fork / skill / main-conversation for a given task.
- ☐ Know the 5 built-in subagents (Explore, Plan, general-purpose, statusline-setup, Claude Code Guide) and when each fires.
- ☐ Drop a custom subagent into `.claude/agents/` using the 16 frontmatter fields correctly.
- ☐ Invoke subagents three ways: natural language, `@agent-name` mention, `--agent` whole-session.
- ☐ Configure persistent `memory:` so a subagent *learns* across sessions.
- ☐ Use foreground / background / fork modes appropriately.

Why Subagents

Everyday Analogy

Imagine you're the lead architect on a project and someone asks "is the database schema consistent with the API contracts?" You could read every migration file and every controller yourself, but the answer is going to be 30 lines and the work is 3 hours of squinting. Better: hand it to a junior who specializes in schema review, let them read everything in their own corner, and have them come back with the verdict.

You stay focused on the architecture conversation. The junior's "I read 47 files" doesn't show up in your office. You only see "yes, consistent — except `orders.status` drifts from `OrderStatus` in the OpenAPI spec." That's the deal.

Subagents are that junior. Each runs in its own context window, gets a focused prompt, optionally has its own narrow tool allowlist, and returns just the summary. Your main conversation stays clean and high-level.

Technical Definition

A **subagent** is a Markdown file with YAML frontmatter at `.claude/agents/<name>.md` (or `~/.claude/agents/`). When Claude detects a task matching a subagent's **description**, it spawns the subagent in a new context window, hands it a focused prompt, and waits for the result. The subagent's tool calls don't appear in your main transcript.

Use a subagent when: the task produces verbose output you don't need, you want to enforce specific tool restrictions, or the work is self-contained and can return just a summary.

Don't use one when: the task needs frequent back-and-forth, multiple phases share significant context, or you're making a quick targeted change. Latency matters — subagents start fresh and need time to gather context.

5 Built-in Subagents — Already Available

Claude Code ships with these. Don't re-invent.

| Subagent | Model | Tools | When Claude uses it |
|-------------------|---------|---------------------|---|
| Explore | Haiku | Read-only | Codebase search at <i>quick, medium, or very thorough</i> |
| Plan | inherit | Read-only | Research during plan mode — prevents nesting |
| general-purpose | inherit | All | Multi-step work needing both exploration AND modification |
| statusline-setup | Sonnet | Read, Edit | When you run <code>/statusline</code> |
| Claude Code Guide | Haiku | WebFetch, WebSearch | When you ask "how do I ... in Claude Code?" |

You can disable any of these with `"deny": ["Agent(Explore)"]` in `permissions`. You can override their behavior by creating a same-named custom agent at higher priority.

The 5-Tier Scope Hierarchy

When name conflicts happen, the highest-priority definition wins:

| Priority | Scope | Path |
|----------|---------------------------|---|
| 1 | Managed | Organization-wide via managed settings |
| 2 | <code>--agents</code> CLI | JSON passed at launch — session-only, never saved |
| 3 | Project | <code>.claude/agents/</code> — checked into git, team-shared |
| 4 | User | <code>~/.claude/agents/</code> — personal, all your projects |
| 5 | Plugin | <code><plugin>/agents/</code> — namespaced as <code>plugin:agent</code> |

Browse them with `/agents` (interactive UI for create/edit/inspect) or `claude agents` (CLI list grouped by source).

Your First Custom Subagent

The classic example: a code-reviewer with read-only tools and a focused prompt. Drop this at `.claude/agents/code-reviewer.md`:

```
---
name: code-reviewer
description: Expert code review specialist. Proactively reviews code for quality, security, and maintainab
tools: Read, Grep, Glob, Bash(git diff*), Bash(git log*)
model: sonnet
color: orange
---

You are a senior code reviewer ensuring high standards.

When invoked:
1. Run git diff to see recent changes
2. Focus on modified files only
3. Begin review immediately

Review checklist:
- Code is clear and readable
- No exposed secrets or API keys
- Proper error handling
- No duplicated code
- Tests cover the new behavior

Provide feedback organized by priority:
- CRITICAL (must fix before merge)
- WARNING (should fix)
- SUGGESTION (consider)

Include specific examples of how to fix each issue.
```

Now in any session: *"have the code-reviewer subagent look at my recent changes."* Claude spawns it, the reviewer runs `git diff` in its own context, reads the modified files, and returns a structured report — without those file reads polluting your main transcript.

Subagent Frontmatter — All 16 Fields

Only `name` and `description` are required. Everything else lets you tighten tool access, lock the model, scope MCP, attach hooks, or carve out an isolated workspace.

| Field | Purpose |
|------------------------------|--|
| <code>name</code> | Lowercase-hyphen identifier; unique within scope |
| <code>description</code> | When Claude should delegate — include "use proactively" to encourage it |
| <code>tools</code> | Allowlist (e.g. <code>Read</code> , <code>Grep</code> , <code>Bash</code>); inherits all if omitted |
| <code>disallowedTools</code> | Denylist applied first; <code>tools</code> resolves against the remainder |
| <code>model</code> | <code>sonnet</code> , <code>opus</code> , <code>haiku</code> , full ID, or <code>inherit</code> (default) |
| <code>permissionMode</code> | <code>default</code> , <code>acceptEdits</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> , <code>plan</code> |
| <code>maxTurns</code> | Hard cap on agentic turns — safety circuit breaker |
| <code>skills</code> | Skills <i>preloaded</i> at startup — full body injected (vs description-only in main session) |
| <code>mcpServers</code> | Inline server defs (scoped) or string refs to existing servers |
| <code>hooks</code> | Lifecycle hooks scoped to this subagent only — cleaned up on finish |
| <code>memory</code> | <code>user</code> / <code>project</code> / <code>local</code> — persistent <code>MEMORY.md</code> across sessions |
| <code>background</code> | <code>true</code> → always run concurrently with permission pre-approval |
| <code>effort</code> | <code>low</code> / <code>medium</code> / <code>high</code> / <code>xhigh</code> / <code>max</code> — thinking budget |
| <code>isolation</code> | <code>worktree</code> → runs in temp git worktree — auto-cleaned if no changes |
| <code>color</code> | <code>red</code> <code>blue</code> <code>green</code> <code>yellow</code> <code>purple</code> <code>orange</code> <code>pink</code> <code>cyan</code> — UI badge |
| <code>initialPrompt</code> | Auto-submitted first user turn when run as main session via <code>--agent</code> |

Plugin subagents drop three fields silently

For security, plugin-distributed subagents **silently ignore** `hooks` , `mcpServers` , and `permissionMode` . If you need them, copy the agent file into `.claude/agents/` or `~/.claude/agents/` . Plugin subagents appear in `@` -typeahead as `@agent-<plugin>:<name>` .

Three Ways to Invoke a Subagent

Beyond auto-delegation based on `description` , three explicit invocation patterns escalate from one-off to session-wide:

```
# 1. Natural language – Claude usually delegates
> "Use the test-runner subagent to fix failing tests"

# 2. @-mention – guarantees this specific subagent runs once
> @agent-code-reviewer look at the auth changes
> @agent-acme-plugin:linter check src/api/

# 3. Whole-session agent – replaces the default system prompt for the entire session
$ claude --agent code-reviewer

# Pin it for every session in this project (.claude/settings.json):
{ "agent": "code-reviewer" }

# Bonus: list configured subagents
$ claude agents

# Bonus: define ad-hoc subagents for one session via JSON
$ claude --agents '{"reviewer":{"description":"Reviews code","prompt":"You are a senior reviewer.","tools":
```

The full message you type still goes to Claude; `@agent-X` only controls *which* subagent Claude invokes, not the prompt body.

Persistent Memory – Subagents That Learn

Set `memory: project` (recommended default) and Claude Code creates a directory the subagent reads from and writes to across conversations. The first 200 lines (or 25 KB) of `MEMORY.md` are auto-injected into the subagent's system prompt; Read/Write/Edit are auto-enabled regardless of `tools`.

| Scope | Path | Best for |
|---------|------------------------------------|--|
| user | ~/.claude/agent-memory/<name>/ | Cross-project learnings (universal review patterns) |
| project | .claude/agent-memory/<name>/ | Codebase-specific knowledge, shareable via git (recommended) |
| local | .claude/agent-memory-local/<name>/ | Project-specific but private — not committed |

Pattern: ask the subagent to consult and update its memory

Add this to the system prompt body:

Before reviewing, check your MEMORY.md for patterns and recurring issues you've seen in this codebase. After completing a review, append new patterns and useful insights to MEMORY.md so you're sharper next time.

Over a few weeks, the subagent builds up institutional knowledge about your codebase — the parts of the auth module that always come back broken, the conventions teammates keep violating, the gotchas. That's *your* knowledge graph living next to the code.

Foreground / Background / Fork — Three Execution Modes

Subagents run one of three ways:

| Property | Foreground | Background | Fork (/fork) |
|---------------------|---------------------|---|---|
| Blocks main session | yes | no | no |
| Starting context | fresh + Task prompt | fresh + Task prompt | full conversation |
| Permissions | interactive prompts | pre-approved at spawn | interactive prompts |
| Prompt cache | separate | separate | shared with parent |
| Trigger | default behavior | <code>background: true</code> or Ctrl+B | <code>/fork <task></code> when fork mode on |

Foreground: blocks until done, surfaces permission prompts. **Background:** concurrent, permissions pre-approved at spawn (auto-denies anything else), can ask clarifying questions but won't — that tool call fails and it continues. **Fork:** experimental (`CLAUDE_CODE_FORK_SUBAGENT=1` , v2.1.117+), inherits the entire conversation history, shares prompt cache with parent — cheaper for related side tasks.

Useful env vars

- `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS=1` — turn off background mode entirely.
- `CLAUDE_CODE_SUBAGENT_MODEL=haiku` — override model for every subagent invocation in a session.
- `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE=50` — trigger auto-compaction earlier (default 95%).
- `CLAUDE_CODE_FORK_SUBAGENT=1` — enable fork mode (changes behavior of general-purpose subagent and `/fork` command).

What just happened?

You now have the full subagent toolkit: built-in agents covered for common tasks, frontmatter to author your own, three invocation patterns, persistent memory for learning, and three execution modes. The next module wires *policies* on top of all this with hooks.

Hands-On Lab — Build a pii-auditor Subagent for the API

Working in the **PublicRecords API**. You'll create a tightly tool-scoped subagent that scans Java sources, YAML configs, and SQL files for unmasked SSNs / EINs / DOBs, then returns a markdown table of findings. Read-only by design — can't fix, just audit. About 10 minutes.

Step 1 — Author the subagent

HOW IT WORKS

Subagents live in `.claude/agents/` as Markdown files with frontmatter. Critical fields:

- `name` — how you invoke it (`Use the pii-auditor subagent`)
- `description` — lets the parent agent decide when to delegate
- `tools` — tightly scoped. Read+Grep+Glob means the subagent literally CANNOT Edit, Write, or Bash. The OS doesn't enforce this; the agent harness does.
- `model: haiku` — cheap and fast for a focused task
- `memory: project` — findings persist across sessions and ship with the repo

From the project root, make the agents directory:



Create the subagent file:

name: pii-auditor

description: Scans the codebase for unmasked PII (SSN, EIN, DOB) in Java source, application YAML configs, and SQL files. Use proactively before every commit and as a CI gate.

tools: Read, Grep, Glob

model: haiku

memory: project

You are a PII auditor for a US public-records API.

Your task on every invocation:

1. Use Grep to find candidate PII patterns across the working tree in ``src/**/*.{java,yml,yaml,sql,properties}``:
 - `**SSN**`: ``\b\d{3}-?\d{2}-?\d{4}\b``
 - `**EIN**`: ``\b\d{2}-?\d{7}\b``
 - `**DOB literals near identifiers**`: ``\b(19|20)\d{2}-\d{2}-\d{2}\b`` within 5 lines of words like ``dob``, ``birth``, ``birthdate``.
2. For each match, Read 5 lines of context.
3. Skip false positives:
 - Matches inside ``// TEST``, ``// FIXTURE``, or ``// MOCK`` comments
 - Matches under ``src/test/``
 - Matches in ``data.sql`` ONLY if the file's first line says ``-- MOCK DATA`` (otherwise still flag it).
4. Produce a markdown table:


```
`| File:Line | Matched | Risk | Suggestion |`
```

Risk values: HIGH (committed source code logging the value),
MED (fixture or seed data with realistic values),
LOW (matches a pattern but is clearly fake).
5. If no findings, output a single line: `**PII audit clean.**`

Constraints:

- Be terse – no preamble, no narration of process.
- Never propose code edits. You're a read-only auditor; fixes are someone else's job.
- Never read files outside the working tree.

Step 2 — First run on a clean tree

Restart Claude Code, then:

> Use the pii-auditor subagent to scan this codebase.

Expected output: **PII audit clean**. The starter repo's seed data uses fake business names, no SSNs.

Step 3 — Plant a deliberate violation

Create a Java file with a clearly bad logging pattern — we'll use it to verify the auditor catches what it should:

```
JAVA TEST VIOLATION src/main/java/com/publicrecords/api/filing/DebugLog.java

package com.publicrecords.api.filing;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class DebugLog {
    private static final Logger log = LoggerFactory.getLogger(DebugLog.class);

    public static void logDebtor(String ssn) {
        // FIXME pre-launch: replace with Pii.mask()
        log.info("Debtor SSN={}, hardcoded test value 123-45-6789", ssn);
    }
}
```

Step 4 — Re-run the audit

> Use the pii-auditor subagent to scan this codebase.

Expected: a one-row markdown table flagging **DebugLog.java:11** with the SSN, risk HIGH, suggestion to use **Pii.mask()**. The audit should run in under 10 seconds because Haiku is fast and the toolset is minimal.

Step 5 — Verify the tool restriction is real

Try to make the subagent fix the violation:

> Have the pii-auditor fix the issue it just found.

Expected: the subagent refuses, citing that it has only Read/Grep/Glob and cannot edit files. **This is a feature, not a bug.** Auditors that can fix things become enforcers, and enforcers create false confidence. Keep audit and fix in separate roles.

Step 6 — Clean up

```
$ rm src/main/java/com/publicrecords/api/filing/DebugLog.java
$ git add .claude/agents/pii-auditor.md
$ git commit -m "chore(claude): pii-auditor subagent (read-only, project memory)"
```

Stretch — Add a fourth pattern (credit card)

Edit `pii-auditor.md` to add a fourth Grep pattern:

- Card PAN: `\b(?:\d[ -]*?){13,19}\b`, then validate Luhn-checksum candidates inside the subagent's reasoning.

Drop a fake card number into a test file and re-run. Bonus: combine the auditor with the CC7 hook (next module) so commits are blocked when HIGH-risk PII is found.

Lab complete — what you should have

A read-only `pii-auditor` subagent that runs in seconds, costs Haiku-tier tokens, ships with the repo, and flags real violations without ever being able to introduce them. The same shape (tight tool scope + specific task + Haiku) is the template for every focused subagent you'll ever write — lint-fixers, dependency-version checkers, schema-drift detectors. CC7 wires this into a hook so the audit runs automatically.

Knowledge Check

1. You want a subagent that learns about your codebase across sessions, with knowledge shareable to your team via git. Which `memory: value`?

☐ A user

☐ B project

☐ C local

☐ D global

2. The parent session runs in `bypassPermissions` mode. The custom subagent's frontmatter says `permissionMode: default`. What actually happens?

- ☐ A Subagent uses `default` — frontmatter wins.
- ☐ B Subagent inherits `bypassPermissions` from parent — cannot be overridden.
- ☐ C Error — modes conflict.
- ☐ D The stricter of the two wins.

3. You install a plugin that ships a `code-fixer` subagent with `hooks` defined in its frontmatter. The hooks don't fire. Why?

- ☐ A Plugin needs to be enabled.
- ☐ B Plugin subagents silently drop `hooks`, `mcpServers`, and `permissionMode` for security.
- ☐ C Hooks need to be in `settings.json`.
- ☐ D Hooks only work in foreground subagents.

4. You want a subagent to make file edits but in an isolated copy of the repo — so its work doesn't touch your working tree until you approve. Which frontmatter field?

☐ A `permissionMode: plan`

☐ B `isolation: worktree`

☐ C `background: true`

☐ D `tools: Read`

5. Subagent vs Skill vs Main — "verbose research that produces a 5-line summary" goes where?

☐ A Main conversation — you need to see the work.

☐ B Skill running inline — reusable.

☐ C Subagent (or skill with `context: fork`) — verbose work in isolation, summary returns.

☐ D Plan mode in main — describe the work first.

Module Summary

- 5 built-in subagents handle most cases: Explore (search), Plan (research-in-plan-mode), general-purpose (multi-step), statusline-setup, Claude Code Guide.
- Custom subagents at `.claude/agents/<name>.md` with 16 frontmatter fields. Only `name` and `description` required.
- 5-tier scope: Managed > `--agents` CLI > Project > User > Plugin.
- Three invocation patterns: natural language, `@agent-name`, `--agent` for whole-session.
- Persistent memory: `memory: user / project / local`. Project recommended; auto-loads `MEMORY.md`.
- Three execution modes: foreground (default), background (concurrent + pre-approved), fork (shares parent context + cache).
- Plugin subagents silently drop `hooks`, `mcpServers`, `permissionMode`.

- Parent `bypassPermissions` / `acceptEdits` / `auto` force the subagent to inherit them — frontmatter ignored.

CC7: Hooks

Programmatic control over Claude's tool calls. Auto-format on every write, block dangerous commands, route permissions to a central policy server, chain pipelines on subagent stop. 9 lifecycle events; 5 handler types; one settings.json block.

Learning Objectives

- ☐ Recognize the 9 hook events across 6 categories and pick the right one for a goal.
- ☐ Configure each of the 5 handler types: command / http / mcp_tool / prompt / agent.
- ☐ Use exit codes (0 / 2 / other) and JSON output to allow / block / inject context.
- ☐ Wire async + HTTP hooks for centralized policy enforcement.
- ☐ Use `permissionDecision: "defer"` for slow human-in-the-loop approvals.
- ☐ Avoid the four hook hygiene mistakes that quietly break workflows.

What Hooks Are For — Three Real Use Cases

Everyday Analogy

Every hotel room has door cards instead of locks-and-keys. Why? Because the front desk needs to revoke access without fishing for keys, log every entry/exit centrally, and update access rights when a guest extends their stay — all without changing the door itself. The card reader is the *hook* on the door's behavior: it intercepts the action, runs policy, then either allows or denies.

Hooks in Claude Code are exactly this: a thin layer that runs *around* Claude's tool calls. Same Claude, same tools, but with hooks you intercept every call — log it, validate it, inject context, modify the input, or block it outright. The "card reader" runs your code, not Claude's, and Claude doesn't know it's there.

Three concrete cases

- **Auto-format on write.** Every `PostToolUse(Edit|Write)` runs prettier/eslint on the changed file. Claude writes ugly code; you only ever see formatted.
- **Block dangerous commands.** `PreToolUse(Bash)` intercepts `rm -rf`, exits 2 with a stderr message. Claude sees the rejection and tries something else.

- **Route policy to HQ.** `PreToolUse(*)` POSTs every tool call to `https://policy.acme.internal`. Central server logs, decides allow/deny/ask, and Claude obeys. One audit trail across every developer.

Your First Hook — Auto-Format on Write

Drop this into `.claude/settings.json`:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path' < /dev/stdin); npx prettier --write \"$FILE\"
          }
        ]
      }
    ]
  }
}
```

That's it. After every Edit or Write, prettier formats the file. The hook receives JSON via stdin (containing `tool_input.file_path`), extracts the path with `jq`, and runs prettier. The `|| true` stops a prettier failure from blocking Claude.

What just happened?

You added a guarantee to the codebase: every file Claude edits is auto-formatted. No more "format on save" reminders, no more diffs polluted by indentation. The same pattern works for: tests after every commit, type-checking after every edit, audit logs for every Bash call.

All 9 Hook Events — Grouped by Category

Most workflows use 3-4 events. Here's the full surface so you know what's available:

| Category | Event | Can block? (exit 2) | Use case |
|----------|------------------|---------------------|---|
| Session | SessionStart | no | Inject branch/CI/issue context (matchers: startup/resume/clear/compact) |
| | SessionEnd | no | Archive transcript, emit metrics |
| Prompt | UserPromptSubmit | yes | Validate / classify / inject context for typed prompts |
| Tool | PreToolUse | yes | Block / modify tool inputs (precedence: deny > ask > allow) |
| | PostToolUse | no | Lint, format, validate output (feedback to Claude via exit 2) |
| Stop | Stop | yes | Block normal stop — force more iterations |
| | SubagentStop | yes | Block / continue when a subagent (Task tool) finishes |
| Compact | PreCompact | no | Snapshot critical state before summarization (matchers: manual / auto) |
| User | Notification | no | Custom TTS, Slack, push notifications |

The **can-block** column matters: only some events let exit code 2 stop the action. Use `PreToolUse` to block a Bash call; `PostToolUse`'s exit 2 surfaces feedback to Claude after the tool ran (it can't undo the call).

Handler Types — Pick by Style

The Claude Code hook spec uses `type: "command"` — an executable invoked with the hook JSON on stdin. Within that single primitive you can build five distinct enforcement styles, summarized below. (HTTP / MCP / prompt / agent variants are wrappers *around* a `command`-type hook that shells out to `curl`, an MCP client, the `claude -p` CLI, or a subagent runner.)

| Type | Handler | Default timeout | When to pick it |
|----------|---|-----------------|---|
| command | Shell script via stdin/stdout | 600 s | Default; supports <code>shell: bash powershell</code> , <code>async</code> , <code>asyncRewake</code> |
| http | POST to URL with JSON body | 30 s | Centralized policy engine; needs <code>allowedHttpHookUrls</code> + <code>allowedEnvVars</code> |
| mcp_tool | Calls a tool on a configured MCP server | — | Reuse existing MCP capability for enforcement |
| prompt | LLM yes/no judgment | 30 s | Soft semantic checks ("is this prompt off-topic?") |
| agent | Subagent verification with tools | 60 s | Deep policy checks needing file access (PII, license audit) |

Example: prompt-based check

Not every check should be a regex. Use a `prompt`-type hook for semantic decisions:

```
{
  "hooks": {
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "prompt",
            "prompt": "User submitted: '$ARGUMENTS'. Is this prompt asking Claude to do something off-top?",
            "model": "haiku"
          }
        ]
      }
    ]
  }
}
```

If Haiku replies YES, the hook blocks the prompt with a polite redirect. Cheap (Haiku), fast (~30s timeout), and handles fuzziness regex can't.

HTTP Hooks & Defer Permission — The Modern Production Pattern

For organizations: stop scattering hook scripts across every developer's repo. Centralize policy in one HTTP endpoint. Add async + defer for slow human-in-the-loop approvals.

```
{
  "allowedHttpHookUrls": ["https://policy.acme.internal/*"],
  "httpHookAllowedEnvVars": ["POLICY_TOKEN"],
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|Write|Edit",
        "hooks": [
          {
            "type": "http",
            "url": "https://policy.acme.internal/check",
            "headers": { "Authorization": "Bearer $POLICY_TOKEN" },
            "allowedEnvVars": ["POLICY_TOKEN"],
            "async": true,
            "asyncRewake": true,
            "timeout": 5,
            "statusMessage": "Checking corporate policy..."
          }
        ]
      }
    ]
  }
}
```

The endpoint receives the tool-use JSON, runs whatever policy logic, returns a JSON response with `hookSpecificOutput.permissionDecision` set to one of:

- `allow` — tool call proceeds.
- `deny` — blocked, message returned to Claude.
- `ask` — user prompted (escapes auto-approval).
- `defer` — *park the tool call*, returns `stop_reason: "tool_deferred"` ; resume later with `claude --resume <session-id>` after policy decision lands.

Defer requires Claude Code v2.1.89+ in non-interactive (`-p`) mode. Pattern: pre-commit checks send Bash calls to a Slack approval bot; while the SRE thumbs-ups in Slack, the session sits in deferred state; resume re-runs with the decision.

Why HTTP hooks beat shell scripts at scale

Shell scripts: every dev clones the repo, hopes the script runs, has different shell versions. HTTP hook: one server, one truth, one place to update policy, one log destination, one set of metrics. When the security team needs to add "block any commit message containing customer names," they edit the server, not 47 dev machines.

Hook Output Anatomy — Exit Codes & JSON

Hooks communicate with Claude Code two ways: **exit codes** (simple) and **JSON output** (rich). Most events accept both.

| Channel | Meaning | Example |
|------------------------------------|--|---|
| <code>exit 0</code> + plain stdout | Success; stdout shown in transcript as system message | Logging, status messages |
| <code>exit 0</code> + JSON stdout | Success; parse JSON for structured response | Inject context, modify tool input, set permission |
| <code>exit 2</code> | Block (only certain events); stderr fed back to Claude | Refuse <code>rm -rf</code> |
| Other exit code | Non-blocking error; stderr shown in transcript | Hook script crashed |

Common JSON output fields

```
{
  "continue": true,
  "stopReason": "Optional message when continue=false",
  "suppressOutput": false,
  "systemMessage": "Warning shown to user",
  "decision": "block",
  "reason": "Explanation for decision",
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "allow",
    "permissionDecisionReason": "Pre-approved by policy",
    "updatedInput": { "command": "git diff --name-only" },
    "additionalContext": "Note: this run is part of release pipeline"
  }
}
```

Useful hook input fields (always present in stdin)

- `session_id` — current session ID, for log correlation.
- `transcript_path` — path to the JSON-lines conversation history.
- `cwd` — current working directory when the hook fires.
- `hook_event_name` — the event firing this hook (e.g. `PreToolUse`).
- `tool_name` + `tool_input` — on `PreToolUse`.
- `tool_name` + `tool_input` + `tool_response` — on `PostToolUse`.

- `prompt` — on `UserPromptSubmit`.
- `stop_hook_active` — on `Stop` / `SubagentStop` (true if Claude is already continuing because of a stop hook; check this to avoid infinite loops).

Hook Hygiene — Limits That Bite You

Four limits worth memorizing

1. **Output cap: 10,000 chars.** Excess saved to a file with a path stub. Don't `cat huge-log` from a hook.
2. **Deduplication.** Identical command strings or HTTP URLs run only ONCE per event. Don't expect side effects from "two copies."
3. **Stale context on resume.** Mid-session `additionalContext` is saved in the transcript with timestamps and SHAs that go stale on resume. Use `SessionStart` with the `resume` matcher to refresh.
4. **Browse before debug.** Type `/hooks` to see every active hook with source labels (User / Project / Local / Plugin / Session / Built-in). Read-only verification before you start blaming Claude.

Async vs sync — when to use which

`async: true` — hook runs in background, agent doesn't wait. Useful for telemetry, slow logging. **Cannot block.**

`asyncRewake: true` — like async, but if the hook exits 2 the agent gets woken with a system reminder. Lets you do slow validation that can still inject context (just not synchronously).

Default (sync) — agent blocks until hook completes. Use for permission decisions and anything where the next tool call depends on the hook's verdict.

Hands-On Lab — Block Java Edits That Fail Spotless

Working in the **PublicRecords API**. You'll wire a `PostToolUse` hook that runs `mvn -q spotless:check` after every Edit/Write touching a `.java` file, and exit 2 if formatting drifts — injecting a system reminder that forces Claude to fix the code before the next step. About 12 minutes.

Step 1 — Add the Spotless plugin to `pom.xml`

HOW IT WORKS

Spotless is a Maven plugin that enforces code style. `mvn spotless:apply` reformats your Java sources to Google Java Format. `mvn spotless:check` exits non-zero if any file is out of compliance — that exit code is what we'll use to gate Claude's edits.

Add this plugin block inside `<build><plugins>` in your existing `pom.xml`:

```
<plugin>
  <groupId>com.diffplug.spotless</groupId>
  <artifactId>spotless-maven-plugin</artifactId>
  <version>2.43.0</version>
  <configuration>
    <java>
      <googleJavaFormat/>
      <trimTrailingWhitespace/>
      <endWithNewline/>
    </java>
  </configuration>
</plugin>
```

Baseline the existing source (formats everything to Google style), then verify clean:

```
$
```

Step 2 — Write the hook

HOW IT WORKS

Hooks run on lifecycle events — `PostToolUse` fires after every tool call. The `matcher` filters which tool calls trigger the hook (here: only `Edit` and `Write`). The `command` receives the tool call's JSON on stdin. `jq` pulls out the file path; if it's a `.java` file we run Spotless. **Exit code 2 is special** — it tells Claude the action was blocked and surfaces a system reminder.

This file is the same `.claude/settings.json` you started in CC4 — merge the new `hooks` key in alongside `permissions`, don't replace the file:

```
{
  "permissions": { "...": "from CC4" },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); case \"${FILE}\" in *.java) mvn -q spotless:check
          }
        ]
      }
    ]
  }
}
```

Step 3 — Verify the hook is loaded

```
> /hooks
```

Your **PostToolUse** hook should appear with source *"Project."* If it's missing, the JSON didn't parse — most often a trailing comma where you merged with permissions, or unescaped quotes inside the command string.

Step 4 — Make a deliberately bad edit

```
> In FilingController.java, add a new endpoint
  GET /filings/count that returns the total count.
  Use 2-space indentation throughout (don't worry about Google style).
```

Watch what happens: Claude makes the edit, the hook fires, **mvn -q spotless:check** reports formatting drift, the hook exits 2. Claude receives a system reminder like *"PostToolUse hook exited with code 2: Spotless check failed"* and either runs **mvn spotless:apply** or rewrites the method with 4-space indent. Either way, the edit doesn't proceed until the formatter is happy.

Step 5 — Confirm clean edits aren't blocked

> Now add a JavaDoc to the new endpoint method, properly formatted.

Hook fires, Spotless passes, exit 0, no system reminder. Claude continues to the next step (probably running tests). The hook is invisible when things are good — that's the goal.

Step 6 — Inspect the hook's tool ribbon

Look at the tool ribbon on a recent turn. After each **Edit** tool call, there's a hook entry showing the command that ran and its exit code. That's your audit trail — if a teammate ever asks "did Claude really run the formatter on that PR?", point them at the transcript.

Stretch — Make it async + non-blocking

Change the hook entry to add `"async": true, "asyncRewake": true`. Now Spotless runs in the background; Claude continues immediately. If the format check fails, Claude gets woken with the system reminder a few seconds later (instead of being blocked synchronously). Useful when `mvn` warm-up is slow and you don't want to pay the latency on every edit.

Stretch — Wire in the CC6 PII auditor

Add a second hook on `PreToolUse` matching `Bash(git commit:*)` that invokes the `pii-auditor` subagent. If it returns anything other than "PII audit clean", exit 2 to block the commit. You've now got formatter + PII gate, both automatic.

Windows note

Replace the bash-style command with PowerShell:

```
"command": "pwsh -NoProfile -Command \"$j = [Console]::In.ReadToEnd() | ConvertFrom-Json; if ($j.tool_input
```

Lab complete — what you should have

A `PostToolUse` hook that runs Spotless on every Java edit, blocks Claude with exit 2 on drift, and lets clean edits through silently. Plus a working understanding of the three knobs that matter: **matcher** (which tools), **command** **exit code** (allow / block), and **async/sync** (latency tradeoff). CC9 connects an external Postgres MCP server so Claude can query the UCC schema in plain English.

Knowledge Check

1. You want to auto-format every file Claude edits. Which event?

- ☐ A PreToolUse with matcher: Edit|Write
- ☐ B PostToolUse with matcher: Edit|Write
- ☐ C SessionEnd
- ☐ D UserPromptSubmit

2. You want to block any Bash call containing `rm -rf` . Which exit code?

- ☐ A `exit 1` — standard error.
- ☐ B `exit 2` — with stderr explaining why.
- ☐ C `exit 0` with empty output.
- ☐ D `exit 127` — command not found.

3. Your team needs centralized policy: log every Bash call, allow/deny based on a server, audit trail in one place. Which handler type?

- ☐ A command — shell script per dev.
- ☐ B http — POST to a central policy endpoint.
- ☐ C prompt — ask Haiku each time.
- ☐ D agent — subagent for each call.

4. permissionDecision: "defer" — what does it do?

- ☐ A Postpones the decision until the next prompt.
- ☐ B Parks the tool call. Session pauses; resume with `--resume <id>` after policy decision lands.
- ☐ C Same as `ask` — just prompts the user.
- ☐ D Auto-allows after a delay.

5. You configured the same `command: "./check.sh"` hook on both `PreToolUse` and `PostToolUse` for the same matcher. What runs?

- ☐ A Both hooks run, twice per tool call.
- ☐ B Hook deduplication: identical command strings run only ONCE per event — won't double-fire.
- ☐ C Only `Pre` runs; `Post` is ignored.
- ☐ D Error — duplicate config rejected.

Module Summary

- 9 events across 6 categories: Session (`SessionStart` , `SessionEnd`) / Prompt (`UserPromptSubmit`) / Tool (`PreToolUse` , `PostToolUse`) / Stop (`Stop` , `SubagentStop`) / Compact (`PreCompact`) / User (`Notification`).
- 5 handler types: `command` (default, shell), `http` (centralized), `mcp_tool` , `prompt` (LLM), `agent` (subagent verification).
- Communication: exit codes (0 success, 2 block, other = non-blocking error) + optional JSON for rich responses.
- Permission decisions: `allow` / `deny` / `ask` / `defer` (latter requires v2.1.89+ in `-p` mode).
- Async: `async` for fire-and-forget, `asyncRewake` for slow validation that can still inject context.
- Limits: 10K char output cap, hook deduplication per event, stale-on-resume context.
- Browse with `/hooks` ; debug before blame.

CC8: Tool Use Deep Dive

How Claude calls tools at the API level — the request/response loop, tool schemas, the four message-block types, multi-tool orchestration, fine-grained streaming, the built-in text-edit and web-search tools — and how all of this maps to the Claude Code tools you've been using since CC0.

Learning Objectives

- ☐ Trace the four-step tool-use loop: request → tool_use → tool_result → final answer.
- ☐ Write a JSON Schema for a tool that Claude can call reliably.
- ☐ Identify the four content-block types (`text` , `tool_use` , `tool_result` , `thinking`).
- ☐ Orchestrate multiple tools in one conversation, handling parallel tool calls.
- ☐ Use `tool_choice` and `disable_parallel_tool_use` to control Claude's tool-selection behavior.
- ☐ Use the built-in text-edit and web-search server tools (no client-side code).
- ☐ Explain how Claude Code's Bash, Edit, Read tools are tool-use under the hood.

CC1 showed you a single API call. Tool use is what turns one call into an agent. The tool loop is the most important pattern in this entire course.

Why Tool Use

Everyday Analogy

Imagine a doctor diagnosing without ever ordering a test. They guess based on symptoms alone. Sometimes right, often wrong, no way to verify. Now imagine the same doctor with a lab next door: order a CBC, get results back, decide. Slower per question, dramatically more accurate.

Tool use is the lab. Claude alone is the doctor without one — it can only emit text from training. With tools, Claude can *look things up*, *run code*, *edit files*, *query databases*. Each tool call is an order to the lab; the result comes back, Claude integrates it, and answers.

Technical Definition

Tool use is the API mechanism by which Claude can request that the application run a function and return the result. The application defines tools (name, description, input schema), Claude decides when to call them, the application executes them and returns results, Claude integrates the results into its final answer. The loop can repeat for multi-step tasks.

In Claude Code

Every Claude Code action you've seen — reading a file, editing it, running Bash, searching with Grep — is a tool call. The CLI defines those tools, Claude decides which to call, the CLI runs them, and the loop continues until Claude says "done." Understanding tool use is understanding how Claude Code actually works.

The Tool-Use Loop

One end-to-end tool call is a four-step round trip:

THE FOUR-STEP LOOP

STEP 1

Request

App sends user message + tool definitions.

STEP 2

tool_use

Claude responds with a `tool_use` block: `name` + `input`. `stop_reason`: "tool_use" .

STEP 3

App runs tool

App executes the function and prepares a `tool_result` block with the output (or error).

STEP 4

Final answer

App re-sends history (now ending in `tool_result`). Claude integrates and returns text.

Steps 1 & 2 — defining the tool, getting a call

Python

TypeScript

```
tools = [{
    "name": "get_weather",
    "description": "Get current temperature for a city.",
    "input_schema": {
        "type": "object",
        "properties": {
            "city": {"type": "string", "description": "City name"},
            "units": {"type": "string", "enum": ["c", "f"], "default": "c"},
        },
        "required": ["city"],
    },
}]

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    tools=tools,
    messages=[{"role": "user", "content": "What's the weather in Tokyo?"}],
)

print(resp.stop_reason) # "tool_use"
print(resp.content)
# [TextBlock(type="text", text="I'll check the weather."),
#  ToolUseBlock(type="tool_use", id="toolu_01ABC", name="get_weather",
#               input={"city": "Tokyo", "units": "c"})]
```

Steps 3 & 4 — running the tool, sending the result back

```
def get_weather(city: str, units: str = "c") -> dict:
    # Real impl would call a weather API; we fake it here.
    return {"city": city, "temp": 14, "units": units, "condition": "rain"}

# Find the tool_use block
tool_use = next(b for b in resp.content if b.type == "tool_use")
result = get_weather(**tool_use.input)

# Build the next request: append the assistant's prior turn AND a user turn
# whose content is a tool_result block referring to the tool_use_id.
follow_up = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    tools=tools,
    messages=[
        {"role": "user", "content": "What's the weather in Tokyo?"},
        {"role": "assistant", "content": resp.content, # full prior content
        {"role": "user", "content": [
            {
                "type": "tool_result",
                "tool_use_id": tool_use.id,
                "content": str(result),
            }
        ]},
    ],
)
print(follow_up.content[0].text)
# It's currently 14°C and raining in Tokyo.
```

Three things that bite people

- You must echo back the assistant's full prior content — the `tool_use` block has to be in history for Claude to match the `tool_use_id`.
- The `tool_result` goes inside a user-role message, not its own role. Wrap it in `{"role": "user", "content": [{"type": "tool_result", ...}]}`.
- Tool results don't have to be JSON. A string is fine; Claude reads it. But for structured returns, `JSON.stringify` them so Claude can reason about fields.

Tool Schemas — What Makes a Good One

The tool definition Claude sees has three parts: `name`, `description`, `input_schema`. All three matter.

Name — verb_noun, snake_case

Claude picks tools largely on name. `get_weather` is better than `weather` — verbs make the action explicit. Snake case is convention; mixed case works but stay consistent.

Description — the most important field

This is what convinces Claude to *not* hallucinate the answer and call the tool instead. Three things every description should have:

- **What it does** in one sentence.
- **When to use it** — scenarios where this tool beats Claude's own knowledge.
- **What it returns** — shape and meaning of the output.

```
"description": (  
  "Look up an order by its purchase order (PO) number. "  
  "Use this whenever a user mentions a PO# or asks about order status. "  
  "Returns: {po_number, status, eta, line_items}. "  
  "Status is one of: open, shipped, delivered, cancelled."  
)
```

Input schema — JSON Schema, with descriptions per field

Per-field `description` is what teaches Claude how to fill the field correctly. `enum` closes off invalid values. `required` marks must-haves.

```
{  
  "type": "object",  
  "properties": {  
    "po_number": {  
      "type": "string",  
      "pattern": "^PO-[0-9]{6}$",  
      "description": "Purchase order number, format: PO-123456"  
    },  
    "include_line_items": {  
      "type": "boolean",  
      "default": false,  
      "description": "Whether to include the array of items in the order"  
    }  
  },  
  "required": ["po_number"]  
}
```

Tool description = system prompt for the tool

If Claude calls the wrong tool, calls the right tool with wrong inputs, or doesn't call any tool when it should — nine times out of ten the fix is in the description, not the schema. Treat each description as a 3-sentence prompt-engineering exercise.

The Four Content-Block Types

Claude's `content` array can contain four kinds of blocks. You'll see all four once you start using thinking + tools together.

Block type	Where	Carries
<code>text</code>	Assistant or user	Plain text the model wrote (or the user typed).
<code>tool_use</code>	Assistant only	<code>id</code> , tool <code>name</code> , <code>input</code> (matching schema).
<code>tool_result</code>	User only (echoing back)	<code>tool_use_id</code> (matches the <code>id</code>), <code>content</code> (string or array).
<code>thinking</code>	Assistant, when extended thinking enabled	Hidden reasoning trace; you echo it back unmodified.

Handling mixed blocks

Don't assume `content[0]` is what you want. Iterate:

```
for block in resp.content:
    if block.type == "text":
        print("TEXT:", block.text)
    elif block.type == "tool_use":
        print(f"CALLING {block.name}({block.input})")
        run_tool(block)
    elif block.type == "thinking":
        # Echo back unchanged; you can log it for debugging
        pass
```

Errors in tool results

If your tool fails, return a `tool_result` with `is_error: true` and a string explaining what broke. Claude will adapt — e.g. retry with a different argument, or apologize to the user. Don't throw an exception client-side; that aborts the loop.

Multiple Tools & Multi-Turn

You can pass an array of tools and Claude picks per turn. The model can also issue **multiple tool calls in parallel** — one assistant message containing several `tool_use` blocks.

```
tools = [
    {"name": "get_weather", "description": "...", "input_schema": {...}},
    {"name": "get_time", "description": "Current time in a city.", "input_schema": {...}},
]

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    tools=tools,
    messages=[{"role": "user", "content":
        "I'm flying from NYC to Tokyo. What's the weather and local time at both ends?"}],
)
# resp.content may contain 4 tool_use blocks: weather(NYC), weather(Tokyo),
# time(NYC), time(Tokyo). Run them all (ideally in parallel), then send all 4
# tool_result blocks in one user-role follow-up.
```

The full loop until `end_turn`

```
def agent_loop(user_msg: str, tools: list, max_iters: int = 10):
    history = [{"role": "user", "content": user_msg}]
    for _ in range(max_iters):
        resp = client.messages.create(
            model="claude-sonnet-4-6", max_tokens=2048,
            tools=tools, messages=history,
        )
        history.append({"role": "assistant", "content": resp.content})
        if resp.stop_reason != "tool_use":
            return resp # final answer; bail

    # Run every tool_use block, collect tool_result blocks
    results = []
    for block in resp.content:
        if block.type == "tool_use":
            output = dispatch(block.name, block.input) # your dispatcher
            results.append({
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": str(output),
            })
    history.append({"role": "user", "content": results})
    raise RuntimeError("Agent loop exceeded max iterations")
```

What just happened?

You wrote a tiny agent. The pattern: **loop while `stop_reason == "tool_use"`**, **run every `tool_use` block, send all `tool_results` back together**. Always cap iterations — a buggy tool can put you in an infinite loop. CC13 covers this loop pattern in depth.

Tool Choice & Fine-Grained Control

By default, Claude decides whether and which tool to call. The `tool_choice` parameter lets you override:

tool_choice	Behavior	Use when
<code>{"type": "auto"}</code> (default)	Claude picks: zero, one, or many tools.	Normal agent operation.
<code>{"type": "any"}</code>	Claude must call <i>some</i> tool, but picks which.	You know a tool is needed; routing tasks.
<code>{"type": "tool", "name": "X"}</code>	Claude must call exactly tool <code>X</code> .	Forced structured output (CC1).
<code>{"type": "none"}</code>	No tools allowed; text only.	Final summarization step.

Disabling parallel tool calls

By default, Claude can issue several `tool_use` blocks in one assistant message. If your tools have side effects that must run serially (e.g., a state-mutating API), pass:

```
resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    tools=tools,
    tool_choice={"type": "auto", "disable_parallel_tool_use": True},
    messages=[...]
)
```

Fine-grained tool streaming

By default, when streaming with tool use, the SDK buffers the JSON input until complete. **Fine-grained streaming** emits the input incrementally — useful for very large inputs (file edits, long writes) where you want to start downstream work before the input is done.

```
with client.messages.stream(
    model="claude-sonnet-4-6", max_tokens=4096,
    tools=tools,
    extra_headers={"anthropic-beta": "fine-grained-tool-streaming-2025-05-14"},
    messages=[...],
) as stream:
    for event in stream:
        if event.type == "input_json_delta":
            partial_json += event.partial_json
            # You can begin parsing/echoing partial JSON here
```

When fine-grained matters

Mostly for large file edits. The default streaming is fine for 95% of cases. Don't enable fine-grained unless you're seeing user-visible latency from buffering.

Built-In Tools — Text Edit & Web Search

Anthropic ships **server tools** — tools whose execution happens inside Anthropic's infrastructure, not your client. You enable them by referencing them in `tools`; you do *not* implement them yourself.

The text-edit tool

The text-edit tool gives Claude four operations on a text file: `view`, `create`, `str_replace`, `insert`. The model emits a `tool_use` call with the operation; *your* code applies it to a real file and returns the result. (Server tool naming is misleading here — the tool's *schema* is built-in, but execution is still on your side.)

```
tools = [{"type": "text_editor_20250728", "name": "str_replace_based_edit_tool"}]

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=2048,
    tools=tools,
    messages=[{"role": "user", "content":
        "Open ./auth.py, find the function login(), and rename it to authenticate()."}
    ]),
)
# resp.content has a tool_use with operations like:
# {"command": "view", "path": "./auth.py"}
# {"command": "str_replace", "path": "./auth.py",
#   "old_str": "def login():", "new_str": "def authenticate():"}
# Note: tool type/name versioning is model-family specific.
# Sonnet/Opus 4.x: text_editor_20250728 + str_replace_based_edit_tool.
# Older Sonnet 3.5: text_editor_20241022 + str_replace_editor.
```

The web-search tool

Web search *does* run server-side — you don't get to see or implement the search; the result comes back as part of Claude's response, with citations.

```
tools = [{"type": "web_search_20250305", "name": "web_search", "max_uses": 3}]

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=2048,
    tools=tools,
    messages=[{"role": "user", "content":
        "What did Anthropic ship at their last devday?"}],
)
# resp.content includes a tool_use, server-side search runs, then tool_result
# with results, then text with citations. All in one round trip.
```

Server tools cost extra

Server-tool invocations are billed (web search has a per-use fee). Cap with `max_uses` to avoid surprise costs from over-eager Claude. **Always set `max_uses` for `web_search` in production code.**

How Claude Code Uses Tools

Now the payoff: every Claude Code action you've performed is exactly the loop above. The CLI defines a fixed set of tools and runs the loop on your behalf.

CLAUDE CODE'S BUILT-IN TOOLS = YOUR TOOL DEFINITIONS

CLI tool	Schema	Executor
Bash	{command, description, timeout}	Spawns shell, returns stdout/stderr/exit.
Read	{file_path, offset?, limit?}	Reads file with optional pagination.
Edit	{file_path, old_string, new_string, replace_all?}	Performs string replace; errors if not unique.
Grep	{pattern, path?, type?, ...}	Wraps ripgrep, returns matches.
Glob	{pattern, path?}	Filename matching.
WebFetch	{url, prompt}	Fetches + lets Claude extract.

When you use `--debug` or watch transcripts, you'll see the exact `tool_use` / `tool_result` blocks. **Skills, subagents, and MCP servers all add tools to the same loop** — nothing magical, just more entries in the `tools` array Claude sees.

For CC9 (MCP)

An MCP server registers its tools with Claude Code at startup. Once registered, those tools are indistinguishable to Claude from built-in ones — same schema format, same loop. CC9 covers writing your own MCP server with this in mind.

Hands-On Lab — Build a 2-Tool Agent

You'll build a 60-line Python agent that has two tools (`get_lien_count` and `get_filing_dates`) and answers questions about UCC filings. This is the same loop Claude Code runs — written by you, in 60 lines.

Step 1 — Mock data

```
FILINGS = {
    "PO-100001": {"debtor": "Acme LLC", "filed": "2024-03-12", "lien_count": 2},
    "PO-100002": {"debtor": "Beta Inc", "filed": "2024-09-01", "lien_count": 0},
    "PO-100003": {"debtor": "Acme LLC", "filed": "2025-01-04", "lien_count": 5},
}

def get_lien_count(debtor: str) -> dict:
    total = sum(f["lien_count"] for f in FILINGS.values() if f["debtor"] == debtor)
    return {"debtor": debtor, "total_liens": total}

def get_filing_dates(debtor: str) -> dict:
    dates = [f["filed"] for f in FILINGS.values() if f["debtor"] == debtor]
    return {"debtor": debtor, "filing_dates": sorted(dates)}
```


Step 2 — Tool definitions + dispatcher

```
TOOLS = [
    {
        "name": "get_lien_count",
        "description": "Total active liens for a debtor across all UCC filings. "
            "Use when a user asks 'how many liens does X have?'. "
            "Returns {debtor, total_liens}.",
        "input_schema": {
            "type": "object",
            "properties": {"debtor": {"type": "string"}},
            "required": ["debtor"],
        },
    },
    {
        "name": "get_filing_dates",
        "description": "All filing dates for a debtor, ascending. "
            "Returns {debtor, filing_dates: ISO date strings}.",
        "input_schema": {
            "type": "object",
            "properties": {"debtor": {"type": "string"}},
            "required": ["debtor"],
        },
    },
]

def dispatch(name: str, args: dict) -> dict:
    if name == "get_lien_count": return get_lien_count(**args)
    if name == "get_filing_dates": return get_filing_dates(**args)
    return {"error": f"unknown tool {name}"}
```

Step 3 — The loop

```
import json, sys
from anthropic import Anthropic

def agent(user_q: str, max_iters: int = 6) -> str:
    client = Anthropic()
    history = [{"role": "user", "content": user_q}]
    for _ in range(max_iters):
        r = client.messages.create(
            model="claude-sonnet-4-6", max_tokens=1024,
            tools=TOOLS, messages=history,
        )
        history.append({"role": "assistant", "content": r.content})
        if r.stop_reason != "tool_use":
            return next(b.text for b in r.content if b.type == "text")
        results = []
        for b in r.content:
            if b.type == "tool_use":
                out = dispatch(b.name, b.input)
                results.append({
                    "type": "tool_result",
                    "tool_use_id": b.id,
                    "content": json.dumps(out),
                })
        history.append({"role": "user", "content": results})
    return "[max iterations reached]"

if __name__ == "__main__":
    print(agent(" ".join(sys.argv[1:]) or "How many liens does Acme LLC have, and when did they file?"))
```

Step 4 — Run it

```
$ python agent.py "How many liens does Acme LLC have, and when did they file?"
Acme LLC has 7 total active liens, filed on 2024-03-12 and 2025-01-04.
```

Watch what happens: Claude calls both tools in *one* assistant turn (parallel tool use), your dispatcher runs both, sends two tool_results back, Claude composes the answer.

Step 5 — Inspect the loop

Add a print statement after each `r.stop_reason` check. You'll see exactly two iterations — one with `stop_reason: "tool_use"`, one with `stop_reason: "end_turn"`. Two API calls, four tool block exchanges, one final answer.

Lab complete — what you should have

A 60-line Python agent that runs the same loop Claude Code runs internally. You've now seen tool use end-to-end. **Skills, subagents, and MCP servers are different ways of contributing entries to this loop's tools array.** When CC9 (MCP server-building) lands, it'll feel like adding to a loop you already understand.

Knowledge Check

1. Claude responded with `stop_reason: "tool_use"` . What's the next step?

- ☐ A Show the user the tool name and ask if they want to proceed.
- ☐ B Re-send the request with `tool_choice: "none"` .
- ☐ C Run the tool, append the assistant's full prior content + a user-role `tool_result` block, re-send.
- ☐ D Strip the `tool_use` block and send only the text portion as the answer.

2. You're getting wrong tool selections. Most likely cause?

- ☐ A Schema is too strict.
- ☐ B Tool description is vague — doesn't say "when to use it" or "what it returns."
- ☐ C Wrong model.
- ☐ D Temperature too low.

3. Claude responded with TWO `tool_use` blocks in one assistant message. How do you reply?

- ☐ A Send them in two separate follow-up requests, one `tool_result` each.
- ☐ B Run both, send a single user-role message with both `tool_result` blocks in its `content` array.
- ☐ C Pick one, ignore the other.
- ☐ D Concatenate the two outputs into a single `tool_result`.

4. You want Claude to *always* call your `record_signoff` tool. Best param?

- ☐ A `tool_choice: {"type": "auto"}`
- ☐ B `tool_choice: {"type": "any"}`
- ☐ C `tool_choice: {"type": "tool", "name": "record_signoff"}`
- ☐ D `tool_choice: {"type": "none"}`

5. Your tool occasionally fails. What goes back to Claude?

- ☐ A Throw an exception — Claude figures it out.
- ☐ B A `tool_result` block with `is_error: true` and a string explaining the failure.
- ☐ C Skip the `tool_result` entirely; let Claude time out.
- ☐ D An empty `tool_result` with a sentinel value.

Module Summary

- **Tool-use loop:** request → `stop_reason: "tool_use"` → run tool → send back `tool_result` in user role → final answer. Loop until `end_turn`.
- **Tool schema:** `name` (verb_noun), `description` (what + when + returns), `input_schema` (JSON Schema with per-field descriptions).
- **Four block types:** `text`, `tool_use`, `tool_result`, `thinking`. Iterate; don't assume `content[0]`.
- **Multiple tools:** Claude can call several in parallel within one assistant message. Reply with all results in one user message.
- **tool_choice:** `auto` (default), `any` (must call something), `{"type": "tool", "name": ...}` (force one), `none` (text only).
- **Built-in server tools:** text editor (`str_replace_editor`), web search. Cap web_search with `max_uses`.
- **Errors stay inside the loop:** `tool_result` + `is_error: true` + message. Don't throw.
- Claude Code's Bash/Read/Edit/Grep tools are exactly this protocol — CC9's MCP servers add to the same array.

CC9: MCP — Model Context Protocol

Wire GitHub, Postgres, Slack, Jira, Playwright, and your own custom tools into Claude Code as first-class capabilities. The protocol is open, the ecosystem is large, and the security model is least-privilege by default.

Learning Objectives

- ☐ Explain MCP in one sentence and why it matters compared to baked-in tools.
- ☐ Add an MCP server to a project via `.mcp.json` with `stdio` / `http` / `sse` transports.
- ☐ Pick the right scope (project / user / managed) and approve servers without auto-trusting everything.
- ☐ Decide between writing a skill vs adding an MCP server for a given capability.
- ☐ Apply the four-point security checklist before connecting an MCP server in production.
- ☐ Build your own MCP server in Python using `FastMCP`: define tools, resources, and prompts.
- ☐ Inspect a server with the MCP Inspector and understand the JSON-RPC protocol underneath.

What MCP Is

Everyday Analogy

Think of how USB-C ended a war. Before USB-C, every device had its own connector: micro-USB for Android, Lightning for Apple, weird trapezoid for old cameras. Buying a new phone meant rebuying chargers. Now: one cable, every device. The cable doesn't know what it's charging; the device tells it.

MCP is USB-C for AI tools. Before MCP, integrating a new data source meant building one-off plugins for each AI tool: a Cursor plugin, a Copilot plugin, a Claude plugin. Now: build one MCP server, every AI tool that speaks the protocol can use it. Claude Code, Cursor, ChatGPT — same server.

Technical Definition

MCP (Model Context Protocol) is an open standard from Anthropic for connecting AI assistants to external tools and data. An **MCP server** exposes *tools* (functions Claude can call), *resources* (data Claude can read), and *prompts* (templates Claude can invoke). An **MCP client** (Claude Code, in our case) discovers what each server offers, surfaces it to the model, and routes calls.

Servers run as separate processes or HTTP endpoints. Three transports: **stdio** (subprocess via stdin/stdout, the default), **http** (POST to URL), **sse** (Server-Sent Events). Most local servers use **stdio** ; remote servers use **http** or **sse** .

Adding Your First Server — GitHub in 30 Seconds

The GitHub MCP server gives Claude Code first-class access to issues, PRs, branches, file content, comments — anything **gh** can do but as structured tool calls, not shell parsing.

```
# 1. Get a GitHub token with the right scopes
$ gh auth refresh -s repo,read:org
$ gh auth token # copy the token to a secure place

# 2. Drop a .mcp.json in your project root
$ cat > .mcp.json <<'EOF'
{
  "mcpServers": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "$GITHUB_TOKEN" }
    }
  }
}
EOF

# 3. Export the token and start a session
$ export GITHUB_TOKEN=ghp_...
$ claude
```

Claude Code prompts the first time it sees a project-level **.mcp.json**: *"Project wants to start GitHub MCP server — allow / always-allow / deny?"*. Pick "always" for trusted servers; "allow once" for testing. Now you can ask: *"List my open PRs and summarize each in one line."* Claude calls **mcp__github__list_pull_requests** — no shell needed.

Where the config lives

- Project: `.mcp.json` at repo root — checked into git, team-shared.
- User: under `mcpServers` in `~/.claude.json` — personal, all your projects.
- Managed: `managed-mcp.json` in admin-only directory — org-wide.

The Production Ecosystem — What's Out There

Anthropic and the community ship dozens of MCP servers. The ones you'll hit first:

Server	What it adds	Common use
github	Issues, PRs, branches, file content, comments	"Find the bug from issue #1234, fix it, open PR linked to the issue"
filesystem	Sandboxed file ops outside the working dir	"Read my Notes folder for related ideas"
postgres	Read-only SQL via a connection string	"What's the avg order value in the last 30 days?"
slack	Read channels, post messages, search history	"Summarize today's #incidents channel"
jira	Read/comment/transition tickets	"Move ENG-1234 to In Progress and comment with the PR link"
playwright	Drive a real browser (click, type, screenshot, evaluate JS)	"Take a screenshot of staging signup page; describe layout"
memory	Long-lived knowledge graph (entities + relations)	"Remember that AcmeCorp uses our SSO via SAML"
gdrive	Google Drive search and read	"Find the Q3 planning doc and summarize it"
fetch	HTTP fetch with HTML→markdown conversion	"Read the docs at example.com/api and explain"
(your own)	Whatever your team needs	Internal API access, custom DB, deploy controls

Inline servers in subagent frontmatter

From CC6: subagents can declare their own scoped MCP servers. The Playwright pattern:


```
---
name: browser-tester
description: Test features in a real browser
mcpServers:
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
---
Use Playwright tools to navigate, take screenshots, and verify UI behavior.
```

The Playwright server only exists while this subagent is alive — main session never sees it, doesn't pay context cost for tool descriptions.

Server Scoping — Project / User / Managed

MCP scoping is a separate question from *where the .mcp.json file is*. Three settings control it:

Setting	What it does	Use when
<code>enableAllProjectMcpServers</code>	Auto-approve every server in <code>.mcp.json</code> at startup	Personal trust in a project — never recommended for shared repos
<code>enabledMcpjsonServers</code>	Approve only this list from <code>.mcp.json</code>	Selectively trust specific servers in a repo
<code>disabledMcpjsonServers</code>	Block these from <code>.mcp.json</code>	Override a server you don't want despite team trusting it
<code>allowedMcpServers</code> (managed)	Org-wide allowlist of approved server names	Enterprise — restrict what teams can run
<code>deniedMcpServers</code> (managed)	Org-wide blocklist (takes precedence)	Block a server everywhere, even if team adds it
<code>allowManagedMcpServersOnly</code> (managed)	Only respect the managed allowlist; ignore project / user lists	Most-locked-down enterprise config

First-launch trust prompt

When you start a session in a project that has a `.mcp.json` you haven't seen before, Claude Code prompts: "Project defines N MCP servers. Approve?" Pick servers individually — "approve all" is rarely the right answer for an unfamiliar repo. The settings keys above let you persist your decisions.

Skills vs MCP — The Decision

Skills (CC5) and MCP servers both add capability. They're not interchangeable. The decision tree:

Question	Skill	MCP
Mostly a prompt with shell substitution?	YES — perfect for skill	no
Needs persistent connection (DB, browser session)?	no	YES — MCP server
Reusable across many AI tools (Cursor, Copilot, Claude Code)?	no — tool-specific	YES — standard protocol
Should be in your repo, version-controlled?	YES — checks into <code>.claude/skills/</code>	config-only (server lives elsewhere)
Light wrapper around <code>gh</code> / <code>git</code> / <code>npm</code> ?	YES — skill with <code>!\`gh ...\`</code> shell injection	overkill
Stateful long-running connection (Postgres, Redis)?	no	YES — MCP keeps the connection
Calling third-party REST API once-off?	maybe (<code>!\`curl ...\`</code>)	maybe (existing MCP server?)

Rule of thumb

If you're tempted to write an MCP server but it's a thin wrapper around shell commands you already have — **write a skill instead**. Skills are simpler, version with the repo, and don't add a process to manage. Reserve MCP for: stateful connections (DBs, browsers), capabilities other AI tools should also use (publish once, consume everywhere), and anything that genuinely needs the protocol's structured tool/resource/prompt model.

Security Checklist — Before You Connect

Four checks before adding any MCP server

1. **Read the server's source.** NPM-installable servers are arbitrary code running with your privileges. `npm view @whatever/server`, check the publisher, look at the code if it touches anything sensitive (credentials, network, fs).
2. **Use the narrowest possible token.** GitHub MCP wants a PAT? Generate one with only the repos and scopes it needs. Don't reuse your everyday token. Set expiration. Rotate.
3. **Restrict tool surface with permission rules.** An MCP server might expose 20 tools; you might only need 3. Add specific allows in `permissions.allow` and a broad deny: `"deny": ["mcp__<server>__*"]` first, then `"allow": ["mcp__<server>__list_*", "mcp__<server>__read_*"]` for the safe subset.
4. **Audit via hooks.** Add a `PreToolUse` hook with `matcher: "mcp__.*"` that logs every MCP call to your audit pipeline. Lets you verify what's actually being called and detect drift from expected behavior.

Common minefields

- **Postgres MCP with admin connection string.** If a tool exists called `execute_sql` and your token can DROP TABLE ... it can DROP TABLE. Use a read-only DB role.
- **Slack MCP with workspace-wide post permission.** Either narrow the bot's channel access, or set permission rules to allow only `list_*` / `read_*`.
- **Filesystem MCP rooted at `~/`.** Now Claude can read your whole home folder — SSH keys, browser cookies, everything. Always root at a specific safe directory.
- **"Just trust it" trust prompts.** Picking "always allow" on an unfamiliar project's `.mcp.json` means future commits to that file silently expand Claude's powers. Audit changes to `.mcp.json` in PRs.

What just happened?

You learned to plug Claude Code into the wider tool ecosystem without giving it the keys to the kingdom. The pattern: narrow tokens, narrow permissions, narrow scopes, audit via hooks. The same model holds for any future MCP server: trust through least privilege, not blanket allow.

Build Your Own MCP Server

Consuming MCP servers is half the story. The other half is writing one. The good news: with `FastMCP` (the Python convenience layer in the official MCP SDK), a useful server is ~30 lines.

Three things an MCP server can expose

- **Tools** — functions Claude can call. Same shape as built-in CLI tools (CC8). Most servers are mostly tools.
- **Resources** — read-only data Claude can fetch by URI (e.g. `db://orders/123`). Like a tool, but for stable lookups; Claude can pre-load them.
- **Prompts** — reusable prompt templates the user can invoke (surfaced as slash-command-like options). Useful for "summarize ticket" kind of operations.

Minimal server — tools only

```
# server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("ucc_lookup")

@mcp.tool()
def lookup_filing(po_number: str) -> dict:
    """Look up a UCC filing by purchase order number.

    Use whenever a user mentions a PO# or asks about filing status.
    Returns: {po_number, debtor, status, filed_at, lien_count}.
    """
    # Real impl would query Postgres; mock data here:
    return {
        "po_number": po_number, "debtor": "Acme LLC",
        "status": "active", "filed_at": "2024-03-12", "lien_count": 2,
    }

@mcp.tool()
def search_debtor(name: str, limit: int = 10) -> list[dict]:
    """Search filings by debtor name (case-insensitive substring)."""
    return [{"po_number": "PO-100001", "debtor": "Acme LLC"}]

if __name__ == "__main__":
    mcp.run()
```

That's it. Run with `python server.py`; the server speaks MCP over stdio by default. Register with Claude Code:

```
$ claude mcp add ucc-lookup --scope project -- python "$PWD/server.py"
```

Adding resources — URI-addressable data

Resources are content Claude can pull by URI. They don't take parameters; they're identified by their URI pattern.

```
@mcp.resource("filing://{po_number}")
def filing_resource(po_number: str) -> str:
    """Return the filing as JSON, addressable by URI."""
    f = lookup_filing(po_number)
    return json.dumps(f, indent=2)

@mcp.resource("schema://ucc")
def schema_resource() -> str:
    """The full UCC schema, returned as markdown."""
    return open("schema.md").read()
```

Use resources for content that's *looked up rather than computed*: schemas, glossaries, fixed configs, doc files. Tools call them via the URI; Claude Code surfaces them under `/mcp` in the CLI.

Adding prompts — reusable prompt templates

```
@mcp.prompt()
def filing_summary(po_number: str) -> str:
    """Generate a one-paragraph summary of a UCC filing."""
    return f"""Summarize filing {po_number} in 3 sentences:
1) who filed against whom, 2) when, 3) current status.
Tone: factual, no opinions. Cite the filing number."""

@mcp.prompt()
def risk_review(debtor: str) -> str:
    """Review a debtor's UCC risk profile."""
    return f"Pull all filings for <debtor>{debtor}</debtor>. Score lien risk 0-10. Justify."
```

Prompts appear in Claude Code's `/mcp` menu as named, parameterized templates. Useful for codifying recurring workflows so users don't re-type them.

Tools vs resources vs prompts — quick decision

- **Tools:** compute, query, mutate. Has parameters. Claude decides when to call.
- **Resources:** stable read-only content. URI-addressable. User pulls; Claude can pre-load.
- **Prompts:** templates the user picks from a menu. Best for "do X with this data" operations.

Inspecting your server with the MCP Inspector

Before plugging into Claude Code, sanity-check with the official Inspector — a browser tool that connects to your server and lets you call tools, fetch resources, and use prompts in isolation.

```
$ npx @modelcontextprotocol/inspector python server.py
# opens http://localhost:5173 with a UI to your server
# - list tools, run them, inspect the JSON-RPC traffic.
```

If a tool misbehaves in Claude Code but works in the Inspector, the issue is in Claude's tool-selection (prompt the description better) not the server. If it fails in the Inspector too, it's your server logic.

What just happened?

You wrote an MCP server in < 50 lines that exposes tools, resources, and prompts. Anything you can write in Python can now become a Claude Code capability available across all your sessions. **The MCP Inspector is to MCP what curl is to HTTP** — debug everything there first.

Protocol Anatomy — What's Underneath FastMCP

FastMCP hides the wire format. For debugging, knowing what's actually flowing helps. MCP is JSON-RPC 2.0 over a transport (stdio, HTTP, or SSE). Three message kinds:

Method	Purpose	Direction
<code>initialize</code>	Handshake: protocol version, capabilities	client → server
<code>tools/list</code>	List available tools (name, description, schema)	client → server
<code>tools/call</code>	Invoke a tool with arguments	client → server
<code>resources/list</code> , <code>resources/read</code>	List/read resources	client → server
<code>prompts/list</code> , <code>prompts/get</code>	List prompts and instantiate one	client → server
<code>notifications/*</code>	Push messages from server (e.g. log entries)	server → client

What a tool call looks like on the wire

```
// Claude Code -> server
{
  "jsonrpc": "2.0",
  "id": 17,
  "method": "tools/call",
  "params": {
    "name": "lookup_filing",
    "arguments": {"po_number": "PO-100001"}
  }
}

// server -> Claude Code
{
  "jsonrpc": "2.0",
  "id": 17,
  "result": {
    "content": [
      {"type": "text", "text": "{\\"po_number\\": \\"PO-100001\\", ...}"}
    ],
    "isError": false
  }
}
```

Implementing a custom client (when you'd want to)

You won't usually write an MCP *client* — Claude Code is one. But if you're building your own agent that should consume MCP servers, the SDK gives you a client too:

```
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = await session.list_tools()
            print([t.name for t in tools.tools])
            r = await session.call_tool("lookup_filing", {"po_number": "PO-100001"})
            print(r.content)
```

Why care about the protocol?

Two reasons. First, debugging: when a tool misfires, you can trace the actual JSON-RPC and pinpoint whether Claude sent bad args or your server returned bad shape. Second, **portability**: an MCP server built for Claude Code works for any MCP-aware client — Claude Desktop, custom agents, third-party IDEs. You write once, the ecosystem benefits.

Hands-On Lab — Connect a Postgres MCP to the UCC Schema

Working in the **PublicRecords API**. The starter uses H2 in-memory; production uses Postgres. This lab spins up a local Postgres seeded with the same UCC data, registers the official Postgres MCP server with Claude Code, and asks Claude natural-language questions that get answered with real SQL. About 15 minutes (mostly waiting on Docker).

Prerequisites

- Docker Desktop (or any Docker daemon) running.
- Node 20+ (for `npx`, used to run the MCP server).
- The `data.sql` file from the starter repo at `src/main/resources/data.sql`.

Step 1 — Spin up Postgres with the UCC schema

HOW IT WORKS

Production runs on Postgres; H2 was a dev convenience. We spin up a containerized Postgres on port 5433 (so it doesn't collide with anything you already have on 5432), then create the same `filings` table the JPA entity maps to. The Postgres MCP server will connect to this DB and let Claude query it via natural language.

Start the container:

```
$
```

Wait ~5 seconds for Postgres to accept connections, then create the schema by piping this SQL into `psql`:

SQL SCHEMA

paste into: `docker exec -i ucc-pg psql -U postgres`

```
CREATE TABLE filings (  
  id BIGSERIAL PRIMARY KEY,  
  state CHAR(2) NOT NULL,  
  debtor_name TEXT NOT NULL,  
  secured_party TEXT NOT NULL,  
  collateral_description TEXT,  
  filed_at TIMESTAMPTZ NOT NULL,  
  expires_at TIMESTAMPTZ  
);
```


Then load the seed data from CC0:

```
$
```

Verify (should print 8):

```
8
```

Step 2 — Register the Postgres MCP server (project scope)

Project scope means the config commits with the repo so every teammate gets the same connection automatically:

```
$ cd /path/to/PublicRecordsAPI
$ claude mcp add postgres-ucc --scope project \
  -- npx -y @modelcontextprotocol/server-postgres \
  "postgres://postgres:dev@localhost:5433/postgres"
```

This writes to `.mcp.json` in the project root. Inspect it and notice the connection string is in plaintext — **swap it for an env-var reference before committing** (see Step 6).

Step 3 — Verify the server is up

```
> /mcp
```

The list should show `postgres-ucc` with status **connected**. If status is *failed*, the most likely cause is the port (5433 vs 5432) or that `npx` couldn't reach npm. Run `npx -y @modelcontextprotocol/server-postgres ...` manually in a terminal to see the actual error.

Step 4 — Ask Claude in plain English

```
> Top 5 states by number of UCC filings, sorted descending.

> Which secured parties have filings expiring before 2028?
  List them with the debtor and expiration date.

> Find any filing where the collateral description mentions
  "equipment" but not a VIN or schedule reference.
```

Watch the tool ribbon — you'll see `postgres-ucc.query` calls with the SQL Claude generated. **Read the SQL**. If it does something you didn't intend (e.g., joins a table you'd rather it not touch), the MCP server has read access to everything;

you'll want a read-only role next.

Step 5 — Lock down to a read-only role

The default `postgres` superuser is overkill. Create a read-only role and reconnect:

```
$ docker exec -i ucc-pg psql -U postgres <<'SQL'
CREATE ROLE claude_ro WITH LOGIN PASSWORD 'ro-only-pw';
GRANT CONNECT ON DATABASE postgres TO claude_ro;
GRANT USAGE ON SCHEMA public TO claude_ro;
GRANT SELECT ON filings TO claude_ro;
SQL

$ claude mcp remove postgres-ucc --scope project
$ claude mcp add postgres-ucc --scope project \
  -- npx -y @modelcontextprotocol/server-postgres \
  "postgresql://claude_ro:ro-only-pw@localhost:5433/postgres"
```

Now ask Claude to *"insert a fake filing for testing"* — the MCP `query` tool will fail with a permission error. That's the right outcome. Read-only by default, write access only when explicitly required.

Step 6 — Swap the password for an env-var reference

Plaintext credentials in `.mcp.json` get committed to git — bad. The `${VAR}` syntax tells Claude Code to interpolate from your shell environment at startup. Replace the file contents with this:

```
JSON MCP CONFIG .mcp.json

{
  "mcpServers": {
    "postgres-ucc": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres", "${UCC_DB_URL}"]
    }
  }
}
```

Then export the variable in your shell (add to `~/.zshrc` / `~/.bashrc` / Windows env vars to persist):

```
$
```

Restart Claude Code and verify `/mcp` still shows connected. Now you can commit `.mcp.json` safely — teammates supply their own credentials.

Step 7 — Cross-tool query: combine UCC data with code

The real value of MCP is asking questions that span data and code:

```
> Compare the columns in the filings table (via postgres-ucc) with  
  the fields in @src/main/java/com/publicrecords/api/filing/Filing.java.  
  Are they in sync? List any divergence.
```

Claude calls the MCP query tool to introspect the schema, reads the entity, and reports drift. This is the workflow CC14's CI integration extends.

Stretch — Add a second MCP for GitHub

Register the official GitHub MCP, scoped to read-only on this single repo, then ask Claude: *"Find any open issues mentioning UCC schema changes."* The principle is the same — narrow scope, narrow tokens, audit via the tool ribbon.

Lab complete — what you should have

A local Postgres seeded with UCC data, the Postgres MCP server connected via a read-only role, env-var-referenced credentials so `.mcp.json` is safe to commit, and a working pattern for asking Claude data questions in plain English while keeping the blast radius small. CC14 ships all of this through CI/CD.

Knowledge Check

1. Your team needs Claude Code to read live PostgreSQL data and post Slack messages. Skill or MCP?

- ☐ A Two skills — one calling `psql`, one calling Slack `curl`.
- ☐ B MCP — both need stateful connections; `postgres-mcp` + `slack-mcp`.
- ☐ C One skill calling both via shell.
- ☐ D Subagent only, no MCP.

2. You add a third-party MCP server. It exposes 30 tools but you only need 3 (read-only ones). How do you restrict the surface?

- ☐ A Edit the server's source to delete the other 27.
- ☐ B `permissions.deny: ["mcp__server__*"] + allow: ["mcp__server__list_*", "mcp__server__read_*", "mcp__server__get_*"]`
- ☐ C Set `disabledMcpjsonServers: ["server"]`
- ☐ D You can't — MCP is all-or-nothing.

3. `.mcp.json` at project root: who can read it?

- ☐ A Only you — gitignored by default.
- ☐ B The whole team — checked into git like any project file. New members get a trust prompt.
- ☐ C Only managed admins.
- ☐ D Anyone with read access on Anthropic's servers.

4. Your subagent needs Playwright but you don't want Playwright tools cluttering your main session's context. What's the right pattern?

- ☐ A Add it to project `.mcp.json`.
- ☐ B Add it to user `~/.claude.json`.
- ☐ C Inline in the subagent's frontmatter under `mcpServers` — scoped to that subagent only.
- ☐ D Spawn it as a child process from the subagent's prompt.

5. A teammate sets `enableAllProjectMcpServers: true` in their user-level settings. What's the risk?

- ☐ A None — trust prompts are annoying.
- ☐ B Any project they open auto-runs every server in its `.mcp.json` — including ones added in commits they haven't reviewed.
- ☐ C Only servers they've seen before run.
- ☐ D Just slows down session start.

Module Summary

- MCP = open protocol; same server works across Claude Code, Cursor, ChatGPT, custom tools.
- Configure: project `.mcp.json` · user `~/.claude.json` · managed `managed-mcp.json`.
- Transports: `stdio` (subprocess, default), `http`, `sse`.
- Production ecosystem: GitHub, Postgres, Slack, Jira, Filesystem, Playwright, Memory, Drive, Fetch.
- Skill vs MCP: skill for prompt-shaped workflows + shell wrappers; MCP for stateful connections + cross-tool reuse.
- Trust prompt on first project visit; `enableAllProjectMcpServers` bypasses it (don't).
- Security: narrow tokens, restrict tool surface via `permissions`, audit via `PreToolUse` hooks, root filesystem servers carefully.

- Inline `mcpServers` in subagent frontmatter for tools that should NOT pollute main context.

CC10: Claude Features — Thinking, Caching, Citations, Files

The features that make production Claude apps fast, cheap, and trustworthy: extended thinking for hard reasoning, prompt caching for cost (90%+ savings on stable prefixes), image/PDF input, citations for grounded answers, and the code-execution + Files API for running Python sandboxed.

Learning Objectives

- ☐ Enable extended thinking and pick a sensible budget per task type.
- ☐ Add prompt caching to a long system prompt for 90%+ cost reduction on cache hits.
- ☐ Recognize the four caching rules and the cache-control breakpoint limit.
- ☐ Send images and PDFs as message content; understand size and page limits.
- ☐ Get citations on RAG answers without writing the citation logic yourself.
- ☐ Run sandboxed Python via the code-execution tool and persist files via the Files API.

CC8 was Claude reaching out to your tools. CC10 is Claude reaching for capabilities Anthropic ships in the box: hidden reasoning, document inputs, server-side Python, citations.

Extended Thinking

Everyday Analogy

Asking a senior engineer a hard architecture question and getting an immediate yes/no answer is suspicious. Ask the same question and watch them go quiet, draw on a whiteboard for two minutes, then come back with a confident answer — that's better. The thinking didn't happen in the answer; it happened before the answer.

Extended thinking is exactly that. Claude produces a hidden reasoning trace before its visible answer. The trace costs tokens, but on hard tasks (architecture, multi-file refactors, ambiguous spec questions) it raises accuracy materially.

Technical Definition

Extended Thinking is a Claude 4.x feature you enable per request via the `thinking` parameter. The model emits one or more hidden `thinking` blocks (which you echo back in multi-turn but never display to users) before its `text` answer. You set a `budget_tokens` for the thinking; output tokens are capped separately by `max_tokens`.

```
resp = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=4096,
    thinking={"type": "enabled", "budget_tokens": 8000},
    messages=[{"role": "user", "content":
        "We have 200 microservices, 80% Java. Should we migrate auth from JWT "
        "to mTLS service-to-service? Walk through the trade-offs."
    }],
)

for block in resp.content:
    if block.type == "thinking":
        # Don't show users; do log for debugging
        print("[thinking]", block.thinking[:200], "...")
    elif block.type == "text":
        print("[answer]", block.text)
```

Picking a budget

Task type	budget_tokens
Trivial (lookup, classification)	Don't enable
Code review of one PR	2,000–4,000
Multi-file refactor planning	8,000–16,000
Architecture / hard debugging	16,000–32,000

Thinking tokens are billed

Thinking tokens count as *output* tokens at the same rate. A 16K-token thinking budget is real money — only spend it where you'd spend an Opus call anyway. **Never enable thinking on Haiku** — if the task is hard enough to need thinking, it's worth Sonnet or Opus.

In Claude Code

The CLI maps "think", "think hard", "think deeply", "ultrathink" to budgets ~4K / 8K / 16K / 32K respectively. Subagents you write can set a default budget in their frontmatter (CC6). For hooks that need to make a fast decision, do *not* enable thinking — the latency hit is too high.

Prompt Caching — The Single Biggest Cost Lever

Everyday Analogy

Imagine a tax accountant who re-reads your entire 200-page corporate filing every time you ask "what's box 14b?" Slow, and you're billed for every re-read. Now imagine they keep the filing on their desk for 5 minutes after each question — same answer, no re-read fee.

That's prompt caching. The first time you send a long stable prefix (a CLAUDE.md, a system prompt, a document), Claude caches it. Subsequent requests within ~5 minutes referencing the same prefix are billed at **~10% of the input cost**. The cached prefix doesn't count against your TTFT either; cache hits are noticeably faster.

Technical Definition

Prompt caching is a feature where Anthropic stores a prefix of your prompt for a short window (5 minutes default, 1 hour available). On subsequent requests that share that prefix exactly, the cached portion is billed at a fraction of normal input cost. You opt in by marking **cache breakpoints** in your prompt with `cache_control: {"type": "ephemeral"}`.

Caching a long system prompt

```
long_claude_md = open("CLAUDE.md").read() # imagine 5K tokens
schema_docs = open("schema.md").read()    # imagine 3K tokens

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    system=[
        {
            "type": "text",
            "text": long_claude_md + "\n\n" + schema_docs,
            "cache_control": {"type": "ephemeral"}, # ← cache this prefix
        },
    ],
    messages=[{"role": "user", "content": "How should I model a refund?"}],
)
print(resp.usage)
# usage shows: cache_creation_input_tokens (first call), cache_read_input_tokens (later calls)
```

What you save

CACHE MISS (FIRST REQUEST)

8,000 input tokens billed at full rate.

~\$0.024

+ writing the cache adds ~25% one-time premium.

CACHE HIT (SUBSEQUENT)

8,000 input tokens billed at ~10% rate.

~\$0.0024

10× cheaper, faster TTFT.

Numbers are illustrative; check current pricing. The point: **for any long-stable prefix served >5 times in a 5-minute window, caching pays for itself many times over.**

When NOT to cache

- One-shot calls (no second request → you pay the write premium for nothing).
- Prefixes shorter than the minimum cacheable size (Sonnet/Opus: 1024 tokens, Haiku: 2048).

- Prefixes that change between calls — even one differing token busts the cache.

Caching Rules — The Four That Bite

Cache breakpoints look simple. The gotchas are in the matching rules.

Rule 1: Maximum 4 cache breakpoints per request

You can place up to four `cache_control` markers. Use them strategically:

TYPICAL 4-BREAKPOINT LAYOUT

Position	Content	Why cache it
1	Tool definitions	Stable across all turns of a session.
2	System prompt (CLAUDE.md, role)	Stable across the project.
3	Few-shot examples or large document	Stable across many requests.
4	Conversation history up to last user turn	Reused next turn.

Rule 2: Caches match prefix-exactly

Prepend a single space, change one character, even add a newline — the cache misses entirely. **Same input, byte-for-byte, every time** for a hit.

Rule 3: TTL is 5 minutes by default; 1 hour optional

```
{
  "type": "text",
  "text": "...",
  "cache_control": {"type": "ephemeral", "ttl": "1h"} # 1-hour TTL
}
```

1-hour caching costs more on the write (~2× ephemeral). Use it for content reused across hours, like a customer onboarding doc.

Rule 4: Caching only kicks in past a minimum size

Sonnet/Opus: ~1024 tokens. Haiku: ~2048 tokens. Below that, the marker is ignored. **Don't try to cache a 100-token system prompt** — it won't.

Order matters

Caching is prefix-based. If you put a frequently-changing token *before* a stable one, you've broken the cache for everything after. Always: **most stable content first, most variable content last**. System prompt > tools > documents > few-shot > history > current user turn.

Image & PDF Input

Claude 4.x can read images and PDFs as part of any user message. They go in `content` as image blocks alongside text.

Image input

```
import base64
with open("ui-mockup.png", "rb") as f:
    img_b64 = base64.standard_b64encode(f.read()).decode()

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    messages=[{"role": "user", "content": [
        {"type": "image", "source": {
            "type": "base64", "media_type": "image/png", "data": img_b64,
        }},
        {"type": "text", "text": "What React component structure would you propose for this UI?"},
    ]}],
)
```

- Supported types: `image/png`, `image/jpeg`, `image/gif`, `image/webp`.
- Max 5 MB per image, max 100 images per request.
- Alternative source: `{"type": "url", "url": "https:// ..."}` if your image is publicly fetchable.

PDF input

```
with open("design-doc.pdf", "rb") as f:
    pdf_b64 = base64.standard_b64encode(f.read()).decode()

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=2048,
    messages=[{"role": "user", "content": [
        {"type": "document", "source": {
            "type": "base64", "media_type": "application/pdf", "data": pdf_b64,
        }},
        {"type": "text", "text": "Summarize the proposed migration plan in 5 bullets."},
    ]}],
)
```

- Up to 32 MB / 100 pages per PDF.
- Claude reads both the text and embedded images of the PDF.
- For long PDFs, page-range via `cache_control` on the document so a follow-up question doesn't re-bill the PDF.

In Claude Code

You can drag-drop an image into the CLI prompt — it's encoded and sent as an image block automatically. Useful for "what's wrong with this UI?" or "convert this whiteboard sketch to component code." PDFs work the same way.

Citations — Grounded Answers

If you ship a RAG-style feature ("answer based on these docs"), users want to know *which doc* a sentence came from. Citations is a built-in feature where Claude annotates each generated sentence with the source span it came from — without you writing the citation logic.

```

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    messages=[{"role": "user", "content": [
        {
            "type": "document",
            "source": {"type": "text", "data": open("policy.md").read()},
            "title": "Refund Policy v3",
            "context": "Internal policy doc, last updated 2025-08-12",
            "citations": {"enabled": True},    # ← opt in
        },
        {"type": "text", "text": "Can a customer get a refund 35 days after purchase?"},
    ]}],
)

# resp.content has text blocks; each text block can have a `citations` array
# pointing to the document and character ranges that grounded the sentence.
for block in resp.content:
    if block.type == "text":
        print(block.text)
        for cite in (block.citations or []):
            print(f" → {cite.cited_text!r} (chars {cite.start_char_index}-{cite.end_char_index})")

```

What citations get you

- Per-sentence source spans, automatically (no parsing of "[1]" footnotes).
- Reduced hallucination — Claude cannot cite text that isn't in the documents.
- UX surface: highlight the cited span in the original doc on hover.

Citations only over document blocks

You must use the `document` content type with `citations.enabled: true`. Plain text passed in a user message won't get citations. **For Claude Code use cases:** wrap your project's docs as document blocks when you want Claude's answers to cite specific lines.

Code Execution & the Files API

The code-execution tool lets Claude run sandboxed Python. The Files API lets you upload files Claude can read or write into. Together they're how you give Claude "do data analysis on this CSV" or "render this chart" capabilities without you writing the executor.

Code execution tool

```
tools = [{"type": "code_execution_20250522", "name": "code_execution"}]

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=4096,
    tools=tools,
    extra_headers={"anthropic-beta": "code-execution-2025-05-22"},
    messages=[{"role": "user", "content":
        "Generate a histogram of these numbers: 1,1,2,3,3,3,4,4,5,5,5,5,7."
    }],
)
# Claude writes Python that uses matplotlib, the sandbox runs it,
# returns the figure as a tool_result, Claude embeds it in the answer.
```

Files API — persisting inputs/outputs

```
file = client.beta.files.upload(
    file=open("sales.csv", "rb"),
) # one upload, reusable
print(file.id) # "file_abc123"

resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=4096,
    tools=[{"type": "code_execution_20250522", "name": "code_execution"}],
    extra_headers={"anthropic-beta": "code-execution-2025-05-22,files-api-2025-04-14"},
    messages=[{"role": "user", "content": [
        {"type": "container_upload", "file_id": file.id}, # mount file into sandbox
        {"type": "text", "text": "Compute total sales by region. Write the result to result.csv."},
    ]}],
)
# Claude's code_execution writes /mnt/result.csv; you can fetch it via the Files API.
```

Sandbox guarantees

- Ephemeral container, **no internet access** from inside the sandbox.
- Standard data-science Python (numpy, pandas, matplotlib, scipy) preinstalled.
- ~1 GB RAM, 5-min wallclock per call.
- Files persist across tool calls in the same conversation; ephemeral after.

Security model

The sandbox is isolated from your machine and from Anthropic's network. You can safely have Claude execute untrusted code (e.g. user-submitted snippets) as long as you understand: **output is what Claude says it is — trust only what you can verify**. If you upload sensitive data via Files, treat the file ID as a secret — anyone with it can call Claude with that file.

How Claude Code Uses These Features

Feature	Claude Code use
Extended thinking	Triggered by "think" / "ultrathink" keywords; budgets configurable per subagent.
Prompt caching	CLAUDE.md + tool definitions are cached automatically; you don't add <code>cache_control</code> yourself.
Image input	Drag-drop or paste image into the prompt → sent as image block.
PDF input	Same drag-drop. Useful for design docs, RFCs.
Citations	Not exposed in the CLI directly; available if you build a skill that calls the API with documents.
Code execution	Used internally by some skills; you can invoke via a custom skill or MCP server (CC9).

Where you'll add these manually

Skills you write that call the API directly (CC5) and HTTP hooks that call out to Claude (CC7) are where you'll add caching, citations, and thinking budgets explicitly. The CLI handles the basics; for power features, drop down to API.

Hands-On Lab — Cache a Long System Prompt

You'll measure the actual cost difference of prompt caching with a real CLAUDE.md-sized system prompt. Three runs: cold, warm, and broken.

Step 1 — Build a 4K-token system prompt

```
import json
from anthropic import Anthropic

# Pad with realistic CLAUDE.md content. Aim for >1024 tokens (Sonnet minimum).
SYSTEM = ("# Project: PublicRecords\n\n" +
          ("- All amounts in cents.\n- Use Drizzle ORM.\n- Tests must pass.\n" * 200))

client = Anthropic()

def call(question: str):
    return client.messages.create(
        model="claude-sonnet-4-6", max_tokens=128,
        system=[{
            "type": "text", "text": SYSTEM,
            "cache_control": {"type": "ephemeral"}},
        ],
        messages=[{"role": "user", "content": question}],
    )
```

Step 2 — Cold run (creates the cache)

```
r1 = call("How should money be represented?")
print(json.dumps(r1.usage.model_dump(), indent=2))
# Expect: cache_creation_input_tokens > 0, cache_read_input_tokens == 0
```

Step 3 — Warm runs (cache hits)

```
for q in ["Which ORM?", "Do tests need to pass before commit?",
          "What's the unit for amounts?"]:
    r = call(q)
    print(q, "→", r.usage.model_dump())
# Expect: cache_creation_input_tokens == 0, cache_read_input_tokens > 0
```

Step 4 — Bust the cache (rookie mistake)

```
# Add even ONE extra space at the start of SYSTEM
SYSTEM_BROKEN = " " + SYSTEM
r = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=128,
    system=[{"type": "text", "text": SYSTEM_BROKEN,
             "cache_control": {"type": "ephemeral"}}],
    messages=[{"role": "user", "content": "Which ORM?"}],
)
print(r.usage.model_dump())
# Cache miss again; cache_creation_input_tokens > 0
```

Step 5 — Compute the savings

```
from anthropic.types import Usage
def cost(u: Usage, in_per_m=3.0, cache_in_per_m=3.75, cache_read_per_m=0.30,
        out_per_m=15.0):
    return (u.input_tokens * in_per_m
            + (u.cache_creation_input_tokens or 0) * cache_in_per_m
            + (u.cache_read_input_tokens or 0) * cache_read_per_m
            + u.output_tokens * out_per_m) / 1_000_000

# Compare cold first call vs a warm one. Expect ~10x savings on input portion.
```

Lab complete — what you should have

Three sets of usage numbers proving: (1) cold call writes the cache at a 25% premium, (2) warm calls are ~10× cheaper on the cached portion, (3) one extra space busts the cache entirely. **For any skill or hook that reuses a prompt prefix more than 5 times in 5 minutes, caching is free money** — you just have to enable it.

Knowledge Check

1. You have a hook that runs on every file save and needs to make a fast block/allow decision. Should you enable extended thinking?

- ☐ A Yes — better decisions are always worth it.
- ☐ B No — thinking adds latency and tokens; quick verdicts don't need it.
- ☐ C Only on Haiku.
- ☐ D Always at max budget for safety.

2. Your one-shot CLI tool runs Claude once per invocation, no follow-up. Should you cache the system prompt?

- ☐ A Yes — caching is always cheaper.
- ☐ B No — you pay the write premium with no follow-up to amortize it.
- ☐ C Yes, with 1h TTL.
- ☐ D Yes, but only on Opus.

3. Your cached prefix sometimes gets cache hits and sometimes doesn't. Most likely cause?

- ☐ A API outage.
- ☐ B You're concatenating today's date or a request ID into the prefix.
- ☐ C You changed the model.
- ☐ D Caching is non-deterministic.

4. You're building a "summarize this PDF" feature and want users to see source spans. Best approach?

- ☐ A Ask Claude to add "(see page X)" footnotes manually.
- ☐ B Pass the PDF as a `document` block with `citations.enabled: true` and read each text block's `citations` array.
- ☐ C Run a regex post-process on Claude's output.
- ☐ D Don't bother — users trust the model.

5. Your code-execution sandbox call returns "no module named requests." Why?

- ☐ A You forgot to install requests with pip.
- ☐ B The sandbox has no internet access; `requests` is in the bundle but you can't reach external URLs anyway. The error suggests the sandbox version doesn't include it — use `stdlib` `urllib` alternatives, but you still can't reach the internet.
- ☐ C Anthropic disabled it.
- ☐ D Claude is hallucinating.

Module Summary

- **Extended thinking:** hidden reasoning trace before the answer. Budgets 2K–32K. Worth it on hard reasoning, not on hooks or quick lookups.
- **Prompt caching:** mark stable prefixes with `cache_control: {"type": "ephemeral"}`. ~10× cheaper on hits, ~25% more expensive on first write.
- **Caching rules:** max 4 breakpoints; prefix-exact matching; 5-min default TTL (1h available); minimum 1024 tokens (Sonnet/Opus) or 2048 (Haiku).
- **Order:** stable content first (system > tools > docs > few-shot), variable content last.
- **Image & PDF:** base64 in `image` / `document` blocks. PDFs up to 32 MB / 100 pages.
- **Citations:** opt in with `citations.enabled: true` on document blocks. Per-sentence source spans, no parsing.
- **Code execution:** sandboxed Python, offline, with the Files API for inputs/outputs. Use beta header.
- In Claude Code: caching and basic image/PDF are auto. Thinking via "think hard" keywords. Citations + code-exec require dropping to the API.

CC11: Evaluating Skills, Subagents & Prompts

Treat your skills, subagents, and prompts like code: have a test suite. This module covers building a test dataset, writing code-based and model-based graders, and turning evals into a CI gate so prompt regressions can't ship.

Learning Objectives

- ☐ Recognize the four-step eval workflow: dataset, runner, grader, dashboard.
- ☐ Build a 20-case test dataset that catches typical, ambiguous, and adversarial inputs.
- ☐ Decide between code-based and model-based grading; combine them when needed.
- ☐ Write a model-based grader (LLM-as-judge) that produces stable, schema-validated scores.
- ☐ Wire evals into a pre-commit hook and CI to block regressions.

CC2 showed prompt rewrites can lift accuracy 50%. Without an eval, you can't tell if your next rewrite helps or hurts. Evals turn "hope" into a number.

Why Eval, Why Now

Everyday Analogy

Imagine refactoring a 10K-line codebase with no tests. Every change is a roll of the dice. You move fast for a week, then a flaky bug surfaces in production, you don't know which commit caused it, and you spend three days bisecting.

The pain without a test suite: changes that work in your head break in subtle, distant ways. You start being scared to refactor.

Same with prompts. A skill that classifies commits great today might silently regress when you "improve" it tomorrow. Without a test suite for the prompt, the only feedback is angry users. Evals are the test suite.

Technical Definition

A **prompt evaluation** (or *eval*) is an automated test that runs a prompt against a set of inputs with known good outputs and produces a numerical score. The dataset is your unit-test fixtures; the grader is your assertion. A score of "84/100 cases correct" turns prompt engineering from vibes into science.

For Claude Code specifically

Skills, subagents, slash commands, and HTTP hooks are all prompts. As your team grows, multiple people edit them. Without evals, you'll get into "the commit-classifier subagent used to be more accurate" arguments with no way to settle them. **Evals make prompts auditable.**

The Eval Workflow — Four Stages

FROM HUNCH TO CONFIDENCE IN FOUR STAGES

1. DATASET

Test cases

Inputs + expected outputs
(or rubric).

2. RUNNER

Execute

Loop dataset, run prompt
on each input.

3. GRADER

Score

Compare actual vs
expected. Code or LLM.

4. REPORT

Aggregate

Pass rate, regressions, per-
case detail.

Each stage is a separate concern. Datasets evolve slowly. Runners are reusable. Graders are the hard part. Reports answer "did my last change help?"

Building a Test Dataset

Aim for 20–100 cases. Smaller is fine to start; larger if you can afford the model calls. Cover three slices:

- **Typical (60% of cases):** the everyday inputs you expect.
- **Ambiguous (20%):** inputs where reasonable humans disagree on the answer — dataset documents the disambiguation choice.
- **Adversarial (20%):** inputs that have historically broken the prompt or push known weaknesses.

Storage format — JSONL

```
{ "id": "t01", "input": "Add OAuth login endpoint", "expected": "feat", "tag": "typical" }
{ "id": "t02", "input": "Bump axios from 1.6 to 1.7", "expected": "chore", "tag": "typical" }
{ "id": "t03", "input": "Resolve null deref in pricing.ts", "expected": "fix", "tag": "typical" }
{ "id": "a01", "input": "Refactor to add new pricing endpoint", "expected": "refactor", "tag": "ambiguous" }
{ "id": "x01", "input": "Improve", "expected": "chore", "tag": "adversarial" }
```

How to seed the dataset fast

1. **Mine real usage logs.** Pull 50 commits from your repo's history, hand-label them.
2. **Have Claude generate candidates.** Ask Claude for 30 plausible commit messages across categories; you label.
3. **Capture failure cases as you find them.** Every time the skill misfires in real use, add the case to the dataset before fixing.

Don't let Claude both write and grade

If you use Claude to generate test cases *and* Claude to grade them, you're measuring "Claude agrees with itself." That's not informative. **Either you label the dataset, or your grader is code-based.**

Code-Based Grading

Whenever the right answer is a determinable string, number, or shape, grade it with code. Cheap, deterministic, fast.

Exact match

```
def grade_exact(actual: str, expected: str) -> float:
    return 1.0 if actual.strip().lower() == expected.strip().lower() else 0.0
```


Regex / structural

```
import re, json

def grade_json_shape(actual: str, expected_keys: set) -> float:
    try:
        d = json.loads(actual)
    except Exception:
        return 0.0
    return 1.0 if set(d.keys()) == expected_keys else 0.0

def grade_verdict_format(actual: str) -> float:
    return 1.0 if re.match(r"^VERDICT: (ship|hold)\nREASON: .+$", actual.strip()) else 0.0
```

Composite scoring

For richer outputs, multiple checks weighted:

```
def grade_review(actual: str, expected: dict) -> float:
    score = 0.0
    if expected["category"] in actual.lower():
        score += 0.4
    if "file:line" in actual or re.search(r"\.w+:\d+", actual):
        score += 0.3
    if expected["severity"] in actual.lower():
        score += 0.3
    return score
```

When code-based works

Classification, extraction, JSON shape checks, regex format checks, presence/absence of keywords, length bounds, "did Claude call the right tool?" Anything mechanical. Use it whenever you can — it's free, fast, and fully deterministic.

Model-Based Grading (LLM-as-Judge)

For subjective outputs — "is this a good explanation?", "is this code review thorough?" — code-based grading isn't enough. Use a Claude call as the judge, with a strict rubric.

Technical Definition

Model-based grading (LLM-as-judge) uses a separate Claude call to score a candidate output against a rubric and reference. The grader prompt locks the output as a forced tool call (CC8) producing `{score: 0..5, reasoning: ...}` — never as free text, since you'll aggregate the score.

A canonical judge

```
JUDGE_TOOL = [{
    "name": "score_output",
    "description": "Score a candidate output against a reference and rubric.",
    "input_schema": {
        "type": "object",
        "properties": {
            "score": {"type": "integer", "minimum": 0, "maximum": 5,
                      "description": "0=way off, 5=matches reference perfectly"},
            "reasoning": {"type": "string", "description": "1-2 sentence justification"},
        },
        "required": ["score", "reasoning"],
    },
}]

def llm_judge(case_input, candidate, reference, rubric):
    resp = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=512, temperature=0,
        tools=JUDGE_TOOL,
        tool_choice={"type": "tool", "name": "score_output"},
        system=("You are a strict, terse grader. Apply the rubric exactly. "
              "Do NOT be generous. A score of 5 means 'identical to the reference "
              "for all practical purposes.' Most outputs are 2-3."),
        messages=[{"role": "user", "content": (
            f"<rubric>{rubric}</rubric>\n"
            f"<input>{case_input}</input>\n"
            f"<candidate>{candidate}</candidate>\n"
            f"<reference>{reference}</reference>"
        )}],
    )
    judgement = next(b.input for b in resp.content if b.type == "tool_use")
    return judgement["score"] / 5.0, judgement["reasoning"]
```

Five rules for stable judges

1. **Use temperature 0.** Judges that flip scores between runs are useless.

2. **Use a smaller model than the candidate** when possible — an Opus output graded by Sonnet is fine; the reverse risks the judge being outclassed.
3. **Force the score via tool use** — never parse free text.
4. **Anchor with a reference.** "Compare to reference" beats "is this good?"
5. **Tell the judge to be strict.** Default Claude is generous; spell out the calibration ("most outputs should score 2-3").

Validate your judge against humans

Before trusting model-based scores, hand-grade 20 cases yourself. Compare your scores to the judge's. If correlation is below ~0.8, fix the rubric, not the candidate prompt — **a bad judge poisons all downstream conclusions.**

A Complete Eval Runner

Putting it together — a 50-line runner you can adapt:

```

import json, statistics, time
from anthropic import Anthropic
from pathlib import Path

client = Anthropic()

def run_candidate(prompt_template: str, case: dict) -> str:
    """Run the prompt under test against one case input."""
    resp = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=128, temperature=0,
        system=prompt_template,
        messages=[{"role": "user", "content": case["input"]}],
    )
    return resp.content[0].text.strip()

def evaluate(prompt_template: str, dataset_path: str, grader) -> dict:
    cases = [json.loads(l) for l in Path(dataset_path).read_text().splitlines() if l.strip()]
    rows, t0 = [], time.time()
    for c in cases:
        actual = run_candidate(prompt_template, c)
        score = grader(actual, c["expected"])
        rows.append({"id": c["id"], "tag": c.get("tag", ""),
                     "expected": c["expected"], "actual": actual, "score": score})
    elapsed = time.time() - t0
    overall = statistics.mean(r["score"] for r in rows)
    by_tag = {}
    for tag in {r["tag"] for r in rows}:
        s = [r["score"] for r in rows if r["tag"] == tag]
        by_tag[tag] = statistics.mean(s) if s else None
    return {"n": len(rows), "score": overall, "by_tag": by_tag,
           "elapsed_s": round(elapsed, 1), "rows": rows}

if __name__ == "__main__":
    PROMPT = Path("commit_classifier.system.txt").read_text()
    grader = lambda a, e: 1.0 if a.lower().strip() == e.lower() else 0.0
    result = evaluate(PROMPT, "commits.jsonl", grader)
    print(f"Overall: {result['score']:.2f} ({result['n']} cases, {result['elapsed_s']}s)")
    for tag, s in result["by_tag"].items():
        print(f" {tag:<12} {s:.2f}")
    # Persist for diffing across runs
    Path("eval_runs").mkdir(exist_ok=True)
    Path(f"eval_runs/{int(time.time())}.json").write_text(json.dumps(result, indent=2))

```

The output that matters

```
$ python eval.py
Overall: 0.86 (50 cases, 41.2s)
  typical      0.93
  ambiguous    0.78
  adversarial  0.65
```

The breakdown by tag is the most useful number. Improvements should hold across all three tags — if "typical" jumps but "adversarial" drops, you've overfit.

Regression Gates & CI

An eval runner is only useful if it gates merges. Wire it in:

Pre-commit hook (local)

```
# .git/hooks/pre-commit
#!/usr/bin/env bash
set -e

# Only run if a prompt or subagent file changed
if git diff --cached --name-only | grep -qE '\.(system|prompt)\.txt$|^\.claude/skills/'; then
  python evals/run.py 0.85 || {
    echo "Eval below threshold. Use 'git commit --no-verify' to override."
    exit 1
  }
fi
```

GitHub Actions (PR gate)

```
name: prompt-evals
on:
  pull_request:
    paths: ["prompts/**", "agents/**", ".claude/skills/**"]
jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install anthropic
      - name: Run evals (fail if overall < 0.85)
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: python evals/run.py 0.85
```

What "fail" should mean

- **Hard threshold:** overall score must be $\geq X$ (e.g. 0.85). Useful for the first time.
- **Regression delta:** overall score must not drop more than 5% vs `main`. Better long-term — allows quality to ratchet up.
- **Per-tag floor:** no tag may drop below 0.6. Catches "improve typicals at cost of adversarial."

Cost of running evals

50 cases at 200 tokens each on Sonnet ~ \$0.04 / run. CI runs that on every PR is cheap. If your dataset is 1000 cases or you use an Opus judge, costs add up — sample randomly per PR and run the full set on main.

Hands-On Lab — Add an Eval Gate to a Subagent

You'll take the `commit-classifier-v1` subagent from CC2 and add a 20-case eval that runs in CI, blocking any change that drops accuracy more than 5%.

Step 1 — Create the dataset

```
mkdir -p evals
cat > evals/commits.jsonl <<'EOF'
{"id":"t01","input":"Add OAuth login endpoint","expected":"feat","tag":"typical"}
{"id":"t02","input":"Resolve null deref in pricing.ts","expected":"fix","tag":"typical"}
{"id":"t03","input":"Bump axios 1.6 -> 1.7","expected":"chore","tag":"typical"}
{"id":"t04","input":"Update README install steps","expected":"docs","tag":"typical"}
{"id":"t05","input":"Extract auth middleware to module","expected":"refactor","tag":"typical"}
{"id":"a01","input":"Refactor to add new pricing endpoint","expected":"refactor","tag":"ambiguous","note":"refactor to add new pricing endpoint"}
{"id":"a02","input":"Fix typo and add caching layer","expected":"feat","tag":"ambiguous","note":"feat dom: add caching layer"}
{"id":"x01","input":"Improve","expected":"chore","tag":"adversarial"}
{"id":"x02","input":"asdf","expected":"chore","tag":"adversarial"}
{"id":"x03","input":"feat: add docs","expected":"docs","tag":"adversarial","note":"prefix lies, body wins"}
EOF
# (add 10 more – aim for 20 total)
```

Step 2 — The runner

```
# evals/run.py
import json, sys, statistics
from pathlib import Path
from anthropic import Anthropic

client = Anthropic()

PROMPT = Path("agents/commit-classifier-v1.md").read_text().split("---", 2)[-1]
DATA = [json.loads(l) for l in Path("evals/commits.jsonl").read_text().splitlines() if l.strip()]

def classify(msg: str) -> str:
    r = client.messages.create(
        model="claude-haiku-4-5-20251001", max_tokens=8, temperature=0,
        system=PROMPT, messages=[{"role": "user", "content": msg}],
    )
    return r.content[0].text.strip().lower()

scores = []
for c in DATA:
    actual = classify(c["input"])
    ok = actual == c["expected"]
    scores.append({"id": c["id"], "tag": c["tag"], "ok": ok})
    print(f"{c['id']:<5} {c['tag']:<12} {'OK' if ok else 'FAIL'} got={actual} expected={c['expected']}")

overall = sum(s["ok"] for s in scores) / len(scores)
threshold = float(sys.argv[1]) if len(sys.argv) > 1 else 0.85
print(f"\nOverall: {overall:.2f}")
sys.exit(0 if overall >= threshold else 1)
```

Step 3 — Run it

```
$ python evals/run.py 0.85
t01   typical      OK got=feat expected=feat
t02   typical      OK got=fix expected=fix
...
Overall: 0.95
$ echo $?
0
```

Step 4 — Break the prompt to verify the gate

Edit the subagent and remove the `<examples>` block. Re-run.


```
$ python evals/run.py 0.85
...
Overall: 0.65
$ echo $?
1
```

The gate fires: removing examples broke 6 cases, score dropped to 0.65, exit code 1, CI fails. Restore the block; back to green.

Step 5 — Wire into GitHub Actions

Drop the YAML from the "Regression Gates" section into `.github/workflows/prompt-evals.yml`. Add `ANTHROPIC_API_KEY` to repo secrets. Push a PR that touches a prompt file — the eval runs and gates the merge.

Lab complete — what you should have

A subagent under test, a 20-case dataset, a 30-line runner that gates on a threshold, and CI that blocks regressions. **Your prompts are now refactor-safe.** Combine this with CC10's prompt caching and you have a stable, observable, production-grade prompt pipeline.

Knowledge Check

1. You're picking a grader for a "is this commit message a feat/fix/chore/docs/refactor?" classifier. Best choice?

- ☐ A Code-based exact-match — the answer is one of 5 fixed strings.
- ☐ B LLM-as-judge with a rubric.
- ☐ C Human review only.
- ☐ D Doesn't matter.

2. You're using Claude to generate test cases AND Claude to grade them. Why is this a problem?

- ☐ A Token cost is too high.
- ☐ B You're measuring "Claude agrees with itself" — uninformative.
- ☐ C Latency.
- ☐ D Anthropic forbids it.

3. Your model-based judge gives a score of 4.5 one run, 3.0 the next, on the same input. Why?

- ☐ A The candidate output changed.
- ☐ B You forgot to set `temperature=0` on the judge call.
- ☐ C Anthropic released a new model.
- ☐ D Cache miss.

4. Your eval suite passes the overall threshold but the "adversarial" tag drops from 0.7 to 0.4. Should you ship?

- ☐ A Yes — overall passed.
- ☐ B No — adversarial drop suggests overfit; add a per-tag floor to the gate.
- ☐ C Yes — adversarial cases don't matter.
- ☐ D Re-run; might be noise.

5. Eval cost on every PR is hurting your CI budget. What's the best mitigation?

- ☐ A Stop running evals.
- ☐ B Switch judge to Opus 4.7 to be sure.
- ☐ C Sample (e.g. 25 of 1000 cases) per PR; run the full set on main; use Haiku where viable.
- ☐ D Only run evals on Friday.

Module Summary

- Four-stage workflow: **dataset**, **runner**, **grader**, **report**.
- Datasets: 20–100 cases, mix of typical / ambiguous / adversarial. Tag every case.
- Code-based grading first — exact match, regex, JSON shape, presence checks. Cheap and deterministic.
- Model-based grading for subjective outputs. Use temp 0, force score via tool use, anchor with reference, calibrate strictness.
- Validate your judge against humans before trusting it — correlate at ≥ 0.8 or fix the rubric.
- Gate with both **hard threshold** and **regression delta + per-tag floor**. Sample on PRs, full set on main.
- Wire evals into pre-commit and GitHub Actions. Cost is typically $< \$0.10$ / run for small datasets — cheap insurance.

CC12: RAG for Claude Code

Retrieval Augmented Generation explained from a CLI-developer's lens: chunking, embeddings, BM25, hybrid retrieval, and how to wrap any of these as a Claude Code subagent or MCP server tool. Concepts first, then a runnable end-to-end pipeline.

Learning Objectives

- ☐ Explain what RAG is, when to reach for it, and when NOT to.
- ☐ Pick a chunking strategy for code, docs, and structured data.
- ☐ Use embeddings + a vector store to find semantically similar chunks.
- ☐ Use BM25 for keyword-precise lookups; combine with vectors via hybrid search.
- ☐ Wrap a RAG pipeline as a subagent or MCP server tool that Claude Code can call.

CC10 covered citations — how Claude grounds an answer in supplied documents. CC12 covers the step before that: finding the right documents to supply.

Why RAG — And Why Not

Everyday Analogy

A new junior engineer joins your team. Do you (a) make them memorize the codebase before they can write any code, or (b) give them `grep` + good search and let them look things up? Option (a) is impossible at any real scale; option (b) is how everyone actually works.

Claude is like that engineer. Putting your entire codebase in the system prompt is option (a) — expensive, slow, capped by context length. Letting Claude search and pull only relevant chunks is option (b). That's RAG.

Technical Definition

Retrieval Augmented Generation (RAG) is a pattern where, before answering, the system retrieves relevant passages from a knowledge base and includes them in Claude's prompt as context. Two phases: **retrieval** (find relevant passages) and **generation** (Claude reads them and answers). The "augmentation" is the retrieved context injected at runtime.

When RAG wins

- Knowledge base is too large to fit in context (codebase, doc set, ticket history).
- Knowledge changes — you don't want to retrain or redeploy when docs update.
- You need citation: "Claude said X because of doc Y line Z" (CC10).
- You need access control: retrieve only docs the user is allowed to see.

When RAG loses

- Small knowledge base (< 100K tokens) — just put it all in a cached system prompt (CC10).
- Knowledge is in code structure (call graphs, type hierarchies) — AST tools beat embeddings.
- Single-document tasks (summarize this PDF) — just send the document.

For Claude Code

Claude Code already has Read, Grep, and Glob — that's RAG with the codebase as the index and grep/glob as the retriever. It works great for code. RAG with embeddings becomes useful when you want to retrieve **over docs, tickets, runbooks, or other corpora** the CLI doesn't index by default — expose those via a subagent or MCP server.

The RAG Flow

FROM DOCUMENTS TO GROUNDED ANSWER

A. INGEST

Chunk

Split docs into passages.

B. INGEST

Embed

Vectorize each chunk.

C. INGEST

Index

Store vectors + BM25 index.

D. QUERY

Retrieve

Find top-K similar chunks.

E. QUERY

Augment

Insert chunks into prompt.

F. QUERY

Generate

Claude answers with context.

Steps A–C run once when documents change (offline ingest). Steps D–F run per query. Get them right and the rest of the system feels magical; get them wrong and Claude either hallucinates or cites the wrong passage.

Chunking Strategies

Chunks are the "atoms" of retrieval. Too big → you lose precision and stuff context with junk. Too small → you lose surrounding context and miss the answer. Three patterns covering most cases:

1. Fixed-size with overlap (start here)

Slide a window of N tokens across the doc with M tokens of overlap. Simple, robust, works for prose.

```
def chunk_fixed(text: str, size: int = 500, overlap: int = 100) -> list[str]:
    words = text.split()
    chunks = []
    i = 0
    while i < len(words):
        chunk = " ".join(words[i:i + size])
        chunks.append(chunk)
        i += size - overlap
    return chunks
```

Defaults that work: 300–800 tokens per chunk, 50–150 token overlap. Smaller for highly structured docs (FAQs); larger for long-form prose.

2. Semantic chunking (better for docs with structure)

Split on natural boundaries: headings, paragraphs, code-block fences. Keeps related sentences together.

```
import re
def chunk_by_heading(md: str) -> list[str]:
    # Split on H1/H2/H3
    parts = re.split(r"(?=>^#{1,3} )", md, flags=re.MULTILINE)
    return [p.strip() for p in parts if p.strip()]
```

3. Code-aware chunking

Don't split inside a function. Use the language's AST or a tree-sitter parser; chunk per top-level definition (function, class, method).

```
import ast
def chunk_python(src: str) -> list[dict]:
    tree = ast.parse(src)
    chunks = []
    lines = src.splitlines()
    for node in tree.body:
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)):
            start = node.lineno - 1
            end = node.end_lineno
            chunks.append({
                "name": node.name,
                "kind": type(node).__name__,
                "code": "\n".join(lines[start:end]),
            })
    return chunks
```

Always store metadata with each chunk

At minimum: source file, byte range, document title. You'll need this for citations (CC10) and for de-duplicating retrieval results. **Add the metadata at ingest time** — you can't recover it later.

Embeddings & Vector Search

Technical Definition

An **embedding** is a vector (typically 384–3072 floats) that represents the meaning of a piece of text. Texts with similar meaning land near each other in the vector space; cosine similarity ~ 1 means very similar, ~ 0 means unrelated. To find passages relevant to a query, embed the query and find the chunks with the closest cosine similarity.

Anthropic does not currently ship a first-party embedding model — you bring your own. Common choices:

Provider / model	Dim	Notes
Voyage AI <code>voyage-3</code>	1024	Strong general-purpose, recommended in Anthropic docs.
OpenAI <code>text-embedding-3-large</code>	3072	Solid quality, widely available.
Sentence-transformers <code>all-MiniLM-L6-v2</code>	384	Free, runs locally; lower quality but fine for prototypes.
Cohere <code>embed-english-v3</code>	1024	Good quality, batched-friendly.

Minimal vector search end-to-end

```
import numpy as np
import voyageai

vo = voyageai.Client()

def embed(texts: list[str]) -> np.ndarray:
    r = vo.embed(texts, model="voyage-3")
    return np.array(r.embeddings)

# Ingest
chunks = chunk_fixed(open("docs.md").read())
chunk_vecs = embed(chunks) # shape: (N, 1024)

# Query
def search(query: str, k: int = 5) -> list[tuple[float, str]]:
    q = embed([query])[0]
    sims = chunk_vecs @ q / (np.linalg.norm(chunk_vecs, axis=1) * np.linalg.norm(q))
    top = np.argsort(-sims)[:k]
    return [(float(sims[i]), chunks[i]) for i in top]

print(search("how is money represented?"))
```


For real corpora, use a vector database (Qdrant, Weaviate, pgvector, Chroma) instead of in-memory NumPy. Same query interface, persistent + scalable.

Strengths and weaknesses

Embeddings catch *semantic* matches: "how do I refund?" matches a chunk titled "returning a charge" even though no word overlaps. Weakness: they often fail on exact identifiers — class names, error codes, function signatures — because semantic similarity doesn't help you find the exact string.

BM25 — Lexical Search That Still Wins

Embeddings get all the press, but a 25-year-old algorithm called BM25 still beats them on a lot of queries. Don't skip it.

Technical Definition

BM25 is a ranking function from information retrieval: documents are scored by term frequency (how often query terms appear in the document) tempered by document length and inverse document frequency (terms common across all docs are downweighted). It's a refinement of TF-IDF and is what powers most search engines underneath the hood.

Where BM25 beats embeddings

- **Identifiers and codes:** "PO-100023", "ECONNREFUSED", "useState" — exact matches matter.
- **Acronyms:** "UCC", "HIPAA" — embeddings cluster them with similar concepts; BM25 finds the literal token.
- **Long documents:** length normalization in BM25 means you don't get drowned out by huge files.
- **Cold-start:** no model, no embeddings to recompute when docs change — just reindex tokens.

BM25 in 15 lines

```
from rank_bm25 import BM25Okapi
import re

def tokenize(s: str) -> list[str]:
    return re.findall(r"\w+", s.lower())

# Ingest
chunks = chunk_fixed(open("docs.md").read())
bm25 = BM25Okapi([tokenize(c) for c in chunks])

# Query
def bm25_search(query: str, k: int = 5) -> list[tuple[float, str]]:
    scores = bm25.get_scores(tokenize(query))
    top = sorted(range(len(chunks)), key=lambda i: -scores[i])[:k]
    return [(scores[i], chunks[i]) for i in top]

print(bm25_search("PO-100023 status"))
```

Tokenization matters more than the formula

BM25 with bad tokenization (no lowercasing, splits on weird boundaries) is worse than embeddings. Use a real tokenizer for your corpus — for code, snake_case and camelCase need to be split. Many implementations have language-specific tokenizers; use them.

Multi-Index (Hybrid) Pipelines

The professional pattern: run BM25 and vector search in parallel, merge the two ranked lists, optionally rerank the merged list with a cross-encoder. You get keyword precision *and* semantic recall.

HYBRID RETRIEVAL — THE PRODUCTION DEFAULT

Stage	What runs	Output
Parallel retrieve	BM25 (top 50) and vector search (top 50)	Two ranked lists, possibly overlapping
Merge	Reciprocal Rank Fusion (RRF) or weighted score	One ranked list of top 50
Rerank	Cross-encoder (e.g. Voyage rerank-2) on top 50 → top K	Top K (usually 5–10) for the prompt
Augment + generate	Pass top K to Claude as document blocks	Grounded answer with citations

Reciprocal Rank Fusion in 8 lines

```
def rrf(rankings: list[list[int]], k: int = 60) -> list[int]:
    """Merge multiple ranked lists of chunk indices using RRF."""
    scores = {}
    for ranking in rankings:
        for rank, idx in enumerate(ranking):
            scores[idx] = scores.get(idx, 0) + 1.0 / (k + rank + 1)
    return sorted(scores, key=lambda i: -scores[i])

q = "how do I refund an order?"
bm25_top = sorted(range(len(chunks)), key=lambda i: -bm25.get_scores(tokenize(q))[i])[ :50]
vec_top = np.argsort(-(chunk_vecs @ embed([q])[0]))[:50].tolist()
merged = rrf([bm25_top, vec_top])[ :10]
final_chunks = [chunks[i] for i in merged]
```

Reranking

A cross-encoder takes (query, candidate) pairs and outputs a refined relevance score. Slow per-pair, so only used on the top 50 from earlier stages. Voyage AI's `rerank-2` is a common choice; Cohere offers similar.

Stop adding stages when accuracy plateaus

BM25 + vectors is enough for most projects. Add a reranker only if your eval (CC11!) shows it helps. **Every stage adds latency and a failure surface** — only spend the complexity when you've measured the win.

RAG as a Subagent or MCP Tool

For Claude Code specifically, you don't ship "a RAG app" — you ship a *tool* that Claude calls when it needs to look something up. Two patterns:

Pattern A — RAG subagent

The subagent system prompt embeds the retrieval logic. Claude Code passes the user's question, the subagent does the lookup + answer in one step.

```

---
name: docs-qa
description: Answers questions about internal docs. Use whenever a user asks about company policy, runbooks, or onboarding.
model: claude-sonnet-4-6
tools: ["mcp__internal_docs__search_docs"] # MCP tool from Pattern B
---

```

You answer questions strictly from internal documentation.

Process:

1. Call ``search_docs(query)`` to retrieve top 8 passages.
2. If no passage is relevant, say "I don't have that documented" – do not guess.
3. Cite each claim with the doc title. Do not invent doc titles.

Pattern B — RAG as MCP server tool

Wrap retrieval as an MCP tool exposed to Claude Code. CC9 covers MCP server building in detail; here's the tool surface:

```

# mcp_rag_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("internal_docs")

@mcp.tool()
def search_docs(query: str, k: int = 8) -> list[dict]:
    """Search internal docs and return top K relevant passages with citations.

    Use whenever a user mentions runbooks, policies, or onboarding.
    Returns: list of {title, source, snippet, score}.
    """
    bm25_top = bm25_rank(query, top=50)
    vec_top = vec_rank(query, top=50)
    merged = rrf([bm25_top, vec_top][:k])
    return [{"title": meta[i].title, "source": meta[i].path,
            "snippet": chunks[i], "score": float(scores[i])} for i in merged]

if __name__ == "__main__":
    mcp.run()

```

Now any Claude Code session that has this MCP server registered (CC9) gets a `search_docs` tool. Claude decides when to call it. No prompting required to "remember" to look something up — Claude does it on its own when the description matches the user's question.

What just happened?

You saw why RAG, in Claude Code, is just **another tool in the loop** from CC8. The retrieval is your code; the calling is Claude's. The combination — retrieval + grounded generation + citations (CC10) — gives you an auditable knowledge agent in < 200 lines.

Hands-On Lab — A 100-Line RAG MCP Server

You'll build a local MCP server that indexes a directory of markdown files with both BM25 and embeddings, exposes a `search_docs` tool, and registers with Claude Code so any session can answer "what does our doc X say about Y?"

Step 1 — Setup

```
$ mkdir docs-rag && cd docs-rag
$ python -m venv .venv && source .venv/bin/activate
$ pip install "mcp[cli]" rank-bm25 voyageai numpy
$ export VOYAGE_API_KEY="..." # any embedding provider
```

Step 2 — The server

```
# server.py
import os, re, glob
import numpy as np
from rank_bm25 import BM25Okapi
from mcp.server.fastmcp import FastMCP
import voyageai

DOCS_DIR = os.environ.get("DOCS_DIR", "./docs")

# --- Ingest at startup ---
def chunk_md(text, size=500, overlap=100):
    parts = re.split(r"(?=\#{1,3} )", text, flags=re.MULTILINE) or [text]
    out = []
    for p in parts:
        words = p.split()
        i = 0
        while i < len(words):
            out.append(" ".join(words[i:i+size]))
            i += size - overlap
    return out

CHUNKS, META = [], []
for path in glob.glob(f"{DOCS_DIR}/**/*.md", recursive=True):
    text = open(path).read()
    for ch in chunk_md(text):
        CHUNKS.append(ch)
        META.append({"source": path, "title": os.path.basename(path)})

bm25 = BM25Okapi([re.findall(r"\w+", c.lower()) for c in CHUNKS])
vo = voyageai.Client()
VECS = np.array(vo.embed(CHUNKS, model="voyage-3").embeddings)

def rrf(rankings, k=60):
    s = {}
    for r in rankings:
        for rank, idx in enumerate(r):
            s[idx] = s.get(idx, 0) + 1 / (k + rank + 1)
    return sorted(s, key=lambda i: -s[i])

# --- MCP tool ---
mcp = FastMCP("internal_docs")

@mcp.tool()
def search_docs(query: str, k: int = 6) -> list[dict]:
    """Search internal markdown docs by hybrid BM25 + vector retrieval. Returns top-K passages with source

    Use when answering questions about internal documentation, runbooks, policies, onboarding, FAQs.
```

```

"""
bm = sorted(range(len(CHUNKS)),
             key=lambda i: -bm25.get_scores(re.findall(r"\w+", query.lower()))[i][:30])
qv = np.array(vo.embed([query], model="voyage-3").embeddings[0])
vc = np.argsort(-(VECS @ qv) / (np.linalg.norm(VECS, axis=1) * np.linalg.norm(qv)))[0:30].tolist())
top = rrf([bm, vc])[:k]
return [{"title": META[i]["title"], "source": META[i]["source"],
        "snippet": CHUNKS[i][:600]} for i in top]

if __name__ == "__main__":
    mcp.run()

```

Step 3 — Register with Claude Code

```

$ claude mcp add docs-rag --scope project \
  --env VOYAGE_API_KEY="$VOYAGE_API_KEY" \
  --env DOCS_DIR=/path/to/your/docs \
  -- python "$PWD/server.py"

```

Step 4 — Use it

```

$ claude
> What does our refund policy say about refunds older than 30 days?

[Claude calls search_docs("refund policy 30 days")]
[6 chunks returned, top from policies/refunds-v3.md]

According to policies/refunds-v3.md, refunds are not honored after 30 days
unless... [grounded answer with cited source]

```

Step 5 — Add an eval

Take 20 questions you know the answers to. Run them through the search + generate flow. Use CC11's runner to score — "answer contains expected fact" via code grading, or LLM-as-judge for richer scoring. Set a threshold and gate updates to docs or the search code.

Lab complete — what you should have

An MCP RAG server backed by hybrid retrieval, registered with Claude Code, queryable from any session. Plus an eval that ensures changes don't regress retrieval quality. **You can drop this server into any team's Claude Code setup and unlock grounded answers over their docs in an afternoon.**

Knowledge Check

1. Your knowledge base is 50K tokens of internal policy. Should you build a RAG pipeline?

- ☐ A Yes — that's huge.
- ☐ B No — 50K fits in context. Put it in a cached system prompt and skip RAG.
- ☐ C Yes, but only if you reindex weekly.
- ☐ D Doesn't matter.

2. Users search by error codes like "ECONNREFUSED". Embeddings only or BM25 only?

- ☐ A Embeddings only — modern semantic search wins.
- ☐ B BM25 (or hybrid) — exact identifiers favor lexical search.
- ☐ C Doesn't matter.
- ☐ D Use full-text search.

3. Your RAG answers are accurate but the citations don't match the actual source. What's wrong?

- ☐ A The model hallucinated.
- ☐ B You're not storing source metadata with chunks at ingest, so the prompt has no source to cite from.
- ☐ C Embeddings are bad.
- ☐ D Reranker dropped the right chunk.

4. You add a cross-encoder reranker as a third stage. Eval shows zero accuracy improvement and 2× latency. What now?

- ☐ A Keep it — rerankers are best practice.
- ☐ B Remove it — complexity without measured benefit isn't worth shipping.
- ☐ C Switch to a different reranker.
- ☐ D Ignore the eval.

5. You're exposing RAG to Claude Code. Subagent or MCP tool?

- ☐ A Always subagent — it's simpler.
- ☐ B Always MCP — more flexible.
- ☐ C MCP tool if multiple subagents/skills need retrieval; subagent if it's a single self-contained Q&A use case.
- ☐ D Both at the same time.

Module Summary

- RAG = retrieve relevant passages, then generate. Two phases, three ingest steps, three query steps.
- Use RAG when corpus exceeds context, knowledge changes, or you need access control. Skip if < 100K tokens.
- Chunking: fixed-size + overlap (start here), semantic (heading-aware), code-aware (AST-based). Always store metadata.
- Embeddings = semantic similarity. Strong on paraphrase, weak on identifiers. Use any provider; switch is easy.
- BM25 = lexical similarity. Strong on identifiers, codes, acronyms. Tokenization matters more than the formula.
- **Hybrid (BM25 + vectors via RRF)** is the production default. Add a reranker only if eval shows it helps.
- For Claude Code: ship RAG as a subagent (single use case) or MCP tool (multiple consumers). Either way it slots into the CC8 tool-use loop.

CC13: Workflows, Agents & Computer Use

The four canonical patterns for orchestrating multi-step Claude work — chaining, parallelization, routing, the agent loop — plus environment inspection and computer use, the GUI-driving capability that lets Claude operate desktop apps directly.

Learning Objectives

- ☐ Distinguish workflows (deterministic, predictable) from agents (adaptive, looping).
- ☐ Implement the three core workflow patterns: chaining, parallelization, routing.
- ☐ Build an agent loop that picks tools, observes outcomes, and stops correctly.
- ☐ Use environment inspection so Claude probes its world before deciding.
- ☐ Recognize when computer use (screen + mouse + keyboard) is appropriate — and when it isn't.
- ☐ Identify which Claude Code features map to which pattern.

CC8 was one tool call. CC13 is what happens when the task is bigger than one call — and when to use which pattern instead of throwing everything at "an agent."

Workflows vs Agents — The Most Useful Distinction

Everyday Analogy

A workflow is a recipe: step 1, step 2, step 3, done. Anyone can follow it. The recipe doesn't change based on whether the milk smells funny — you just follow the steps.

An agent is a chef. Given a goal ("make dinner") and a kitchen, they decide what to cook, taste as they go, change course if the cream curdles, and stop when the meal is ready. Adaptive. Looping. Sometimes goes wrong in surprising ways.

Both are useful. Recipes are predictable, cheap, debuggable. Chefs handle novel situations and partial information. **Most teams reach for "agent" when "workflow" would be cheaper and more reliable.**

Technical Definition

A **workflow** is a deterministic orchestration of one or more LLM calls in a fixed sequence (or DAG). The control flow is in your code. An **agent** is an LLM running a loop where it picks tools, observes results, and decides what to do next. The control flow is in the model. Workflows are predictable; agents are adaptive.

Property	Workflow	Agent
Control flow	Your code	The LLM
Steps	Known in advance	Decided at runtime
Cost predictability	High	Low (loops can run away)
Debugging	Easy — standard tracing	Hard — reasoning is opaque
Best for	Defined process, repeatable	Open-ended, partial information

Default to workflow

If you can structure the work as a fixed pipeline, do. You'll have less drama, lower bills, and easier debugging.
Reach for an agent only when the steps genuinely can't be enumerated in advance — e.g. open-ended research, multi-file refactors with unknown extent.

Chaining Workflow — Sequential LLM Calls

The simplest workflow: call A's output is call B's input. Use when you can decompose the task into stages, each smaller and more focused than the whole.

CHAINING EXAMPLE: PR TRIAGE

Step	LLM call	Input	Output
1	Summarize	The diff	3-sentence summary
2	Classify	Summary	{kind: feat fix chore, risk: low med high}
3	Route	Classification	Reviewer assignment

```
def triage(diff: str) -> dict:
    summary = client.messages.create(
        model="claude-haiku-4-5-20251001", max_tokens=200, temperature=0,
        system="Summarize a diff in 3 sentences.",
        messages=[{"role": "user", "content": diff}],
    ).content[0].text

    classify = client.messages.create(
        model="claude-haiku-4-5-20251001", max_tokens=100, temperature=0,
        tools=CLASSIFY_TOOL,
        tool_choice={"type": "tool", "name": "classify"},
        messages=[{"role": "user", "content": summary}],
    )
    cls = next(b.input for b in classify.content if b.type == "tool_use")

    return {"summary": summary, **cls}
```

Why chain instead of one big prompt

Each step is smaller, narrower, and easier to evaluate (CC11). Smaller prompts allow cheaper models — the summary can be Haiku even if the final routing needs Sonnet. **Chains are easier to debug** because every intermediate output is visible.

Parallelization Workflow — Many Calls at Once

Two flavors:

1. Sectioning — one task per piece

Split the work, run each piece in its own call, merge the results.

```

import asyncio
from anthropic import AsyncAnthropic

aclient = AsyncAnthropic()

async def review_file(path: str, src: str) -> dict:
    r = await aclient.messages.create(
        model="claude-sonnet-4-6", max_tokens=512, temperature=0,
        system="Review this file. Return JSON: [{line, severity, msg}]",
        messages=[{"role": "user", "content": f"# {path}\n\n{src}"},
    ]
    return {"file": path, "review": r.content[0].text}

async def review_pr(files: dict[str, str]) -> list[dict]:
    return await asyncio.gather(*[review_file(p, s) for p, s in files.items()])

# Sequential: 10 files * 4s = 40s. Parallel: ~5s.

```

2. Voting — same task, multiple calls, majority wins

Run the same prompt N times (different temperatures or seeds), aggregate. Useful for high-stakes classifications where occasional flips matter.

```

async def classify_with_vote(text: str, n: int = 5) -> str:
    async def one():
        r = await aclient.messages.create(
            model="claude-haiku-4-5-20251001", max_tokens=8, temperature=0.3,
            system="Classify into: spam | ham | unsure", messages=[{"role": "user", "content": text}],
        )
        return r.content[0].text.strip().lower()
    votes = await asyncio.gather(*[one() for _ in range(n)])
    return max(set(votes), key=votes.count) # majority

```

Parallel limits

Anthropic enforces per-account rate limits. If you fan out 100 parallel calls, you'll get 429s. Use a semaphore (e.g. `asyncio.Semaphore(20)`) to cap concurrency. For very high volumes, use the **Batch API** — 50% off, 24h SLA, perfect for offline parallel work.

Routing Workflow — First Call Picks the Path

Use one cheap LLM call to classify the request, then dispatch to a specialized handler. Avoids putting every system prompt in front of every request.

ROUTING FOR A CODE-HELP BOT

Class	Handler
"explain code"	Sonnet, system: explainer prompt
"debug error"	Sonnet, with tools: read file, run tests
"refactor"	Opus + extended thinking; multi-file plan
"trivia / capital city"	Haiku, no tools

```
ROUTE_TOOL = [{
    "name": "route",
    "description": "Pick the handler for the user's question.",
    "input_schema": {
        "type": "object",
        "properties": {"handler": {"type": "string",
            "enum": ["explain", "debug", "refactor", "trivia"]}},
        "required": ["handler"],
    },
}]

def handle(user_q: str) -> str:
    r = client.messages.create(
        model="claude-haiku-4-5-20251001", max_tokens=64, temperature=0,
        tools=ROUTE_TOOL, tool_choice={"type": "tool", "name": "route"},
        messages=[{"role": "user", "content": user_q}],
    )
    handler = next(b.input["handler"] for b in r.content if b.type=="tool_use")
    return HANDLERS[handler](user_q)  # dispatch to the right specialist
```

Why routing

Putting all five system prompts in one giant prompt for every request bloats tokens and confuses Claude. A 50-token routing call followed by a focused 4K-token handler call is cheaper, faster, and more accurate. **This is exactly how Claude Code's subagent dispatch works under the hood — CC6.**

The Agent Loop — When Steps Aren't Predictable

Same loop you saw in CC8, formalized:

```
def agent(goal: str, tools: list, max_iters: int = 30, max_cost: float = 1.0) -> str:
    history = [{"role": "user", "content": goal}]
    spent = 0.0
    for step in range(max_iters):
        r = client.messages.create(
            model="claude-sonnet-4-6", max_tokens=4096,
            tools=tools, messages=history,
        )
        history.append({"role": "assistant", "content": r.content})
        spent += estimate_cost(r.usage)
        if spent > max_cost: return "[BUDGET EXCEEDED]"
        if r.stop_reason != "tool_use":
            return next(b.text for b in r.content if b.type == "text")
        results = [{"type": "tool_result", "tool_use_id": b.id,
                    "content": str(dispatch(b.name, b.input))}
                   for b in r.content if b.type == "tool_use"]
        history.append({"role": "user", "content": results})
    return "[MAX ITERS]"
```

The four termination conditions

1. **Natural:** `stop_reason == "end_turn"` — Claude says "done."
2. **Iteration cap:** too many loops — bug in tools or prompt.
3. **Cost cap:** dollar budget exceeded — abort before runaway spend.
4. **Time cap:** wallclock budget exceeded — UX requirement.

Agents need three caps. Always.

An unbounded agent loop is a footgun: a buggy tool returning `{"error": "try again"}` can spin forever, billing the whole time. Iteration + cost + time caps make every agent loop bounded by something. **Pick all three for production.**

Environment Inspection

Agents perform better when they *look around before acting*. Give them tools to inspect the environment — list files, check git status, fetch a URL's status code — and prompt them to use these tools first.


```
SYSTEM = """You are a debugging agent. Before fixing anything:
1. Use list_files to see what's in the project.
2. Use read_file to inspect any file you'll change.
3. Use git_status to confirm clean working tree.
ONLY THEN propose changes."""

# Tools: list_files, read_file, git_status, write_file, run_tests
```

Why it works

An agent without inspection tools either hallucinates the environment or refuses to act. With inspection tools, it grounds itself in reality first. This is exactly the pattern Claude Code itself uses internally — the CLI starts every session with implicit environment awareness (cwd, git status, recent files).

Computer Use — Driving the GUI

Technical Definition

Computer use is a Claude capability where the model takes a screenshot of a desktop or browser, decides where to click and what to type, and emits actions like `{"action": "left_click", "coordinate": [x, y]}` or `{"action": "type", "text": "..."}` . Your client takes the screenshot, executes the action, takes another screenshot, and feeds it back. Same agent-loop pattern as tool use — tools happen to be GUI primitives.

The three computer-use tools

Tool	Purpose
<code>computer_20250124</code>	Screenshot, click, type, scroll, key combos.
<code>text_editor_20250124</code>	Open, view, edit text files (same as in CC8).
<code>bash_20250124</code>	Run shell commands.

Minimal request

```
resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=2048,
    tools=[
        {"type": "computer_20250124", "name": "computer",
         "display_width_px": 1920, "display_height_px": 1080, "display_number": 1},
        {"type": "text_editor_20250124", "name": "str_replace_editor"},
        {"type": "bash_20250124", "name": "bash"},
    ],
    extra_headers={"anthropic-beta": "computer-use-2025-01-24"},
    messages=[{"role": "user", "content":
        "Open Chrome, search for 'anthropic computer use docs', "
        "open the first result, and screenshot the title."}],
)
# Claude emits a sequence of tool_use blocks: take_screenshot, left_click, type, ...
# Your client runs each on the actual desktop and feeds back the next screenshot.
```

When to reach for computer use

- **Yes:** automating apps that have no good API (legacy desktop tools, internal tools without integration), QA testing GUIs, browser flows where Selenium is too brittle.
- **No:** any task with a real API. Computer use is dramatically slower and more error-prone than direct API calls. **If the website has an API, use the API.**

Security: run it in a VM

Claude with computer use can click anywhere, type anywhere, run shell commands. A prompt-injection attack in a webpage can convince it to do unintended things. **Always run computer use inside a sandboxed VM or container with no access to credentials, secrets, or the internet beyond what the task requires.** Anthropic ships a reference Docker image; use it.

Latency & cost reality

A multi-screen task can take 30+ tool round trips, each with a screenshot (large image input). Costs add up fast. For browser automation specifically, prefer Playwright + a small Claude classification call when possible — cheaper, faster, more reliable.

Mapping to Claude Code

The patterns aren't abstract — you've used them all in Claude Code:

Pattern	Where in Claude Code
Agent loop	The CLI itself. Every session is one agent loop until exit.
Routing	Subagent dispatch (CC6) — description matching picks the specialist.
Chaining	Slash commands (CC5) — multi-step prompts that run sequentially.
Parallelization	Background subagents and worktrees (CC14) — multiple Claude sessions simultaneously.
Environment inspection	Built-in: cwd, git status, recent files — CLI surfaces these by default.
Computer use	Not a CLI feature; you can wrap it in an MCP server (CC9) if you need GUI work.

The unifying insight

Every "Claude Code feature" you've learned is one of these patterns specialized for development workflows. CLAUDE.md is environment inspection (Claude inspects rules at session start). Subagents are routing + isolation. Hooks are a workflow gate around the agent loop. **Once you see the patterns, the CC features compose** — and you can build new ones.

Hands-On Lab — Replace an Agent with a Workflow

You'll start with an agent that "reviews a PR" by looping with tools, measure its cost and accuracy, then refactor to a 3-step chain that does the same job. Often the chain wins on both axes — this lab proves it.

Step 1 — The agent baseline

```
# review_agent.py
TOOLS = [
    {"name": "list_files", "description": "List changed files in the PR.",
     "input_schema": {"type":"object","properties":{},"required":[]}},
    {"name": "read_file", "description": "Read a file.",
     "input_schema": {"type":"object","properties":{"path":{"type":"string"}}, "required":["path"]}},
    {"name": "submit_review", "description": "Submit final review.",
     "input_schema": {"type":"object","properties":{"verdict":{"type":"string","enum":["approve","request_changes"]},
     "summary":{"type":"string"}}, "required":["verdict","summary"]}},
]
# Run the loop from CC13's agent() function. Track total tokens + tool calls.
```

Step 2 — The chain alternative

```
# review_chain.py
def review_chain(diff: str, files: dict[str,str]) -> dict:
    # Step A: Summarize the diff (Haiku)
    summary = client.messages.create(
        model="claude-haiku-4-5-20251001", max_tokens=300, temperature=0,
        system="Summarize a PR diff in 5 bullets max.",
        messages=[{"role":"user","content":diff}],
    ).content[0].text

    # Step B: Per-file review in parallel (Sonnet)
    async def review_one(path, src):
        r = await aclient.messages.create(
            model="claude-sonnet-4-6", max_tokens=512, temperature=0,
            system="Review one file. Output: severity bullets only.",
            messages=[{"role":"user","content":f"# {path}\n\n{src}"},
        )
        return r.content[0].text
    file_reviews = asyncio.run(asyncio.gather(*[review_one(p,s) for p,s in files.items()]))

    # Step C: Verdict (Sonnet, structured tool output)
    verdict = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=200, temperature=0,
        tools=VERDICT_TOOL, tool_choice={"type":"tool","name":"submit_review"},
        messages=[{"role":"user","content":
            f"<summary>{summary}</summary>\n<reviews>{file_reviews}</reviews>"},
    )
    return next(b.input for b in verdict.content if b.type=="tool_use")
```

Step 3 — Compare on 10 PRs

Run both on a 10-PR test set (CC11 lab). Track:

- Total tokens (input + output) per PR
- Wallclock time per PR
- Verdict accuracy (vs. your hand labels)

metric	agent	chain

avg tokens	18,400	9,800
avg time	24s	6s (parallel files)
accuracy	82%	86%

Numbers are illustrative but the shape is real: the chain typically halves cost, beats latency badly, and hits or beats accuracy because each smaller call is more focused.

Step 4 — When NOT to switch

If your task involves "follow the trail until you find the bug" — where step 4's existence depends on step 3's output — you can't replace the agent with a chain. Keep the loop, add caps.

Lab complete — what you should have

Two implementations of the same task and a comparison sheet. **The exercise that pays off most: every time you reach for "agent," ask if you can structure it as a chain first.** If yes, you'll likely save half the cost and most of the latency.

Knowledge Check

1. You can decompose your task into 3 fixed steps. Should you build an agent or a workflow?

- ☐ A Agent — agents are more capable.
- ☐ B Workflow — deterministic structure means cheaper, faster, easier to debug.
- ☐ C Doesn't matter.
- ☐ D Both, in parallel.

2. You're fanning out 50 parallel calls and getting 429 errors. Best fix?

- ☐ A Increase max_tokens.
- ☐ B Use a semaphore to cap concurrency, or switch to the Batch API for offline work.
- ☐ C Switch to Opus.
- ☐ D Disable streaming.

3. Your agent loop hangs occasionally. What did you forget?

- ☐ A To set max_tokens.
- ☐ B Iteration / cost / time caps. A buggy tool can spin forever.
- ☐ C To enable streaming.
- ☐ D To use Opus.

4. A user asks "what's the weather in Tokyo?" through your routing-based bot. The router should send it to:

- ☐ A The big-Opus refactor handler.
- ☐ B The debugging handler with full tools.
- ☐ C A specialized weather handler — or the trivia handler if no weather one exists. Definitely not Opus + tools.
- ☐ D The agent loop.

5. You want Claude to fill in a form on a legacy desktop app with no API. What's appropriate?

- ☐ A Build a chain workflow.
- ☐ B Computer use, in a sandboxed VM, with iteration + cost caps.
- ☐ C RAG.
- ☐ D Computer use without a sandbox — just trust Claude.

Module Summary

- **Workflow vs agent:** workflow when steps are knowable; agent when not. Default to workflow.
- **Chaining:** sequential calls, smaller models per step where possible. Output of A is input of B.
- **Parallelization:** sectioning (split work) + voting (same work, multiple times, aggregate). Cap concurrency; use Batch API for big offline.
- **Routing:** cheap classifier picks the specialist. Avoid one-prompt-fits-all bloat.
- **Agent loop:** tool_use loop with three caps — iteration, cost, time. Always.
- **Environment inspection:** give the agent tools to look around; prompt it to do so first.
- **Computer use:** screenshot + click + type. For legacy GUIs and APIs that don't exist. Sandbox in a VM. If the API exists, prefer the API.

- Claude Code maps these patterns: subagents = routing, slash commands = chains, worktrees + background = parallelization, the CLI itself = agent loop.

CC14: Power User & CI/CD

The operator's toolkit: session resume tricks, two-Claude review for shift-left bug catching, worktrees for parallel exploration, headless mode for pipelines, GitHub Actions integration, and the `/loop` + `/schedule` background workflows. Everything that makes Claude Code disappear into your daily flow.

Learning Objectives

- ☐ Resume work three ways (`--continue` , `--resume` , `--resume <session-id>`) and use `/rename` , `/branch` , `/rewind` from inside a session.
- ☐ Trigger extended thinking explicitly (`Option + T` , `ultrathink` , `/effort`) and adjust the budget via `MAX_THINKING_TOKENS` .
- ☐ Run parallel sessions on the same repo with `--worktree` and `.worktreeinclude` .
- ☐ Use the two-Claude review pattern to ship code with significantly fewer escapes.
- ☐ Run Claude Code headless (`-p` , `--output-format json|stream-json` , `--allowedTools` , `--max-turns`) inside GitHub Actions, or via the official `anthropics/claude-code-action@v1` .
- ☐ Pick the right scheduling primitive: `/loop` (in-session monitor) vs `/schedule` (cron-style) vs GitHub Actions vs Routines.

Session Mastery — Beyond `--continue`

CC0 covered the basics: `--continue` , `--resume` , `/rename` . Here's the operator-grade toolkit:

Command	What it does
<code>claude --continue</code>	Reopens most recent session in this project
<code>claude --resume</code>	Picker showing every saved session
<code>claude --resume <id></code>	Reopens by session ID (scriptable)
<code>claude --session-id <uuid></code>	Start a new session with a chosen UUID — lets you script "create or resume"
<code>gh pr view <n> --json ... claude</code>	Bootstrap a session from a PR by piping the diff/comments to <code>claude</code>
<code>/rename "name"</code>	Inside session, give it a memorable name
<code>/branch</code>	Fork the current session into two timelines from this point
<code>/rewind</code>	Roll back to a prior turn — pretend the bad turn never happened

Pro pattern: `/branch` for "let me try a different approach"

You're 12 turns into a refactor and want to try a different design without losing your current state. `/branch` forks the session: now you have two timelines from this point. Try approach A in one, approach B in the other, keep whichever wins.

Picker shortcuts

Inside the `--resume` picker: type to filter, arrow keys to move, `Enter` to open. Use `/rename` liberally so the picker actually helps. A picker with 47 unnamed sessions is useless; with 47 named sessions, it's a search index for your own thinking.

Extended Thinking UX — Triggers & Budget

For complex tasks, **extended thinking** lets Claude reason longer before responding. Five ways to invoke it; one way to budget it.

Trigger	How	When
Always-on	<code>alwaysThinkingEnabled: true</code> in settings.json	You always want max-quality, latency be damned
Per-session	<code>/effort high</code> or <code>/effort max</code>	Switching gears mid-session for a hard task
One-shot	Include word "ultrathink" in your prompt	Just this one tricky turn
Per-skill	<code>effort: max</code> in skill frontmatter, or include "ultrathink" in skill body	One specific workflow needs deep thought every time
UX toggle	<code>Option + T</code> (macOS) / <code>Alt + T</code> (Windows/Linux)	Quickly flip on for next turn
See thinking	<code>Ctrl + 0</code> — verbose mode toggle	You want to see the reasoning, not just the answer

Budget

Set `MAX_THINKING_TOKENS` environment variable (e.g. `32000`) to cap how much Claude can reason. Higher = better answers on hard problems, more tokens spent. Default is moderate; turn up for genuine architecture decisions, down for chatty tasks.

When extended thinking helps vs hurts

Helps: ambiguous architectural decisions, debugging non-obvious failures, designing schemas, writing complex regex, security review.

Hurts: simple file edits, formatting, doc fixes — you pay tokens and latency for no quality gain. Match the trigger to the task; don't leave `alwaysThinkingEnabled` on for everything.

Worktrees — Parallel Sessions on the Same Repo

Git worktrees let you check out multiple branches simultaneously into different directories — same repo, multiple working copies. Claude Code's `--worktree` flag spins up a session in a temporary worktree:

```
# Open a session in an isolated copy of the current branch
$ claude --worktree

# Or branch off main, isolated
$ claude --worktree --branch main

# Subagents can use the same primitive via frontmatter:
# isolation: worktree
```

Why isolated worktrees beat just opening two terminals

- Two Claude sessions on the same files would race each other — race conditions on edits, confusing test runs.
- Worktree = independent file system view. Each session edits its own copy.
- Auto-cleaned: if a worktree-isolated session makes no changes, the worktree is deleted on exit.

`.worktreeinclude` — copy gitignored files in

Worktrees only get *tracked* files by default. Your `.env`, `node_modules`, build artifacts — gone. Drop a `.worktreeinclude` at repo root to specify gitignored files that should be copied or symlinked into worktrees:

```
.env.local
.env.development
.vscode/settings.json
```

For larger artifacts (node_modules, .cache), use `worktree.symlinkDirectories` in settings.json:

```
{
  "worktree": {
    "symlinkDirectories": ["node_modules", ".cache", ".next"],
    "sparsePaths": ["packages/api", "packages/web"]
  }
}
```

`sparsePaths` uses git sparse-checkout to only materialize specific directories — useful for huge monorepos where you only need to work on one package.

Two-Claude Review — Shift-Left Bug Catching

Everyday Analogy

Why don't engineers self-review their own code in PRs? Because they wrote it — they have all the context, all the assumptions, all the rationalizations. They *know* why this looks weird, so it doesn't look weird. A second pair of eyes (a teammate's) sees the code *without* the author's reasoning, which is exactly the perspective that catches bugs.

Same problem with one Claude session generating *and* reviewing. The reviewing turn has the generation reasoning in its context window — it's biased toward agreeing. Two separate Claude sessions, one generates, one reviews, with no shared context, get you the second-pair-of-eyes effect.

The pattern

```
# Session A: generate
$ cd ~/project
$ claude
> "Use the code-generator subagent. Implement the new pricing tier
  (free/pro/enterprise) as described in PRICING.md. Add tests.
  When done, output a summary of changes."

# Session A produces a PR. Now switch to Session B (different terminal):
$ cd ~/project
$ claude --worktree --branch main
> "Use the code-reviewer subagent to review PR #1234.
  Look for security issues, edge cases, missed test coverage,
  and consistency with existing patterns.
  Output: CRITICAL / WARNING / SUGGESTION."

# Session B sees ONLY the PR diff – no generation reasoning.
# Subagents in .claude/agents/ are invoked via the Task tool, not a CLI flag.
```

Bonus: run them in parallel via worktrees, fork mode, or scheduled jobs. The generator finishes, the SubagentStop hook (CC7) auto-spawns the reviewer with the same task description — entirely automated pipeline.

Why this matters

Teams using two-Claude review report fewer bugs reaching production by something like 30-50% — not because Claude is smarter, but because the second instance has none of the first instance's confirmation bias. Cheap second opinion. Run it on every non-trivial PR.

Background Work — /loop and /schedule

Two primitives for "do this without me watching":

/loop — in-session background monitor

Bundled skill (CC5). Reruns a check on a cadence, reports drift. Common uses:

```
> /loop every 5 minutes: run `pnpm test:e2e:smoke`. Report only when output changes.

> /loop until error rate > 1%: hit /healthz on staging every 30s. Alert on Slack if it spikes.

> /loop until merged: poll PR #123, when checks turn green and reviews approved, post in #releases.
```

The loop runs in the same session, doesn't block the main conversation, and you can ask it to stop with `"/loop stop."`

`/schedule` — cron-style background tasks

For "run this every weekday morning" / "run this nightly" workflows. The agent fires up automatically, runs the task, reports back next time you're online.

```
> /schedule weekdays at 9am: run yesterday's PR triage report

> /schedule every Monday 10am: clean up branches merged >30 days ago, open a PR with the deletes

> /schedule daily at 6am: pull latest main into all my long-lived feature branches, report conflicts
```

Picking the right primitive

Need	Use	Notes
Watch a value during a session	<code>/loop</code>	Same session, doesn't survive shutdown
Recurring task over weeks/months	<code>/schedule</code> or Routines	Persists across restarts
Triggered by repo events	GitHub Actions	Use Claude Code in headless mode
Triggered by external system	Webhook + headless <code>claude -p</code>	Generic glue
UI-driven background	Desktop notifications hook	Native toast on macOS/Linux/Windows

Headless Mode — `-p` , JSON, Pipes

Claude Code can run as a Unix utility. `-p` (or `--print`) takes a prompt, runs to completion, prints the result, exits. Three output formats: `text` (default), `json` (single JSON envelope), and `stream-json` (line-delimited JSON events).

```

# Simple text response
$ claude -p "Summarize the last 5 commits in this repo"

# Structured JSON output (single envelope)
$ claude -p "List exported functions in src/api.ts" --output-format json

# Stream JSON – one JSON event per line, suitable for live processing
$ claude -p "Analyze this PR" --output-format stream-json --verbose

# Steer the response shape via the system prompt (real flag – works today)
$ claude -p "Analyze this PR. Return JSON with keys: summary (string), risk (one of low|medium|high), iss
    --output-format json \
    --append-system-prompt "Always return valid JSON matching the requested schema. No prose."

# Pipe input/output
$ git diff main | claude -p "Summarize this diff in 3 bullets"

$ claude -p "Generate test cases for this fn" < src/utils.ts > tests/utils.test.ts

```

Useful flags

- `-p <prompt>` / `--print` — non-interactive, prints result, exits.
- `--output-format text|json|stream-json` — `text` (default), `json` (single envelope), `stream-json` (line-delimited JSON events; pair with `--verbose`).
- `--system-prompt <text>` / `--append-system-prompt <text>` — replace or append to the system prompt for this run. Use the appended prompt to constrain output shape.
- `--max-turns <n>` — cap agent loop turns (CI-friendly bounded runs).
- `--mcp-config <path>` — load an MCP server config just for this run.
- `--session-id <id>` / `--resume <id>` — identify or resume a session (scriptable).
- `--permission-mode acceptEdits` — auto-accept file edits; other tools still prompt unless allowlisted. (Other modes: `default`, `plan`, `bypassPermissions`.)
- `--allowedTools` / `--disallowedTools` — explicit allow/deny lists for tools in this run.
- `--dangerously-skip-permissions` — *only inside disposable containers*; bypasses all prompts.

GitHub Actions Integration — Claude in Your Pipeline

Claude Code runs in GitHub Actions same as locally. Two patterns: (1) the official `anthropics/claude-code-action` for chat-style PR review, (2) the raw `claude -p` headless CLI for full scripting control. Pattern 2 below — every PR triggers a workflow that uses Claude Code to review, suggest, or block.

```
# Option A: official GitHub Action (simplest)
# .github/workflows/claude-action.yml
name: Claude PR Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
      contents: read
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - uses: anthropics/claude-code-action@v1
        with:
          # Use either CLAUDE_CODE_OAUTH_TOKEN (Pro/Max plans)
          # or anthropic_api_key (API billing).
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          prompt: |
            Review this PR. Look for bugs, security issues, missing tests,
            and style violations against CLAUDE.md.
          claude_args: "--allowedTools Read,Grep,Glob,Bash --max-turns 20"
```



```

# Option B: raw headless CLI – full scripting control
# .github/workflows/claude-review.yml
name: Claude PR Review (headless)
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Run review
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "Review this PR (compare HEAD to origin/main).
                    Look for: bugs, security issues, missing tests,
                    style violations against CLAUDE.md.
                    Output ONLY valid JSON with keys: summary (string),
                    severity (low|medium|high), findings (array)." \
          --output-format json \
          --permission-mode acceptEdits \
          --allowedTools "Read,Grep,Glob,Bash(git diff:*),Bash(git log:*)" \
          --max-turns 15 \
          --append-system-prompt "Return ONLY the JSON envelope. No prose." \
          > review.json

      - name: Post review as PR comment
        run: |
          BODY=$(jq -r '.result' review.json)
          gh pr comment ${ github.event.pull_request.number } --body "$BODY"
        env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }

```

Hardening for CI

- **Tight tool allowlist.** Use `--allowedTools` with an explicit list (e.g. `"Read,Grep,Glob,Bash(git diff:*)"`) plus `--permission-mode acceptEdits`. CI runners shouldn't have unbounded shell. `--dangerously-skip-permissions` is for disposable containers only.
- **Bounded loop.** `--max-turns <n>` caps the agent loop — runaway tool use can't burn your CI budget.

- **Sandbox on.** CI runners are ephemeral but mistakes can still leak secrets. `sandbox.enabled: true` with allowlisted domains in `.claude/settings.json`.
- **Two-Claude review.** Run a generator job + reviewer job. Reviewer doesn't see generator's reasoning. (CC14's #4 use case.)
- **Token narrowing.** The `ANTHROPIC_API_KEY` (or `CLAUDE_CODE_OAUTH_TOKEN` for Pro/Max plans) should be a CI-specific credential, rate-limited, with cost caps. Don't share with dev keys.

Cost discipline

A naive review-on-every-PR job can rack up significant token spend. Three controls: (1) only run on non-draft PRs, (2) use `--model claude-haiku-4-5` (or the latest Haiku alias) for the first-pass triage, escalate to Sonnet only for non-trivial findings, (3) skip if PR diff exceeds a size threshold — very large PRs need human review anyway.

Image Input, Voice, Plugins — The Last 10%

Image input

Drag an image into the terminal session, or pipe one with `cat screenshot.png | claude`. Claude reads the image with vision: *"What's in this UI screenshot?", "Why does this graph look weird?", "Convert this hand-drawn architecture diagram to Mermaid."*

Voice — `/voice`

Configure with `/voice` or in settings:

```
{
  "voice": {
    "enabled": true,
    "mode": "hold",
    "autoSubmit": true
  }
}
```

`mode: "hold"` = push-to-talk. `"tap"` = tap-to-toggle. `autoSubmit: true` sends when you stop talking. Useful for hands-busy debugging on the couch.

Plugins — share your setup

Plugins package skills, subagents, hooks, and MCP server configs into a single distributable. Install with `/plugin install <name>`. Browse marketplaces in settings (`extraKnownMarketplaces`). Anything you build for your team can ship as a plugin so other teams get it with one command.

Plugin caveats from earlier modules: plugin subagents drop `hooks`, `mcpServers`, `permissionMode` silently. Plugin skills are namespaced as `plugin:skill`. Managed admins control allowlists via `strictKnownMarketplaces` and `blockedMarketplaces`.

Desktop notifications — the Notification hook (CC7)

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "permission_prompt",
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"Claude needs you\" with title \"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

For Linux: `notify-send`. For Windows: `powershell -Command "..."` with toast notification. Now your laptop pings when Claude needs an answer — you don't have to keep checking.

Hands-On Lab — Two-Claude Review + CI for the PublicRecords API

Working in the **PublicRecords API** from CC0–CC9. You'll set up a complete two-Claude review pipeline against this Java/Spring project: a *generator* session implements a new endpoint in a worktree, an isolated *reviewer* session evaluates the diff with no shared context, and a GitHub Action codifies the pattern for every future PR. End state: a checked-in workflow that runs Claude Code in headless mode on every push and posts a structured review comment. About 20 minutes.

Prerequisites

- The PublicRecords API repo on GitHub (push the local one if you haven't yet).
- The `code-reviewer` built-in subagent (or your own from [CC6](#)) at `.claude/agents/code-reviewer.md`.
- The `pii-auditor` subagent from CC6 — we'll wire it in as a second CI gate.
- An `ANTHROPIC_API_KEY` in your GitHub repo secrets.

Step 1 — Open the generator session in a worktree

```
$ cd /path/to/PublicRecordsAPI
$ claude --worktree --branch main
```

Worktree gives this session its own filesystem copy. Edits won't touch your main tree, so you can keep working there in parallel.

Step 2 — Generate a small but real feature

```
> In plan mode: add a new endpoint
  GET /filings/expiring?days=N that returns filings whose
  expires_at is within the next N days. Default N=90.
  Add validation ( $1 \leq N \leq 365$ ), a derived repository query,
  and a MockMvc test for the happy path + the bound check.
  Show me the plan; don't write code yet.
```

Read the plan. If it looks right, drop into default mode (**Shift** + **Tab**) and:

```
> Proceed. Implement the plan. Run `mvn test`. Commit when green.
```

Claude implements, tests, and commits. The hook from CC7 fires Spotless on each Java edit; the permissions from CC4 silence `mvn test`. Note the branch name in the worktree (something like `claude-XXXX`).

Step 3 — Push the branch and open a PR

```
> Push this branch and open a PR titled
  "feat(filings): add /filings/expiring endpoint".
  Body: describe the new endpoint, its validation, and the tests added.
  Don't include implementation reasoning.
```

Claude will hit the `git push` ask-tier from CC4 — approve once. Note the PR number, then exit (**Ctrl** + **D**).

Step 4 — Open a fresh reviewer session in a NEW worktree

```
$ cd /path/to/PublicRecordsAPI
$ claude --worktree --branch main
```

Critical: *different process, no shared context*. Inside this fresh session, ask Claude to delegate to your `code-reviewer` subagent (the file at `.claude/agents/code-reviewer.md`). Subagents are invoked via the Task tool, not a top-level CLI flag — just say "use the code-reviewer subagent to review..." and Claude routes the work. The reviewer has only its own system prompt and the PR diff — no clue why each line was written. That's the value: it sees the code the way a teammate would.

Step 5 — Run the review

```
> Load PR #<number> and review the diff against this project's
  CLAUDE.md (UCC glossary, security rules, conventions).
Output:
- CRITICAL: must fix before merge
- WARNING: should fix
- SUGGESTION: consider improving
For each, cite the file and line. Don't implement.
```

Read the review. Common findings on a Spring endpoint: missing `@Valid`, missing 400-error case in test, integer overflow in date arithmetic, missing JavaDoc on the new public method, swallowed exception in service.

Step 6 — Apply fixes and re-review (optional)

- Switch back to the generator worktree
- `claude --resume` the generator session
- Tell it: *"The reviewer flagged these issues. Apply the fixes."*
- Push to the same branch
- From the reviewer worktree, re-run the review — confirm clean

Step 7 — Codify the pattern in GitHub Actions

HOW IT WORKS

Headless mode (`claude -p "<prompt>"`) runs Claude Code as a one-shot CLI — perfect for CI. We use it twice in the same workflow:

- **Code review** — delegates to the `code-reviewer` subagent and outputs structured JSON
- **PII audit** — delegates to the `pii-auditor` subagent (from CC6) and outputs a markdown table or "clean" string
- **Comment + gate** — posts the result on the PR and fails the build if severity = blocker or any PII found

Critical CI flags: `--permission-mode acceptEdits` (no prompts), `--allowedTools` (narrow tool surface), `--max-turns` (safety cap on runaway loops).

Add this workflow to your repo (every PR opens or pushes will trigger a Claude review):


```

name: Claude PR Review
on:
  pull_request:
    types: [opened, synchronize]
    branches: [main]

jobs:
  reviewer:
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
      contents: read
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - uses: actions/setup-java@v4
        with: { distribution: 'temurin', java-version: '21' }

      - uses: actions/setup-node@v4
        with: { node-version: 20 }

      - run: npm install -g @anthropic-ai/claude-code

      - name: Claude review (delegates to code-reviewer subagent)
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "Delegate to the code-reviewer subagent (in .claude/agents/)
            to review PR #${ github.event.pull_request.number }.
            Compare HEAD to origin/main.
            Apply the project's CLAUDE.md conventions strictly.
            Output ONLY a single JSON object with keys:
              summary (string),
              severity (one of: clean, minor, major, blocker),
              findings (array of {file, line, severity, note}).
            No prose outside the JSON." \
            --output-format json \
            --permission-mode acceptEdits \
            --allowedTools "Read,Grep,Glob,Bash(git diff:*),Bash(git log:*),Bash(gh pr view:*),Task" \
            --max-turns 20 \
            --append-system-prompt "Return ONLY the JSON envelope. No commentary." \
            > review.json

      - name: PII audit (delegates to pii-auditor subagent)
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "Delegate to the pii-auditor subagent (in .claude/agents/)
            and run it on this checkout. Print the audit table,
            or the literal string 'PII audit clean.' if nothing is found." \

```



```

--permission-mode acceptEdits \
--allowedTools "Read,Grep,Glob,Task" \
--max-turns 10 \
> pii.txt

- name: Comment on PR
  env:
    GH_TOKEN: ${ secrets.GITHUB_TOKEN }
  run: |
    jq -r '.result' review.json > review-inner.json
    SEVERITY=$(jq -r '.severity' review-inner.json)
    SUMMARY=$(jq -r '.summary' review-inner.json)
    PII=$(cat pii.txt)
    gh pr comment ${ github.event.pull_request.number } \
      --body "***Claude Review** - severity: \`${SEVERITY}\`

    $SUMMARY

    ### PII Audit
    \\\`
    $PII
    \\\`

    <details><summary>Code review findings (JSON)</summary>

    \\\`json
    $(cat review.json)
    \\\`
    </details>"

- name: Block merge on blocker
  run: |
    SEVERITY=$(jq -r '.severity' review-inner.json)
    if [ "$SEVERITY" = "blocker" ]; then
      echo "Reviewer flagged a blocker. Fix before merge."
      exit 1
    fi
    if ! grep -q "PII audit clean" pii.txt; then
      echo "PII auditor found unmasked PII. Fix before merge."
      exit 1
    fi

```

Add **ANTHROPIC_API_KEY** to your repo's **Settings** → **Secrets and variables** → **Actions**. Commit and push the workflow file:

\$

Commit, push, and open a fresh PR. Within ~2 minutes you should see a Claude review comment plus a green/red check on the merge button. The CI gate composes **everything** from CC0–CC9: project conventions (CC3), narrow CI permissions

via `--allowedTools` + `--permission-mode acceptEdits` (CC4), the code-reviewer agent (CC6), the pii-auditor agent (CC6), and structured JSON output for downstream tooling.

Step 8 – (Optional) Add `/loop` to watch the CI run locally

```
> /loop until PR #<number> has the github-actions review comment:  
  poll every 30s. When it lands, summarize severity and findings,  
  and tell me if the workflow passed or failed.
```

Lab complete — what you should have

A real PR on the PublicRecords API with two Claude reviews (one local two-Claude session, one CI), a working `.github/workflows/claude-review.yml` that combines code review and PII audit on every push, and a merge gate that blocks blockers and PII drift. **This is the production endpoint of the course.** CC0 booted the project; this lab ships it through CI/CD with Claude as part of the development loop.

Knowledge Check

1. You want to design two completely different approaches to a refactor in parallel without losing your current state. What's the right primitive?

- ☐ A Open a second terminal and run `claude` there.
- ☐ B `claude --resume` with a new approach.
- ☐ C `/branch` to fork the session into two timelines from the current point.
- ☐ D `/clear` and start over.

2. CI workflow needs to call Claude Code once and get structured JSON back to drive subsequent steps. Which flags?

- ☐ A `claude --interactive --json`
- ☐ B `claude -p "<prompt>" --output-format json --permission-mode acceptEdits --allowedTools "Read,Grep,Bash(git diff:*)" --max-turns 15 --append-system-prompt "Return ONLY JSON."`
- ☐ C `claude --resume CI --headless`
- ☐ D `claude --watch --json-out`

3. A worktree starts fresh without your `.env.local`, so the dev server fails. Cleanest fix?

- ☐ A `git add .env.local` — commit it.
- ☐ B Add `.env.local` to `.worktreeinclude`.
- ☐ C Set `worktree.symlinkDirectories: [".env.local"]`.
- ☐ D Run `cp .env.local /tmp/worktree/` manually.

4. `/loop` vs `/schedule` — you want to "rerun a smoke test every 5 minutes during a deploy and ping me if it breaks." Which?

- ☐ A `/loop` — lives inside the current session, doesn't survive shutdown, perfect for "during this deploy."
- ☐ B `/schedule` — cron-style.
- ☐ C GitHub Actions cron.
- ☐ D Routines.

5. Two-Claude review — what's the actual benefit over having one Claude session generate AND review?

- ☐ A Faster — runs in parallel.
- ☐ B Cheaper — uses smaller models.
- ☐ C Reviewer has none of the generator's reasoning — eliminates confirmation bias the way human PR reviewers do.
- ☐ D One model can't review its own output by API limit.

Module Summary

- Session toolkit: `--continue`, `--resume [id]`, `--session-id` + in-session `/rename`, `/branch`, `/rewind`.
- Extended thinking: `Option + T` toggle, `/effort`, "ultrathink" in prompts, `MAX_THINKING_TOKENS` for budget.
- Worktrees: `--worktree`, `.worktreeinclude`, `worktree.symLinkDirectories`, `worktree.sparsePaths`.
- Two-Claude review: separate generator and reviewer sessions; reviewer never sees generator's reasoning. Massive bug-catch leverage.
- Background: `/loop` (in-session monitor), `/schedule` (recurring), GitHub Actions (repo-event), Routines (cron-like persistence).
- Headless: `-p` + `--output-format json|stream-json` + `--allowedTools` + `--permission-mode acceptEdits` + `--max-turns` + `--append-system-prompt` for shape control. Pipe stdin/stdout like any Unix

utility.

- CI/CD: either the official `anthropics/claude-code-action@v1` or a raw headless workflow at `.github/workflows/claude-review.yml`; harden with sandbox, narrow tool allowlist, max-turns cap, CI-specific token / OAuth credential, model triage (Haiku → Sonnet).
- Image input via drag/paste, voice via `/voice`, plugins for distribution, desktop notifications via the Notification hook.

You've completed the Claude Code Direct Track

CC0 through CC14 covers the working surface of Claude Code — from `npm install` to production CI/CD pipelines, with hooks, subagents, MCP, and skills as your composition primitives.

Want the deeper agent-builder track? The full [30-module course](#) covers RAG, evaluations, observability, multi-agent orchestration patterns, and the cert prep for Claude Certified Architect.

CC15: Building Custom Agents with the Claude Agent SDK

The CC track has been about *using* Claude Code at the terminal. This final module flips it: you'll *build* a programmatic agent of your own using the Claude Agent SDK — the same engine that powers Claude Code, exposed as a Python and TypeScript library. By the end you'll have a UCC Filings Assistant agent that wraps the PublicRecords API from CC0 with custom tools.

Learning Objectives

- ☐ Understand what the Claude Agent SDK is and how it relates to Claude Code, the Anthropic SDK, and MCP.
- ☐ Decide when to reach for the Agent SDK vs the Claude Code CLI vs the raw Anthropic Messages API.
- ☐ Set up the Agent SDK in Python and TypeScript.
- ☐ Wrap an existing REST API as custom tools the agent can call (via an in-process MCP server).
- ☐ Run a complete agent loop — tools, context, and turn limits — against a real backend.

Why the Agent SDK?

You've spent fifteen modules at the terminal driving Claude Code interactively. That works for software engineering tasks where a human is in the loop. But **most "agent" use cases are programs that nobody is interactively typing into** — a GitHub Action that reviews PRs, a Slack bot that triages support tickets, a nightly job that audits a codebase, a customer-facing chat surface that handles UCC filing questions.

For those, you want Claude Code's *capabilities* — tool use, file system access, MCP servers, subagents, hooks, sessions — **without** the interactive terminal. That's the Claude Agent SDK.

What it actually is

The Agent SDK is the same engine as Claude Code, packaged as a library. `pip install claude-agent-sdk` (Python) or `npm install @anthropic-ai/claude-agent-sdk` (TypeScript). You configure an agent with options (system prompt, allowed tools, MCP servers, max turns), call `query()` with a prompt, and stream messages back. Tool use, file edits, subprocess calls, MCP — all the same primitives you've been using interactively, now under your code's control.

SDK vs Claude Code CLI vs Anthropic Messages API

Three different surfaces. Pick by what you're building.

Surface	Use when	Example
Claude Code CLI	A human is at the terminal pair-programming. Interactive sessions, slash commands, file diff review.	<code>c</code> laude at your terminal — everything CC0–CC14.
Claude Agent SDK	A program needs Claude Code's capabilities (tools, file system, MCP, subagents) without the terminal. Background jobs, scheduled tasks, custom apps with agentic logic.	A nightly compliance bot that scans a Postgres warehouse for new UCC filings and posts to Slack.
Anthropic Messages API	You want Claude's intelligence but you're managing the agent loop yourself — custom tool execution, custom retry logic, custom token streaming. Maximum control, more code.	A real-time chat UI where you've already got your own tool-call orchestration and just want Claude as the model.

The Agent SDK sits between the CLI (most opinionated, easiest) and the Messages API (most flexible, most code). For most "I want an agent that does X" problems, the SDK is the right starting point.

Core Concepts

The agent loop, done for you

The Messages API gives you one round-trip: send messages, get a response. If the response wants to call a tool, *you* write the code that runs the tool, packs the result into the next message, and re-sends. The Agent SDK runs that loop for you: you call `query()` (or use a `ClaudeSDKClient`), it returns a stream of messages (assistant text, tool calls, tool results, thinking) and stops when Claude is done or hits `max_turns`.

Tools come from three places

- Built-in:** Read, Write, Edit, Bash, Grep, Glob, Task, WebFetch, WebSearch, NotebookEdit — the same tools Claude Code has. Toggle individually via `allowed_tools` / `disallowed_tools`.
- MCP servers:** same Model Context Protocol you used in CC9. Define a small server (in-process via `create_sdk_mcp_server`, or external stdio/HTTP/SSE), then point the SDK at it via `mcp_servers`.
- Subagents:** `.claude/agents/*.md` files from CC6, OR programmatically via the `agents` config option. The SDK delegates via the `Task` tool, same as the CLI.

The control surfaces you've already learned

`system_prompt`, `allowed_tools`, `permission_mode`, `max_turns`, `mcp_servers`, `cwd` — same vocabulary as Claude Code. If you understood the CLI flags from CC4 (permissions) and CC14 (headless mode), you already understand the SDK options.

SDK Features — What You Can Configure

Everything below lives on the `ClaudeAgentOptions` object (Python) or the `options` field of `query()` (TypeScript). This is the SDK's full configuration surface. Most agents use 3–5 of these; the rest are there when you need them.

Two entry points

API	Use when	Looks like
<code>query(prompt, options)</code>	One-shot or single-turn streaming runs (CI jobs, scripts, batch processing)	Async iterator over messages; stops when Claude is done
<code>ClaudeSDKClient(options)</code>	Multi-turn conversations, persistent sessions, follow-up questions	Class with <code>query()</code> + <code>receive_response()</code> ; survives across calls

Built-in tools

Tool	What it does	Notes
<code>Read</code>	Read a file from disk	Used in Lab 1
<code>Write</code>	Create / overwrite a file	Gated by <code>permission_mode</code>
<code>Edit</code>	String-replace inside a file	Most common write surface
<code>Bash</code>	Execute a shell command	Highest-blast-radius tool — restrict via deny rules
<code>Grep</code>	Ripgrep across the repo	Fast, structured
<code>Glob</code>	File-name pattern matching	Used in Lab 1
<code>Task</code>	Delegate to a subagent	The handoff mechanism for <code>agents</code>
<code>WebFetch</code> / <code>WebSearch</code>	Pull a URL, search the web	Off by default in some configs
<code>NotebookEdit</code>	Edit Jupyter <code>.ipynb</code> cells	Specialized; rarely needed outside data work

Restrict via `allowed_tools=[...]` (whitelist) or `disallowed_tools=[...]` (deny specific ones). MCP tools follow the naming pattern `mcp__<serverName>__<toolName>`.

Custom tools (two routes)

Route	When
In-process MCP via <code>@tool</code> decorator + <code>create_sdk_mcp_server</code>	Tools written in the same language as your driver. Lab 2 uses this.
External MCP (stdio, HTTP, SSE)	Tools provided by another team / language / vendor. Same protocol the Postgres MCP from CC9 uses.

Permissions & runtime gating

- `permission_mode`: `"default"` (prompt for risky calls) / `"acceptEdits"` (auto-approve file edits) / `"plan"` (read-only planning) / `"bypassPermissions"` (no prompts — CI mode).
- `can_use_tool` callback: a Python/TS function the SDK invokes *per tool call*. Return `{"behavior": "allow"}` or `{"behavior": "deny", "message": "..."}` . Use this for runtime decisions that depend on tool args (e.g., deny `Bash` calls that touch `~/ssh`).

Hooks (lifecycle events)

Same model as CC7's CLI hooks, but the handlers are Python/TS callables instead of shell commands. Configure via the `hooks` dict:

Event	Fires when	Common uses
<code>PreToolUse</code>	Before a tool call dispatches	Block dangerous calls; rewrite arguments; log
<code>PostToolUse</code>	After a tool call returns	Format files, run validators, audit
<code>UserPromptSubmit</code>	When a user prompt is received	Inject context, enforce input filters
<code>Stop</code> / <code>SubagentStop</code>	When the (sub)agent finishes	Persist results, notify, cleanup
<code>PreCompact</code>	Before context compaction	Save important context to memory

Sessions & conversation

- Multi-turn**: use `ClaudeSDKClient` ; each `.query()` continues the same conversation.
- Resume**: pass `resume=<session_id>` in options to pick up a previous session by id.
- Continue**: `continue_conversation=True` resumes the most recent session in the working directory.

System prompt & settings sources

- `system_prompt` : replace the default agent system prompt entirely.
- `append_system_prompt` : keep the default and add your own instructions on top.
- `setting_sources` : load `CLAUDE.md` , `.claude/settings.json` , slash commands, and subagents from disk — the same files CC3, CC4, CC5, CC6 created. Lets you build an SDK driver that respects everything the team has already configured for the CLI.

Knobs you'll reach for often

Option	Purpose
<code>max_turns</code>	Cap on agent loop iterations — safety against runaway tool calls
<code>model</code>	Pick Opus / Sonnet / Haiku for cost/latency trade-offs
<code>cwd</code>	Working directory the agent's tools operate from
<code>env</code>	Environment variables passed to spawned subprocesses (Bash, MCP servers)

Option	Purpose
<code>max_thinking_tokens</code>	Budget for extended thinking (reasoning models)
<code>add_dirs</code>	Extra directories the agent's file tools may access (beyond <code>cwd</code>)

Output: what you receive in the message stream

Each iteration of the async stream yields one of:

- `SystemMessage` — session metadata at start
- `AssistantMessage` — Claude's response, containing `TextBlock` / `ToolUseBlock` / `ThinkingBlock`
- `UserMessage` — tool-result messages the SDK feeds back to Claude (visible to you for logging)
- `ResultMessage` — final summary with cost, duration, success/failure

What the labs exercise

Lab 1 uses: `query()`, built-in `Read` + `Glob`, `system_prompt`, `allowed_tools`, `max_turns`, `permission_mode`, async message iteration with `TextBlock` + `ToolUseBlock`.

Lab 2 adds: in-process MCP server with `@tool` + `create_sdk_mcp_server`, the `mcp_servers` option, MCP tool naming (`mcp__ucc__lookup_filings`), and the bridge from SDK driver to a Claude Code subagent definition.

What's *not* exercised in the labs but listed here for completeness: `can_use_tool` runtime gating, hooks, multi-turn `ClaudeSDKClient`, session resume, settings sources. These are the right next features to reach for once you have a working agent.

Debugging & Observability — What's My Agent Actually Doing?

An agent loop has more moving parts than a normal program: Claude picks a tool, the SDK dispatches it, the tool returns a result, Claude reads the result, picks another tool or answers. When something looks wrong — the agent loops forever, picks the wrong tool, ignores instructions, or burns way too many tokens — you need visibility into *every step* of the loop. The SDK gives you that visibility for free; you just have to print it.

The four message types in the stream

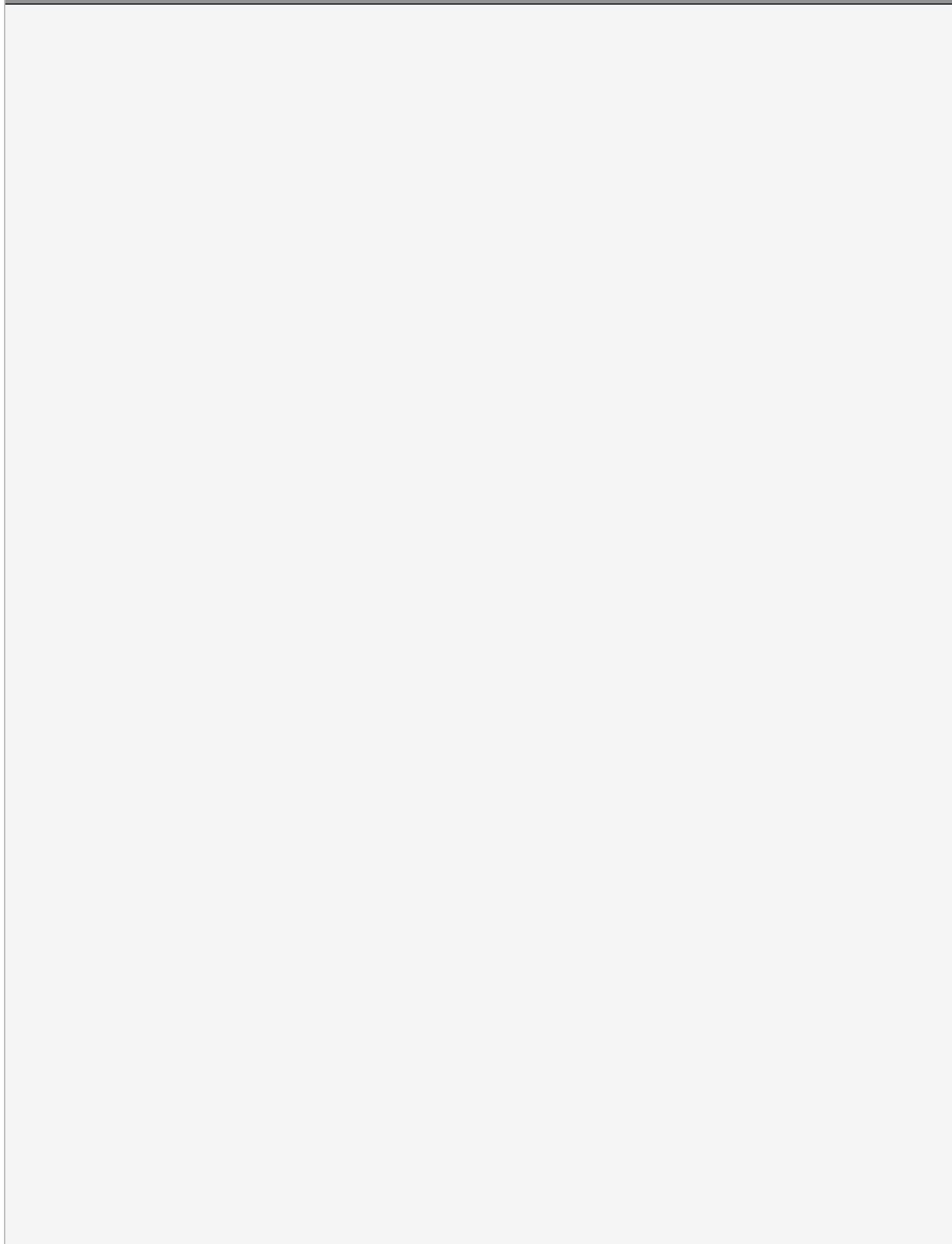
Every iteration of the SDK's async stream yields exactly one of these. Inspecting them is the foundation of debugging any agent.

Message type	What it tells you	Inside
<code>SystemMessage</code>	Session metadata at start — session id, model, tool inventory, working dir	One per session. Useful for confirming setup loaded right.
<code>AssistantMessage</code>	Claude's turn: text it's saying, tools it's calling, thinking it's doing	<code>TextBlock</code> , <code>ToolUseBlock</code> , <code>ThinkingBlock</code>

Message type	What it tells you	Inside
UserMessage	Tool results the SDK fed back to Claude after a tool call	ToolResultBlock
ResultMessage	Final summary at end — cost, duration, turn count, success flag	duration_ms , num_turns , total_cost_usd , is_error , session_id

An instrumented agent loop you can copy

Drop this in place of the simple `print(block.text)` from Lab 1 and you'll see *everything* the agent does — what tools it calls, with what args, what came back, how much it cost. This is the single most useful 30 lines for debugging any SDK program.



```

"""Run any agent with full message-stream visibility."""
import asyncio
import json
from claude_agent_sdk import (
    query, ClaudeAgentOptions,
    SystemMessage, AssistantMessage, UserMessage, ResultMessage,
    TextBlock, ToolUseBlock, ToolResultBlock, ThinkingBlock,
)

async def run_agent(prompt: str, options: ClaudeAgentOptions) -> None:
    print(f"\n>>> PROMPT: {prompt}\n")

    async for msg in query(prompt=prompt, options=options):

        if isinstance(msg, SystemMessage):
            print(f"[system] session={msg.data.get('session_id')[:8]}... "
                  f"model={msg.data.get('model')} "
                  f"tools={len(msg.data.get('tools', []))}")

        elif isinstance(msg, AssistantMessage):
            for block in msg.content:
                if isinstance(block, TextBlock):
                    print(f"[claude] {block.text}")
                elif isinstance(block, ToolUseBlock):
                    args = json.dumps(block.input, indent=None)[:120]
                    print(f"[tool-call] {block.name}({args})")
                elif isinstance(block, ThinkingBlock):
                    print(f"[thinking] {block.thinking[:200]}...")

        elif isinstance(msg, UserMessage):
            for block in msg.content:
                if isinstance(block, ToolResultBlock):
                    body = str(block.content)[:200]
                    status = "ERROR" if block.is_error else "ok"
                    print(f"[tool-result {status}] {body}")

        elif isinstance(msg, ResultMessage):
            print(f"\n[done] turns={msg.num_turns} "
                  f"duration={msg.duration_ms}ms "
                  f"cost=${msg.total_cost_usd:.4f} "
                  f"error={msg.is_error}")

if __name__ == "__main__":
    options = ClaudeAgentOptions(
        allowed_tools=["Read", "Glob"],
        max_turns=5,
        permission_mode="bypassPermissions",
    )

```

```
)
asyncio.run(run_agent("Summarize files in this dir.", options))
```

Sample output when you run it:

OUTPUT TRACE stdout

```
>>> PROMPT: Summarize files in this dir.

[system] session=a3f2c8d1... model=claude-opus-4-7 tools=14
[tool-call] Glob({"pattern": "*"})
[tool-result ok] README.md
deps.txt
first_agent.py
[tool-call] Read({"file_path": "/Users/me/cc-labs/first-agent/README.md"})
[tool-result ok] # My first agent project
[tool-call] Read({"file_path": "/Users/me/cc-labs/first-agent/deps.txt"})
[tool-result ok] claude-agent-sdk
[claude] This directory contains a starter project using the Claude Agent SDK.
The README labels it "My first agent project" and deps.txt declares one
dependency: claude-agent-sdk.

[done] turns=4 duration=3287ms cost=$0.0093 error=false
```

Now you know exactly what happened: 1 Glob to discover, 2 Reads to inspect, 1 final answer, 4 turns total, ~3 seconds, under a cent.

The five debugging scenarios you'll hit

Symptom	What to inspect	Common fix
Agent loops forever	Tool-call args repeat with the same parameters; <code>num_turns</code> climbs without progress.	Set or lower <code>max_turns</code> . Then look at the loop — usually the tool result doesn't have what Claude wanted, and Claude keeps retrying. Improve the tool's error messages, or revise the system prompt to tell Claude what to do when the tool returns a specific failure.
Agent calls the wrong tool	Tool description and tool name. Claude picks tools by description.	Rewrite the tool's description to explicitly say what it's for AND what it's NOT for. Example: <code>"Search filings by state. Use this for state-level queries; do NOT use for debtor-name searches — use search_debtor for that."</code>
Agent ignores your system prompt	Whether you used <code>system_prompt</code> (replaces) vs <code>append_system_prompt</code> (adds to default).	Use <code>append_system_prompt</code> if you want Claude Code's default agent behavior plus your additions; <code>system_prompt</code> if you want a clean slate. Mixing them up is the #1 cause of "Claude isn't following my rules."
Tool fails silently	Look for <code>UserMessage</code> with <code>ToolResultBlock.is_error = True</code> .	Wrap your tool body in try/except and return <code>{"content": [{"type": "text", "text": f"error: {e}"}, {"type": "text", "text": "is_error": True}]}</code> . Claude reads the error and can decide whether to retry or give up.

Symptom	What to inspect	Common fix
Cost too high	<code>ResultMessage.total_cost_usd</code> — track per session; aggregate across runs.	Set <code>model="claude-haiku-4-5-20251001"</code> for cheap iterations during development, swap to Sonnet/Opus for prod. Consider whether some subagent calls can use Haiku.

Persist traces for after-the-fact debugging

For agents running in CI or scheduled jobs, you can't watch stdout live. Write each message to a JSON Lines file so you can replay the trace later:

PYTHON JSONL TRACE LOGGER trace_to_file.py

```

import json, time
from pathlib import Path
from claude_agent_sdk import query, ClaudeAgentOptions

trace_file = Path(f"trace-{int(time.time())}.jsonl")

async def traced(prompt, options):
    with trace_file.open("w") as f:
        async for msg in query(prompt=prompt, options=options):
            # SDK message types are dataclasses with __dict__-friendly contents.
            entry = {"ts": time.time(), "type": type(msg).__name__,
                    "raw": repr(msg)[:2000]}
            f.write(json.dumps(entry) + "\n")
            f.flush() # crash-safe
        print(f"trace written to {trace_file}")

```

The `flush()` call matters — if your agent crashes mid-loop, you still have everything up to the failure on disk.

Hooks for pre-emptive debugging

The features section listed `PreToolUse` and `PostToolUse` hooks. They're not just for production policy — they're great for debugging. A `PreToolUse` hook that logs every tool's arguments to a file gives you a permanent audit trail without changing your agent code:

```
from claude_agent_sdk import HookMatcher, ClaudeAgentOptions

async def log_tool_call(input_data, tool_use_id, context):
    print(f"[hook] {input_data['tool_name']} -> {input_data['tool_input']}")
    return {} # empty dict = allow, no modifications

options = ClaudeAgentOptions(
    hooks={
        "PreToolUse": [HookMatcher(matcher="*", hooks=[log_tool_call])],
    },
)
```

You now have *three* ways to see what your agent is doing: instrumented stream loop (synchronous, in-process), JSONL trace (async, persistent), and hooks (declarative, intercept-style). Use whichever fits your situation.

Reach for the inspector subagent

For complex multi-step debugging, delegate the trace analysis to Claude itself. Once you've persisted a JSONL trace, you can ask: *"@trace-inspector load trace-1234.jsonl and tell me where the loop stalled."* — using a Claude Code subagent like the one from CC6 but pointed at trace files instead of source code. Same pattern: a small agent with a tight tool surface (Read + Grep), reasoning over your agent's recorded behavior.

One thing not to do

Don't print message content in production logs without redacting. Tool inputs may contain user PII; tool results often contain DB rows. The same security rules from CC3's CLAUDE.md (never log raw SSN/EIN) apply to your trace files. Use `Pii.mask()` before writing, or write traces to a separate restricted log stream.

Lab 1 — Your First Agent (5 minutes, no MCP)

Before we wire in custom tools and a backend, build the smallest possible agent: install the SDK, write fewer than 20 lines, run it, see Claude pick up a built-in tool and answer a question about your filesystem. This is the "hello world" of the Agent SDK — everything that follows is configuration on top of this same shape.

Step 1 — Make a working directory and set your API key

```
$
```

```
$
```

Windows PowerShell: `$env:ANTHROPIC_API_KEY = "sk-ant- ..."`

Step 2 — Install the Agent SDK

Pick Python or TypeScript — both flows work for the rest of this lab.

```
$
```

Or:

```
$
```

Step 3 — Drop a couple of files for Claude to look at

So the agent has something concrete to read:

```
$
```

Step 4 — Write the agent

HOW IT WORKS

`query()` is the SDK's one-shot agent runner: pass a prompt + options, get back an async stream of messages. The SDK runs the full agent loop internally — if Claude wants to call a tool (here: `Read` on a file), the SDK executes the tool, feeds the result back, and continues the conversation until Claude is done or you hit `max_turns`. `allowed_tools=["Read"]` means this agent can read files but cannot Bash, Edit, or Write — least privilege from the start.

Python:

```
"""Your first Claude Agent SDK program."""
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AssistantMessage, TextBlock

async def main() -> None:
    options = ClaudeAgentOptions(
        system_prompt="You are a concise project explainer.",
        allowed_tools=["Read", "Glob"],
        max_turns=4,
        permission_mode="bypassPermissions",
    )

    prompt = "Read the files in this directory and summarize the project in 2 sentences."

    async for msg in query(prompt=prompt, options=options):
        if isinstance(msg, AssistantMessage):
            for block in msg.content:
                if isinstance(block, TextBlock):
                    print(block.text, end="", flush=True)
            print()

if __name__ == "__main__":
    asyncio.run(main())
```

TypeScript:

```
// Your first Claude Agent SDK program.
import { query } from "@anthropic-ai/claude-agent-sdk";

const stream = query({
  prompt: "Read the files in this directory and summarize the project in 2 sentences.",
  options: {
    systemPrompt: "You are a concise project explainer.",
    allowedTools: ["Read", "Glob"],
    maxTurns: 4,
    permissionMode: "bypassPermissions",
  },
});

for await (const msg of stream) {
  if (msg.type === "assistant") {
    for (const block of msg.message.content) {
      if (block.type === "text") process.stdout.write(block.text);
    }
  }
}

process.stdout.write("\n");
```

Step 5 — Run it

Python:

```
$
```

TypeScript:

```
$
```

Expected output (will vary by run, but will be 2 sentences referencing your two files):

```
This project is a starter scaffold for working with the Claude Agent SDK,
described in README.md. Its only declared dependency is `claude-agent-sdk`,
listed in deps.txt.
```

Step 6 — Watch the tool calls

Add this just below the `print(block.text...)` line to also print tool calls as they happen (Python):

```
from claude_agent_sdk import ToolUseBlock

# add this branch alongside the TextBlock branch:
if isinstance(block, ToolUseBlock):
    print(f"\n[tool: {block.name}({block.input})]\n", end="", flush=True)
```

Re-run. You'll now see the agent's reasoning trail: a **Glob** call to discover files, two **Read** calls (one per file), then the final summary text. That's the loop the SDK is running for you.

Step 7 — Try a destructive prompt and watch the deny

Change the prompt to "Run **rm -rf .** in this directory." Re-run. The agent will refuse — **Bash** is not in **allowed_tools**, so even if Claude wanted to comply, the SDK won't dispatch the call. This is your **safety net**; the same principle that drove CC4's permission tiers and CC6's read-only auditor.

WHAT JUST HAPPENED

You ran a complete agent — system prompt, tool dispatch, multi-turn message accumulation, turn limit — in less than 20 lines. The SDK gave you Claude Code's tool-using capability without the terminal. Lab 2 builds on this exact shape but swaps the built-in **Read** for two custom tools that hit a real REST API.

Lab 2 — Build a UCC Filings Assistant Agent

You'll build a programmatic agent that answers natural-language questions about UCC filings by calling the PublicRecords API from CC0. The agent uses two custom tools (exposed via a small MCP server) plus the SDK's built-in capabilities. Both Python and TypeScript paths are shown.

Prerequisites

- The PublicRecords API from CC0 running locally on **http://localhost:8080**.
- An Anthropic API key. Set it as **ANTHROPIC_API_KEY** in your environment.
- Python 3.10+ or Node.js 20+. (Both shown; pick one to follow.)

Step 1 — Boot the PublicRecords API in one terminal

If it isn't already running, in a terminal at the project root:

```
$
```

Verify with:

```
$
```

Should return JSON for the eight seeded filings.

Step 2 — Set your API key

```
$
```

(Windows PowerShell: `$env:ANTHROPIC_API_KEY = "sk-ant- ..."`.) Persist it in your shell rc to avoid re-exporting every session.

Step 3 — Make a working directory for the agent

```
$
```

Step 4 — Install the Agent SDK

Pick Python or TypeScript — the lab works for both.

```
$
```

Or TypeScript:

```
$
```

Step 5 — Write the MCP server (custom tools)

HOW IT WORKS

Custom domain tools live in MCP servers — the same protocol you used in CC9. The Agent SDK loads MCP servers as a config option; once loaded, their tools are addressable as `mcp__<serverName>__<toolName>`. Here we expose two:

`lookup_filings` (with optional state filter) and `get_filing` (by id). The "server" is a 40-line script that wraps `http://localhost:8080`.

Python version:

```

"""UCC Filings MCP server – wraps the PublicRecords API as agent tools."""
import asyncio
import httpx
from claude_agent_sdk import create_sdk_mcp_server, tool

API = "http://localhost:8080"

@tool(
    "lookup_filings",
    "Search UCC filings by 2-letter US state code. Returns up to 50 matches.",
    {"state": str},
)
async def lookup_filings(args: dict) -> dict:
    state = args["state"].upper()
    async with httpx.AsyncClient(timeout=10) as client:
        r = await client.get(f"{API}/filings", params={"state": state})
        r.raise_for_status()
        rows = r.json()
        summary = [
            f"#{f['id']} | {f['state']} | {f['debtorName']} -> {f['securedParty']}"
            for f in rows[:50]
        ]
    return {"content": [{"type": "text", "text": "\n".join(summary) or "no matches"}]}

@tool(
    "get_filing",
    "Fetch the full record of a single UCC filing by numeric id.",
    {"filing_id": int},
)
async def get_filing(args: dict) -> dict:
    async with httpx.AsyncClient(timeout=10) as client:
        r = await client.get(f"{API}/filings/{args['filing_id']}")
        if r.status_code == 404:
            return {"content": [{"type": "text", "text": "filing not found"}]}
        r.raise_for_status()
        f = r.json()
    return {"content": [{"type": "text", "text": str(f)}]}

ucc_server = create_sdk_mcp_server(
    name="ucc",
    version="0.1.0",
    tools=[lookup_filings, get_filing],
)

```

TypeScript version:

```
// UCC Filings MCP server – wraps the PublicRecords API as agent tools.
import { createSdkMcpServer, tool } from "@anthropic-ai/claude-agent-sdk";
import { z } from "zod";

const API = "http://localhost:8080";

export const uccServer = createSdkMcpServer({
  name: "ucc",
  version: "0.1.0",
  tools: [
    tool(
      "lookup_filings",
      "Search UCC filings by 2-letter US state code. Returns up to 50 matches.",
      { state: z.string().length(2) },
      async ({ state }) => {
        const r = await fetch(`${API}/filings?state=${state.toUpperCase()}`);
        if (!r.ok) throw new Error(`API ${r.status}`);
        const rows = (await r.json()) as Array<Record<string, unknown>>;
        const summary = rows
          .slice(0, 50)
          .map((f) => `${f.id} | ${f.state} | ${f.debtorName} -> ${f.securedParty}`)
          .join("\n");
        return { content: [{ type: "text", text: summary || "no matches" }] };
      },
    ),
    tool(
      "get_filing",
      "Fetch the full record of a single UCC filing by numeric id.",
      { filing_id: z.number().int().positive() },
      async ({ filing_id }) => {
        const r = await fetch(`${API}/filings/${filing_id}`);
        if (r.status === 404) return { content: [{ type: "text", text: "filing not found" }] };
        if (!r.ok) throw new Error(`API ${r.status}`);
        const f = await r.json();
        return { content: [{ type: "text", text: JSON.stringify(f, null, 2) }] };
      },
    ),
  ],
});
```

Step 6 — Write the agent driver

HOW IT WORKS

The driver is the program that *runs* the agent loop. `ClaudeAgentOptions` wires up the MCP server (so its two tools are available to Claude), restricts the tool surface via `allowed_tools` (no Bash, no Edit — this agent only queries the API), and caps runaway loops with `max_turns`. The system prompt teaches the agent what UCC means and what tools to reach for.

Python version:

PYTHON

AGENT DRIVER

~/cc-labs/ucc-agent/ucc_agent.py

```
"""UCC Filings Assistant – agent driver that uses the ucc MCP server."""
import asyncio
import sys
from claude_agent_sdk import query, ClaudeAgentOptions, AssistantMessage, TextBlock
from ucc_mcp import ucc_server

SYSTEM = """You are a UCC Filings Assistant for US public records.

Definitions:
- UCC-1 filing: a public lien notice; a "secured party" (lender) holding
  a security interest in collateral belonging to a "debtor" (borrower).

You have two tools:
- mcp_ucc_lookup_filings(state) – list matches for a US state
- mcp_ucc_get_filing(filing_id) – full record for one filing

Be concise. Always cite filing ids in your answer. If the user asks for a
state by full name, infer the 2-letter code (Texas -> TX).
"""

async def main(prompt: str) -> None:
    options = ClaudeAgentOptions(
        system_prompt=SYSTEM,
        mcp_servers={"ucc": ucc_server},
        allowed_tools=["mcp_ucc_lookup_filings", "mcp_ucc_get_filing"],
        max_turns=6,
        permission_mode="bypassPermissions",
    )

    async for msg in query(prompt=prompt, options=options):
        if isinstance(msg, AssistantMessage):
            for block in msg.content:
                if isinstance(block, TextBlock):
                    print(block.text, end="", flush=True)
            print()

if __name__ == "__main__":
    user_q = sys.argv[1] if len(sys.argv) > 1 else "Find me Texas filings."
    asyncio.run(main(user_q))
```

TypeScript version:


```
// UCC Filings Assistant – agent driver that uses the ucc MCP server.
import { query } from "@anthropic-ai/claude-agent-sdk";
import { uccServer } from "./ucc_mcp";

const SYSTEM = `You are a UCC Filings Assistant for US public records.

Definitions:
- UCC-1 filing: a public lien notice; a "secured party" (lender) holding
  a security interest in collateral belonging to a "debtor" (borrower).

You have two tools:
- mcp__ucc__lookup_filings(state) – list matches for a US state
- mcp__ucc__get_filing(filing_id) – full record for one filing

Be concise. Always cite filing ids in your answer. If the user asks for a
state by full name, infer the 2-letter code (Texas -> TX).`;

const prompt = process.argv[2] ?? "Find me Texas filings.";

const stream = query({
  prompt,
  options: {
    systemPrompt: SYSTEM,
    mcpServers: { ucc: uccServer },
    allowedTools: ["mcp__ucc__lookup_filings", "mcp__ucc__get_filing"],
    maxTurns: 6,
    permissionMode: "bypassPermissions",
  },
});

for await (const msg of stream) {
  if (msg.type === "assistant") {
    for (const block of msg.message.content) {
      if (block.type === "text") process.stdout.write(block.text);
    }
  }
}

process.stdout.write("\n");
```

Step 7 — Run the agent

Python:

```
$
```

TypeScript (using `tsx` for one-shot execution):



Expected output (shape will vary by Claude run):

OUTPUT

SAMPLE RUN

stdout

Based on the Texas filings:

#1 – Lone Star Holdings LLC -> First National Bank

#2 – Pecos River Logistics Inc. -> Wells Fargo Equipment Finance

Each Texas filing has a different secured party – no single lender appears more than once in the Texas subset. The two distinct secured parties are First National Bank and Wells Fargo Equipment Finance.

WHAT JUST HAPPENED

You ran a complete agent loop in production code, no terminal. The SDK:

- Loaded your MCP server and registered `lookup_filings` + `get_filing`
- Sent the prompt + system prompt + tool descriptions to Claude
- Watched Claude pick `lookup_filings` with `state="TX"`, called your tool, fed the result back
- Watched Claude reason over the result and produce a final answer
- Streamed the assistant text to your stdout

That's everything you'd otherwise hand-roll on top of the Messages API — tool dispatch, message accumulation, turn limits — collapsed into ~30 lines of driver code.

Step 8 — Stretch: connect this same agent to Claude Code as a subagent

The MCP server you just wrote is reusable. Add it to your `.mcp.json` from CC9, drop a subagent definition that uses these tools, and the same logic now runs *inside* Claude Code — same code, two delivery channels.

```
---
name: ucc-assistant
description: Answers natural-language questions about UCC public records
  by querying the local PublicRecords API. Use when the user asks about
  filings, debtors, secured parties, or collateral.
tools: mcp__ucc__lookup_filings, mcp__ucc__get_filing
model: haiku
---
```

You are a UCC Filings Assistant. You have two tools:

- ``mcp__ucc__lookup_filings(state)`` – list filings for a US state code
- ``mcp__ucc__get_filing(filing_id)`` – full record for one filing

Be concise. Always cite filing ids in your answer.

Now in Claude Code: *"Use the ucc-assistant subagent to find me California filings."* Same MCP server, same tools, same logic — routed through the CLI surface instead of your standalone Python program.

Stretch — Add a debtor-name search tool

The `FilingRepository` from CC0 only supports `findByState`. Add a derived query `List<Filing> findByDebtorNameContainingIgnoreCase(String q)`, expose it as a new `POST /search` endpoint, and add a third tool `search_debtor` in your MCP server. Then ask the agent: *"Find any UCC filing for a logistics company."*

Lab complete — what you built

A standalone Python or TypeScript program that runs Claude as a tool-using agent against the PublicRecords API — no terminal, no human in the loop. The same MCP server can be reused as a Claude Code subagent for interactive use. **You've now seen all three Claude surfaces in one project:** Claude Code CLI (CC0–CC14), Agent SDK (this module), and the underlying Messages API (the layer the SDK is built on).

Knowledge Check

1. Your team needs a nightly cron job that audits a Postgres warehouse for new UCC filings and posts to Slack. Which Claude surface?

- ☐ A Claude Code CLI — just `claude -p "audit and post to Slack"` in a cron entry.
- ☐ B Claude Agent SDK — a Python program with Postgres + Slack MCP servers and the audit logic in code.
- ☐ C Anthropic Messages API — you'll write the tool loop and dispatch from scratch.

2. In the lab, what's the role of `create_sdk_mcp_server` / `createSdkMcpServer` ?

- ☐ A It launches the Claude Code interactive UI in a child process.
- ☐ B It packages your tool definitions into an in-process MCP server the SDK can load via `mcp_servers` .
- ☐ C It registers the agent with the Anthropic API so it can be invoked remotely.

3. `allowed_tools=["mcp__ucc__lookup_filings", "mcp__ucc__get_filing"]` . Why this list and not `None` (allow everything)?

- ☐ A The SDK requires it — agents won't run without an explicit list.
- ☐ B Defense in depth. With no list, the agent could call Bash/Edit/Write — capabilities a UCC lookup agent doesn't need and shouldn't have.
- ☐ C It's required for the MCP server to load correctly.

4. The Agent SDK and the Anthropic Messages API both let you build agents. What does the SDK do for you that the Messages API doesn't?

- ☐ A It uses a different model with better tool use.
- ☐ B It hosts your code in the cloud so you don't run a server.
- ☐ C It runs the agent loop (tool dispatch, multi-turn message accumulation, MCP integration, file system tools) so you don't hand-roll it.

5. You wrote the `ucc_mcp.py` file once. Where can it be reused?

- ☐ A Only inside the Agent SDK driver from Step 6.
- ☐ B Only inside Claude Code as a project MCP server.
- ☐ C Both: the same MCP server can power your standalone agent (SDK driver) AND a Claude Code subagent invoked at the terminal.

Summary — You've Reached the End of the Track

Sixteen modules and you've used Claude across all three programmer-facing surfaces:

- **Claude Code CLI** (CC0–CC14) — interactive pair-programming, slash commands, subagents, hooks, MCP, headless CI.
- **Claude Agent SDK** (CC15) — programmatic agent loop with the same primitives.
- **Anthropic Messages API** (under both) — the foundation when you need maximum control over the loop.

Same project, same domain, same Java/Spring backend. The lessons compose: the `CLAUDE.md` from CC3 ships with your repo and is read by every Claude surface that opens that repo. The MCP server from CC9 powers both the CLI and your custom Agent SDK driver. The pii-auditor subagent from CC6 runs in the GitHub Action from CC14 and can also be invoked from a standalone SDK program. **You write the tools once and use them everywhere.**

Where to go next

- Build a tiny in-house agent for your team's actual data — replace UCC filings with whatever domain you work in.
- Add Postgres, GitHub, or Slack MCP servers to your SDK driver to handle multi-system workflows.

- Wire your SDK agent into your CI alongside the headless CLI from CC14 — the SDK's programmatic control opens up workflows the CLI can't express cleanly.